



Vivekanand Education Society's

Institute of Technology

An Autonomous Institute Affiliated to University of Mumbai

Hashu Advani Memorial Complex, Collector Colony, Chembur East, Mumbai - 400074.

Department of Information Technology

CERTIFICATE

This is to certify that Vanshika Khatri of D15C semester VI, have successfully completed necessary experiments in the NITPC62 : Machine and Deep Learning Lab under my supervision in **VES Institute of Technology** during the academic year 2025-2026.

Subject Teacher

Name: Mrs. Kajal Joseph

Signature:

Head of Department

Name: Dr. Mrs. Shalu Chopra

Signature:

Lab Teacher

Name: Kajal Joseph / Swapnil Yadav

Signature:



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY**
Department of Information Technology
Academic Year: 2025-26

Lab Code	Lab Name	Credit
NITPC62	Machine and Deep Learning(Lab)	1

Course Outcomes: Student will be able	
NITPC62.1	Select appropriate machine learning models for real-world problems and evaluate their performance effectively.
NITPC62.2	Apply various supervised and unsupervised algorithms to solve complex, high-dimensional machine learning problems.
NITPC62.3	Identify and address key security considerations in machine learning systems, including anomaly detection challenges, real-world security applications, and adversarial vulnerabilities.
NITPC62.4	Analyze fundamental deep learning concepts and design multi-layered neural architectures for complex data processing tasks.
NITPC62.5	Implement and evaluate diverse deep learning networks, including CNNs and RNNs, for specialized computational applications.
NITPC62.6	Evaluate emerging trends and deploy deep learning solutions to address sophisticated, real-world industry application challenges.

NITPC62 Machine and Deep Learning (Lab)													
CO	PO										PSO		
	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PSO 1	PSO 2
NITPC62.1	3	3	2	3	3	–	–	–	2	2	3	3	2
NITPC62.2	3	3	3	3	3	–	–	–	2	2	3	3	2
NITPC62.3	2	3	2	3	3	2	3	–	2	–	3	3	2
NITPC62.4	3	3	3	3	3	–	–	–	2	2	3	3	2
NITPC62.5	3	3	3	3	3	–	–	–	2	2	3	3	3
NITPC62.6	3	3	3	3	3	2	–	2	2	3	3		



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY**
Department of Information Technology
Academic Year: 2025-26

NITPC62 Machine and Deep Learning (Lab)											
CO	Experiment										
	EXP 1	EXP 2	EXP 3	EXP 4	EXP 5	EXP 6	EXP 7	EXP 8	EXP 9	EXP 10	EXP 11
NITPC62.1	3	3	3	3	3	3	2	2	2	2	3
NITPC62.2	3	3	3	3	3	3	1	1	1	1	3
NITPC62.3	–	–	–	–	–	–	–	–	–	–	3
NITPC62.4	–	–	–	–	–	–	3	3	3	3	3
NITPC62.5	–	–	–	–	–	–	3	3	3	3	3
NITPC62.6	1	1	1	1	1	1	3	3	3	3	3

INDEX

Sr. No.	List of Experiments
1	Implement Linear and Logistic Regression on real-world datasets
2	Implement Multi Regression, Lasso, and Ridge Regression on real-world datasets
3	Apply Decision Tree and Random Forest for classification tasks
4	Implement K-Nearest Neighbors (KNN) and evaluate model performance
5	Implement Support Vector Machine (SVM) for classification with hyperparameter tuning
6	Apply K-Means and Hierarchical Clustering on sample datasets
7	Build an Artificial Neural Network (ANN) using Keras/TensorFlow
8	Design and train a Convolutional Neural Network (CNN) on image datasets (e.g., MNIST/CIFAR)
9	Implement Recurrent Neural Network (RNN) / LSTM for time series or text data
10	Explore and implement Autoencoders for image denoising
11	Mini Project: Apply machine/deep learning on a domain-specific dataset



Vivekanand Education Society's

Institute of Technology

An Autonomous Institute Affiliated to University of Mumbai

Hashu Advani Memorial Complex, Collector Colony, Chembur East, Mumbai - 400074

Department of Information Technology

NITPC62: Machine and Deep Learning (Lab)

Experiment No. 01

Aim: Implement Linear and Logistic Regression on real-world datasets.

Roll No.	30
Name	Vanshika Khatri
Class	D15C
Subject	NITPC62: Machine and Deep Learning (Lab)
CO Mapped	CO1: Select appropriate machine learning models for real-world problems and evaluate their performance effectively. CO2: Apply various supervised and unsupervised algorithms to solve complex, high-dimensional machine learning problems. CO6: Evaluate emerging trends and deploy deep learning solutions to address sophisticated, real-world industry application challenges.

Name: Vanshika Khatri

Class: D15C

Roll No.: 30

EXPERIMENT – 1

Aim

To study and implement Linear Regression and Logistic Regression algorithms on a real-world dataset and analyze their performance for prediction and classification tasks.

Dataset Source

<https://www.kaggle.com/datasets/mirichoi0218/insurance>

Dataset Description

The Medical Cost Personal Dataset is a practical healthcare dataset frequently used in machine learning to analyze cost prediction and classification problems related to insurance.

This dataset includes demographic and medical information of individuals along with the corresponding insurance charges. Since it contains both a continuous dependent variable and a binary categorical variable, it is suitable for applying both regression and classification algorithms.

- Total number of records: 1338
- Data format: CSV
- Missing values: None

Dataset Attributes:

Variable Name	Data Type	Measuring Unit / Format	Description
Age	Numerical (Integer)	Years	Age of the individual insured under the policy.
Sex	Categorical (Binary)	Male / Female	Biological gender of the individual.
BMI	Numerical (Float)	kg/m ²	Body Mass Index, representing body fat based on height and weight.
Children	Numerical (Integer)	Count	Number of dependent children covered by the insurance policy.
Smoker	Categorical (Binary)	Yes / No	Indicates whether the individual is a smoker.
Region	Categorical (Nominal)	Northeast / Northwest / Southeast / Southwest	Residential region of the individual in the United States.
Charges	Numerical (Float)	USD (\$)	Medical insurance cost charged to the individual.

Target Variables Used

- **Charges:** Used as the output variable in Linear Regression for predicting medical expenses
 - **Smoker:** Used as the output variable in Logistic Regression for classifying individuals into smoker or non-smoker categories
-

Mathematical Formulation of Algorithms

Linear Regression

Linear Regression is a supervised learning method used to estimate a continuous dependent variable based on one or more independent variables by fitting a straight-line relationship.

In the case of a single independent variable, the model is represented as:

$$y = \beta_0 + \beta_1 x$$

Where:

- y represents the predicted output (insurance charges)
- x represents the input feature
- β_0 is the intercept
- β_1 is the coefficient indicating the rate of change

The optimal parameters are computed using the **Ordinary Least Squares (OLS)** technique, which minimizes the total squared difference between actual and predicted values.

This approach ensures the best-fit line that reduces prediction error.

Logistic Regression

Logistic Regression is a classification technique used when the dependent variable is categorical, especially binary.

Unlike Linear Regression, it does not directly predict output values but instead estimates the probability that a given input belongs to a specific class.

A linear equation alone is not suitable for classification because:

1. It can generate values outside the range of 0 and 1
2. It cannot properly model class boundaries

To overcome this, Logistic Regression applies a **sigmoid transformation** to the linear equation:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Where:

$$z = \beta_0 + \beta_1 x$$

The sigmoid function converts any real-valued input into a probability between 0 and 1.

Key properties:

- Large positive values of (z) give output close to 1
- Large negative values of (z) give output close to 0
- When $(z = 0)$, output = 0.5 (decision boundary)

This S-shaped curve helps in separating two classes effectively.

Algorithm Limitations

Limitations of Linear Regression

1. Linearity Assumption:

It assumes a straight-line relationship between variables. If the actual relationship is non-linear, the model may fail to capture patterns correctly.

2. Impact of Outliers:

Since the model minimizes squared errors, extreme values can significantly influence the regression line.

3. Multicollinearity Issue:

Highly correlated input features make it difficult to determine individual feature contributions accurately.

4. Constant Error Variance Assumption:

The model assumes equal spread of errors across all input values, which may not always hold true.

Limitations of Logistic Regression

1. Linear Decision Boundary:

It can only create linear separation between classes, which is ineffective for complex patterns.

2. Outlier Sensitivity:

Extreme values can distort probability estimation and shift decision boundaries.

3. Dependency on Dataset Size:

Smaller datasets may lead to unstable or biased probability predictions.

Methodology / Workflow

1. Data Collection

The dataset is imported from a CSV file into a Pandas DataFrame for processing.

2. Data Preprocessing

- o Categorical variables such as sex, smoker status, and region are converted into numerical values
- o Dataset is checked for missing values (none found)

3. Feature and Target Selection

- o Linear Regression: Predict charges using selected features
- o Logistic Regression: Predict smoker status

4. Splitting the Dataset

The dataset is divided into training and testing sets in an 80:20 ratio to evaluate model performance on unseen data.

5. Model Training

- o Linear Regression learns coefficients that minimize prediction error
- o Logistic Regression learns probabilities using the sigmoid function

6. Prediction

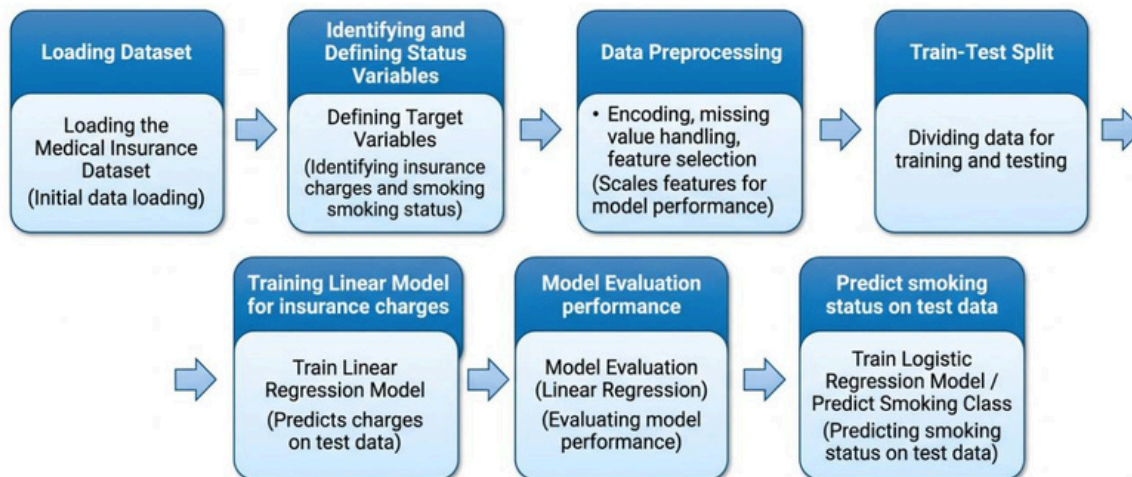
- o Linear model predicts insurance charges
- o Logistic model predicts class labels

7. Model Evaluation

- o Linear Regression evaluated using MSE, RMSE, and R^2 score

- o Logistic Regression evaluated using accuracy, confusion matrix, and classification metrics

Medical Insurance Prediction Workflow



CODE:

```

# =====
# STEP 1: IMPORT REQUIRED LIBRARIES
# =====

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import LabelEncoder
from sklearn.linear_model import LinearRegression, LogisticRegression

from sklearn.metrics import (
    mean_squared_error, r2_score,
    accuracy_score, confusion_matrix, classification_report
)

#=====
#STEP 2: LOAD DATASET
#=====

data = pd.read_csv("insurance.csv")
  
```

```
#=====
```

```
#STEP 3: DATA PREPROCESSING
```

```
#=====
```

```
encoder = LabelEncoder()
```

```
data['sex'] = encoder.fit_transform(data['sex'])
```

```
data['smoker'] = encoder.fit_transform(data['smoker'])
```

```
data['region'] = encoder.fit_transform(data['region'])
```

```
#=====
```

```
#LINEAR REGRESSION MODEL
```

```
#=====
```

```
#Selecting features and target
```

```
X_lr = data[['age', 'bmi', 'children', 'smoker', 'region']]
```

```
y_lr = data['charges']
```

```
#Splitting dataset
```

```
X_train_lr, X_test_lr, y_train_lr, y_test_lr = train_test_split(
```

```
    X_lr, y_lr, test_size=0.2, random_state=42
```

```
)
```

```
#Model creation and training
```

```
lr_model = LinearRegression()
```

```
lr_model.fit(X_train_lr, y_train_lr)
```

```
#Prediction
```

```
pred_lr = lr_model.predict(X_test_lr)
```

```
#Evaluation metrics
```

```
mse_val = mean_squared_error(y_test_lr, pred_lr)
```

```
rmse_val = np.sqrt(mse_val)
```

```
r2_val = r2_score(y_test_lr, pred_lr)
```

```
print("\n===== LINEAR REGRESSION OUTPUT =====")
print("MSE :", mse_val)
print("RMSE:", rmse_val)
print("R2 Score:", r2_val)
```

```
#Residual Plot
```

```
errors = y_test_lr - pred_lr
```

```
plt.figure()
plt.scatter(pred_lr, errors)
plt.axhline(y=0)
plt.xlabel("Predicted Values")
plt.ylabel("Residual Errors")
plt.title("Residual Plot for Linear Regression")
plt.show()
```

```
#=====
```

```
#LOGISTIC REGRESSION (WITHOUT TUNING)
```

```
#=====
```

```
#Feature and target
```

```
X_log = data[['age', 'bmi', 'children', 'region', 'charges']]
```

```
y_log = data['smoker']
```

```
#Splitting dataset
```

```
X_train_log, X_test_log, y_train_log, y_test_log = train_test_split(
    X_log, y_log, test_size=0.2, random_state=42
)
```

```
#Model training
```

```
log_model = LogisticRegression(max_iter=1000)
```

```
log_model.fit(X_train_log, y_train_log)
```

```

# Prediction

pred_log = log_model.predict(X_test_log)


#Evaluation

acc = accuracy_score(y_test_log, pred_log)

conf_mat = confusion_matrix(y_test_log, pred_log)

class_rep = classification_report(y_test_log, pred_log)


print("\n===== LOGISTIC REGRESSION (NO TUNING) =====")

print("Accuracy:", acc)

print("\nConfusion Matrix:\n", conf_mat)

print("\nClassification Report:\n", class_rep)


#Confusion Matrix Plot

plt.figure()

plt.imshow(conf_mat)

plt.title("Confusion Matrix (Without Tuning)")

plt.colorbar()


plt.xticks([0, 1], ["Non-Smoker", "Smoker"])

plt.yticks([0, 1], ["Non-Smoker", "Smoker"])


foriinrange(2):
    forjin range(2):
        plt.text(j, i, conf_mat[i, j], ha="center", va="center")

plt.xlabel("Predicted")

plt.ylabel("Actual")

plt.show()


#=====

#LOGISTIC REGRESSION (WITH TUNING)

#=====

```

```

# Hyperparameter setup
parameters = {
    'C': [0.001, 0.01, 0.1, 1, 10, 100],
    'penalty': ['l1', 'l2'],
    'solver': ['liblinear']
}

#Grid Search
grid_search = GridSearchCV(
    LogisticRegression(max_iter=1000),
    parameters,
    cv=5,
    scoring='accuracy'
)

#Training with best parameters
grid_search.fit(X_train_log, y_train_log)

#Best model
best_model = grid_search.best_estimator_

#Prediction
pred_tuned = best_model.predict(X_test_log)

#Evaluation
acc_tuned = accuracy_score(y_test_log, pred_tuned)
cm_tuned = confusion_matrix(y_test_log, pred_tuned)
rep_tuned = classification_report(y_test_log, pred_tuned)

print("\n===== LOGISTIC REGRESSION (WITH TUNING) =====")
print("Best Parameters:", grid_search.best_params_)
print("Accuracy:", acc_tuned)
print("\nConfusion Matrix:\n", cm_tuned)
print("\nClassification Report:\n", rep_tuned)

```



```

#Confusion Matrix Plot (Tuned)

plt.figure()

plt.imshow(cm_tuned)

plt.title("Confusion Matrix (With Tuning)")

plt.colorbar()


plt.xticks([0, 1], ["Non-Smoker", "Smoker"])

plt.yticks([0, 1], ["Non-Smoker", "Smoker"])


foriinrange(2):
    forjin range(2):
        plt.text(j, i, cm_tuned[i, j], ha="center", va="center")


plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

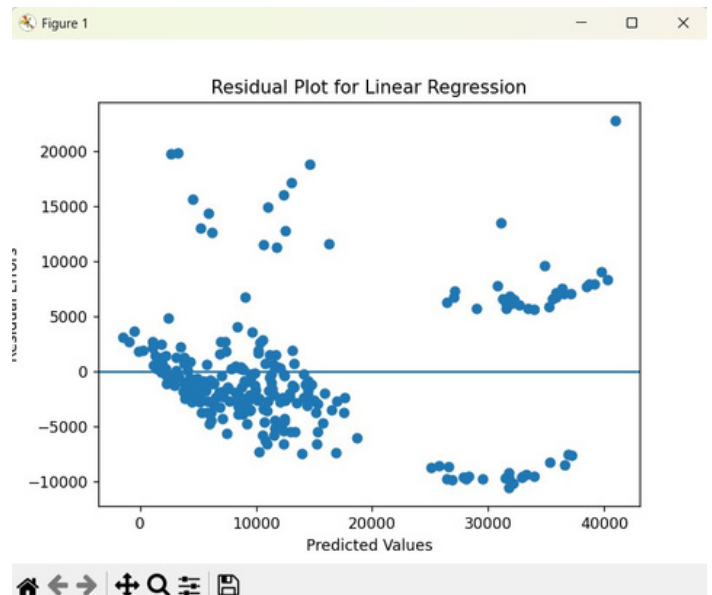
```

OUTPUT:

```

>>>
===== RESTART: C:\Users\vrkha
===== LINEAR REGRESSION OUTPUT =====
MSE : 33640657.13645164
RMSE: 5800.056649417455
R2 Score: 0.7833112270019789

```



===== LOGISTIC REGRESSION (NO TUNING) =====

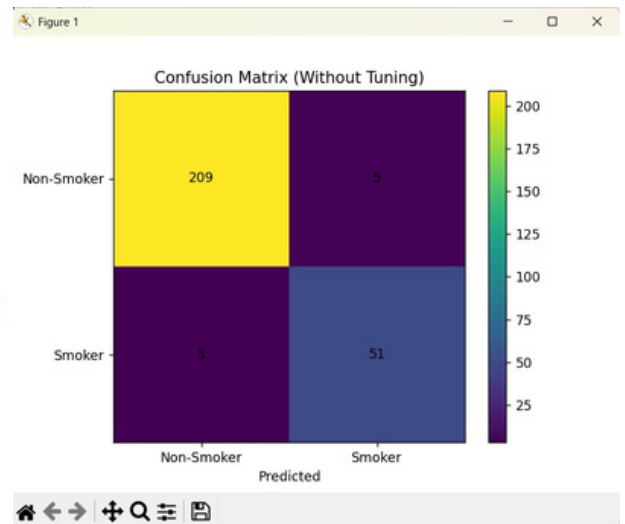
Accuracy: 0.9701492537313433

Confusion Matrix:

```
[[209  5]
 [  3 51]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.99	0.98	0.98	214
1	0.91	0.94	0.93	54
accuracy			0.97	268
macro avg	0.95	0.96	0.95	268
weighted avg	0.97	0.97	0.97	268



===== LOGISTIC REGRESSION (WITH TUNING) =====

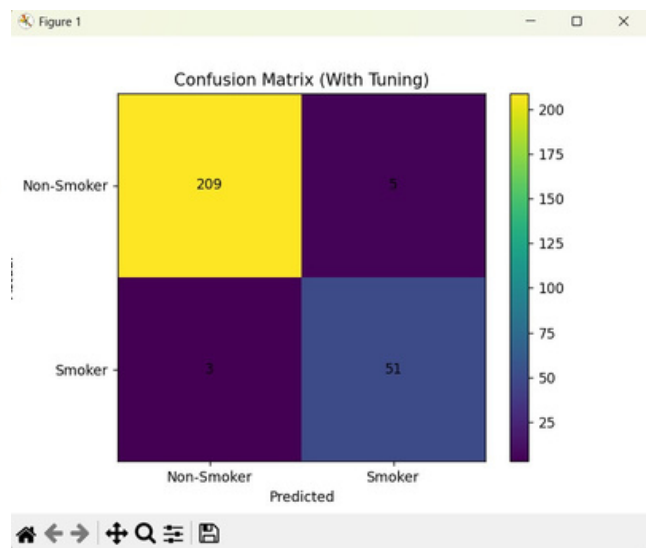
Best Parameters: {'C': 10, 'penalty': 'l1', 'solver': 'liblinear'}
Accuracy: 0.9701492537313433

Confusion Matrix:

```
[[209  5]
 [  3 51]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.99	0.98	0.98	214
1	0.91	0.94	0.93	54
accuracy			0.97	268
macro avg	0.95	0.96	0.95	268
weighted avg	0.97	0.97	0.97	268



Performance Analysis

Linear Regression

The model achieved an R^2 score of **0.7833**, meaning that around 78.33% of the variation in insurance charges is explained by the input features.

The Mean Squared Error (MSE) is **33,640,657.14**, and the RMSE is approximately **5800.06**, indicating the average prediction error.

This shows that the model performs reasonably well, considering the wide variation in insurance costs.

Logistic Regression (Without Hyperparameter Tuning)

The model achieved an accuracy of **97.01%**, correctly classifying most instances.

Confusion Matrix:

- 209 correctly identified non-smokers
- 51 correctly identified smokers

- 5 incorrect smoker predictions
- 3 missed smoker cases

The classification report shows strong precision, recall, and F1-scores, indicating reliable performance.

Logistic Regression (With Hyperparameter Tuning)

After applying GridSearchCV, the best parameters obtained were:

- $C = 10$
- Penalty = l1
- Solver = liblinear

The model achieved the same accuracy (**97.01%**) as before, indicating that the initial configuration was already near optimal.

Conclusion

This experiment successfully demonstrated the implementation of both regression and classification techniques on a real-world dataset.

- Linear Regression effectively modeled the relationship between input features and insurance charges
- Logistic Regression provided accurate classification of smoking behavior

The results highlight the importance of selecting appropriate algorithms based on the type of output variable and show that both models can perform efficiently when applied correctly.



Vivekanand Education Society's

Institute of Technology

An Autonomous Institute Affiliated to University of Mumbai

Hashu Advani Memorial Complex, Collector Colony, Chembur East, Mumbai - 400074.

Department of Information Technology

NITPC62: Machine and Deep Learning (Lab)

Experiment 02

Aim: Implement Multi Regression, Lasso, and Ridge Regression on real-world datasets.

Roll No.	30
Name	Vanshika Khatri
Class	D15C
Subject	NITPC62: Machine and Deep Learning (Lab)
CO Mapped	CO1: Select appropriate machine learning models for real-world problems and evaluate their performance effectively. CO2: Apply various supervised and unsupervised algorithms to solve complex, high-dimensional machine learning problems. CO6: Evaluate emerging trends and deploy deep learning solutions to address sophisticated, real-world industry application challenges.

Name: Vanshika Khatri
Class: D15C
Roll No.: 30

EXPERIMENT – 2

Aim

To implement and analyze **Multiple Linear Regression, Ridge Regression, and Lasso Regression** on a real-world dataset and evaluate their effectiveness in predicting housing prices.

Dataset Source

<https://www.kaggle.com/datasets/vedavyasv/usa-housing>

Dataset Description

The USA Housing Dataset is a real-world dataset that provides information about housing prices across different regions of the United States. It is widely used for analyzing how economic and housing-related factors influence property pricing.

The dataset contains approximately **5000 entries**, where each row corresponds to a specific housing instance. The data is available in **CSV format** and is organized in a structured tabular form. Since most of the variables are numerical, the dataset is well-suited for regression algorithms and requires minimal preprocessing.

Dataset Attributes:

Variable Name	Data Type	Measuring Unit	Description
Avg. Area Income	Numerical (Float)	Currency	Average income of residents in the area
Avg. Area House Age	Numerical (Float)	Years	Average age of houses in the area
Avg. Area Number of Rooms	Numerical (Float)	Count	Average number of rooms per house
Avg. Area Number of Bedrooms	Numerical (Float)	Count	Average number of bedrooms per house
Area Population	Numerical (Float)	Count	Population of the area
Price	Numerical (Float)	Currency	Price of the house

Target Variable

- **Price:** A continuous numerical variable representing the cost of a house. This variable is used as the dependent variable for Multiple Linear Regression, Ridge Regression, and Lasso Regression models.
-

Theory

Multiple Linear Regression

Multiple Linear Regression is an extension of Simple Linear Regression and is used to model the relationship between a dependent variable and multiple independent variables simultaneously.

In real-world scenarios like housing price prediction, relying on a single feature is insufficient. House prices are influenced by several factors such as income levels, population, number of rooms, and property age. Multiple Linear Regression allows all these variables to contribute collectively to the prediction.

This results in a model that represents a multidimensional relationship between features and the target variable.

Mathematical Formulation

For a single variable:

$$y = \beta_0 + \beta_1 x$$

For multiple variables:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n + \varepsilon$$

Where:

- y is the dependent variable (house price)
- $x_1, x_2, \dots, x_n$ are the independent variables (housing features)
- β_0 is the intercept, representing the predicted value of y when all x values are zero
- $\beta_1, \beta_2, \dots, \beta_n$ are regression coefficients that represent the effect of each feature on the house price
- ε is the error term, representing unexplained variation

Assumptions of Multiple Linear Regression

- Linear relationship between predictors and target
 - No strong correlation among independent variables
 - Constant variance of residuals
 - Residuals follow a normal distribution
-

Algorithm Limitations

Multicollinearity

When independent variables are highly correlated, the model struggles to estimate their individual effects.

Failure Condition: Leads to unstable coefficients and reduces interpretability.

Overfitting

Including too many features can make the model overly complex.

Failure Condition: The model may perform well on training data but poorly on unseen data.

Lack of Interaction Modeling

The model assumes each feature independently contributes to the output.

Failure Condition: Cannot capture relationships where features influence each other.

Methodology / Workflow

Data Collection

The dataset is imported into a Pandas DataFrame for analysis and processing.

Data Preprocessing

- The non-numeric **Address** column is removed
- Remaining features are verified
- Feature scaling is applied to normalize data

Train-Test Split

The dataset is divided into training (80%) and testing (20%) subsets.

Model Training and Prediction

A Multiple Linear Regression model is trained using the training data and then used to generate predictions for the test set.

Model Evaluation

The model is evaluated using:

- Root Mean Squared Error (RMSE)
- R^2 Score

USA Housing Price Prediction Model Building Process



Code:

```
# =====  
  
# STEP 1: IMPORT LIBRARIES  
  
# =====  
  
import pandas as pd  
import numpy as np  
import matplotlib.pyplot as plt  
import seaborn as sns  
  
  
from sklearn.model_selection import train_test_split  
from sklearn.linear_model import LinearRegression  
from sklearn.preprocessing import StandardScaler  
from sklearn.metrics import mean_squared_error, r2_score  
  
  
# =====  
  
#STEP 2: LOAD DATASET  
  
# =====  
  
data = pd.read_csv("USA_Housing.csv")  
  
  
# =====  
  
#STEP 3: DEFINE FEATURES & TARGET  
  
# =====
```



```
features = data.drop(['Price', 'Address'], axis=1)
target = data['Price']

#=====

#STEP 4: SPLIT DATA
#=====

X_train, X_test, y_train, y_test = train_test_split(
    features, target, test_size=0.2, random_state=42
)

#=====

#STEP 5: FEATURE SCALING
#=====

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

#=====

#STEP 6: TRAIN MODEL
#=====

model = LinearRegression()
model.fit(X_train_scaled, y_train)

#=====

#STEP 7: MAKE PREDICTIONS
#=====

predictions = model.predict(X_test_scaled)

#=====

#STEP 8: EVALUATE MODEL
```

```

# =====

rmse_value = np.sqrt(mean_squared_error(y_test, predictions))
r2_value = r2_score(y_test, predictions)

print("===== MULTIPLE LINEAR REGRESSION OUTPUT =====")
print("RMSE:", rmse_value)
print("R2 Score:", r2_value)

#=====

#STEP 9: VISUALIZATION

#=====

plt.figure(figsize=(8, 6))

sns.scatterplot(x=y_test, y=predictions)

#Ideal line

plt.plot(
    [y_test.min(), y_test.max()],
    [y_test.min(), y_test.max()],
    linestyle='--'
)

plt.xlabel("Actual House Prices")
plt.ylabel("Predicted House Prices")
plt.title("Actual vs Predicted Prices (Multiple Linear Regression)")

plt.show()

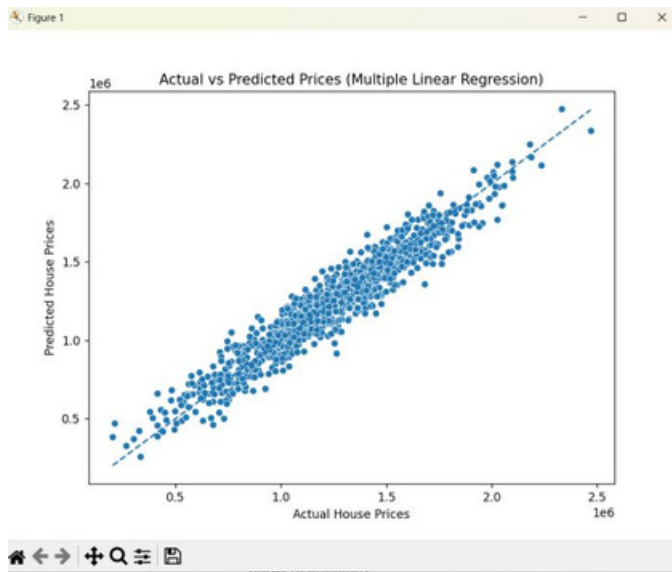
```

OUTPUT:

```

===== RESTART: C:\Users\vrkha\Desktop
===== MULTIPLE LINEAR REGRESSION OUTPUT =====
RMSE: 100444.06055558482
R2 Score: 0.9179971706834331
|

```



Performance Analysis

- **RMSE:** Indicates the average deviation between predicted and actual values
- **R^2 Score:** Represents how well the model explains variance in the data

The obtained results show that the model effectively captures key relationships between input variables and housing prices, making it a strong baseline model.

Hyperparameter Tuning

Multiple Linear Regression does not involve tunable hyperparameters, as it computes coefficients directly using the Ordinary Least Squares method.

To overcome limitations such as overfitting and multicollinearity, regularized models like Ridge and Lasso are introduced.

RIDGE REGRESSION

Theory

Ridge Regression is a variation of Multiple Linear Regression that introduces a penalty term to reduce model complexity and improve generalization.

In datasets where features are correlated, such as housing data, standard regression may produce unstable coefficients. Ridge Regression addresses this by shrinking coefficient values, resulting in a more stable model.

Mathematical Formulation

$$Cost = \underbrace{\sum_{i=1}^n (y_i - \hat{y}_i)^2}_{\text{Error (RSS)}} + \lambda \underbrace{\sum_{j=1}^p \beta_j^2}_{\text{L2 Penalty}}$$

Where:

- y_i is the actual house price
- $\hat{y}_i$ is the predicted house price
- β_j are the regression coefficients
- λ (alpha) is the regularization

The first term represents the sum of squared errors, while the second term penalizes large coefficient values. As λ increases, the penalty becomes stronger, forcing coefficients to shrink toward zero. Unlike Lasso Regression, Ridge Regression does not eliminate features entirely; instead, it reduces their influence smoothly.

Assumptions

- Linear relationship between variables
- Features must be scaled
- Handles correlated features effectively

Limitations

No Feature Elimination

All features remain in the model, though their influence is reduced.

Dependence on λ

Choosing an incorrect value can lead to underfitting or overfitting.

Scaling Requirement

Unscaled features may distort regularization effects.

Methodology/Workflow:

Data Collection:

The USA Housing dataset is imported into the working environment, which includes various numerical features related to housing along with the target variable, Price.

Data Preprocessing:

The Address column is removed since it contains textual (non-numeric) information that cannot be directly used for model training. The remaining features are already numerical. To ensure that all features contribute equally during model training, feature scaling is performed using StandardScaler.

Train–Test Split:

The dataset is partitioned into two subsets: 80% of the data is used for training the model, while the remaining 20% is reserved for testing its performance on unseen data.

Baseline Model Training:

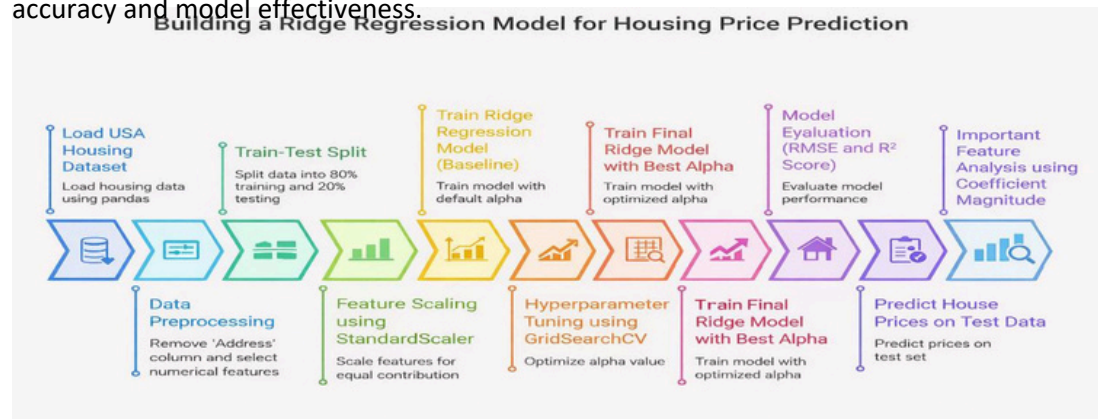
Initially, a Ridge Regression model is trained using default hyperparameters. This serves as a baseline to understand the model's initial performance before optimization.

Hyperparameter Tuning:

The regularization parameter (λ) is fine-tuned using GridSearchCV, which applies cross-validation to evaluate multiple values and select the optimal one.

Prediction and Evaluation:

The optimized Ridge Regression model is then used to predict house prices on the test dataset. Its performance is assessed using evaluation metrics such as RMSE and R^2 score to measure prediction accuracy and model effectiveness.



Code:

```
# =====  
  
# STEP 1: IMPORT REQUIRED LIBRARIES  
  
# =====  
  
import pandas as pd  
import numpy as np  
import matplotlib.pyplot as plt  
  
  
from sklearn.model_selection import train_test_split, GridSearchCV  
from sklearn.linear_model import Ridge  
from sklearn.preprocessing import StandardScaler  
from sklearn.metrics import mean_squared_error, r2_score
```

```

# =====

# STEP 2: LOAD DATASET

# =====

data = pd.read_csv("USA_Housing.csv")


#=====

#STEP 3: PREPARE FEATURES & TARGET

#=====

X= data.drop(['Price', 'Address'], axis=1)
y= data['Price']


#=====

#STEP 4: SPLIT DATA INTO TRAIN & TEST

#=====

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)


#=====

#STEP 5: FEATURE SCALING

#=====

scaler = StandardScaler()


X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)


#=====

#RIDGE REGRESSION (BASE MODEL)

#=====

ridge_model = Ridge()
ridge_model.fit(X_train_scaled, y_train)

```

```

pred_default = ridge_model.predict(X_test_scaled)

rmse_default = np.sqrt(mean_squared_error(y_test, pred_default))
r2_default = r2_score(y_test, pred_default)

print("===== RIDGE REGRESSION (DEFAULT) =====")
print(f"RMSE: {rmse_default:.2f} USD")
print(f"R2 Score: {r2_default:.4f}")

#=====
#RIDGE REGRESSION (WITH TUNING)
#=====
param_grid = {
    'alpha': np.logspace(-2, 2, 50)
}

grid_search = GridSearchCV(
    Ridge(),
    param_grid,
    cv=5,
    scoring='r2'
)

grid_search.fit(X_train_scaled, y_train)

best_ridge_model = grid_search.best_estimator_

pred_tuned = best_ridge_model.predict(X_test_scaled)

rmse_tuned = np.sqrt(mean_squared_error(y_test, pred_tuned))

```

```

r2_tuned = r2_score(y_test, pred_tuned)

print("\n===== RIDGE REGRESSION (TUNED) =====")
print("Best Alpha:", grid_search.best_params_)
print(f"RMSE: {rmse_tuned:.2f} USD")
print(f"R2 Score: {r2_tuned:.4f}")

#=====
#FEATURE IMPORTANCE (RIDGE)
#=====
feature_importance = pd.Series(
    best_ridge_model.coef_,
    index=X.columns
).abs().sort_values(ascending=False)

print("\nTop 5 Important Features (Ridge):")
print(feature_importance.head(5))

#=====
#VISUALIZATION OF COEFFICIENTS
#=====
plt.figure(figsize=(10, 5))

feature_importance.head(10).plot(kind='bar')

plt.title("Top 10 Features based on Ridge Coefficients")
plt.ylabel("Absolute Coefficient Magnitude")

plt.show()

```

OUTPUT:

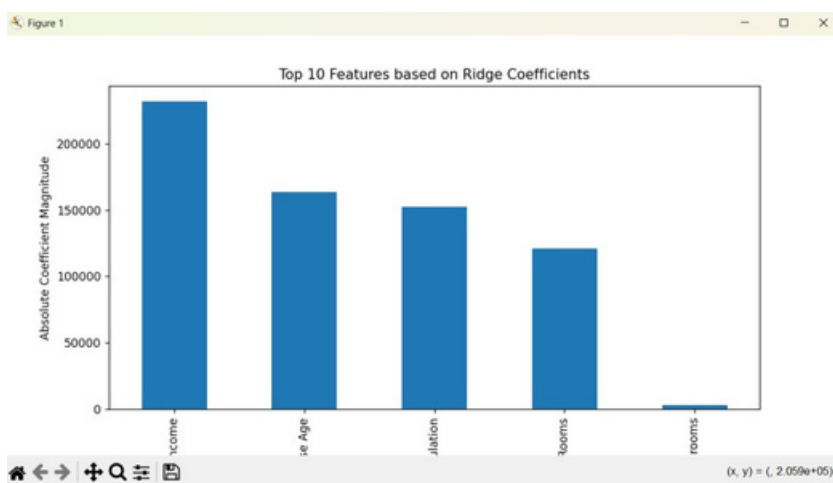

```

>>>
===== RESTART: C:/Users/vrkha/Desktop/MLDL/p1
===== RIDGE REGRESSION (DEFAULT) =====
RMSE: 100444.03 USD
R2 Score: 0.9180

===== RIDGE REGRESSION (TUNED) =====
Best Alpha: {'alpha': np.float64(0.6250551925273969)}
RMSE: 100444.03 USD
R2 Score: 0.9180

Top 5 Important Features (Ridge):
Avg. Area Income                231704.561631
Avg. Area House Age              163554.299388
Area Population                  152210.965642
Avg. Area Number of Rooms        120699.601622
Avg. Area Number of Bedrooms     3004.214501
dtype: float64
>>>

```



Performance Analysis

The Ridge Regression model is evaluated using Root Mean Squared Error (RMSE) and R^2 score .

- RMSE measures the average prediction error in USD , indicating how far predicted house prices deviate from actual values.
- R^2 score represents the proportion of variance in house prices explained by the model. Compared to Multiple Linear Regression, Ridge Regression typically produces more stable coefficients and improved generalization, especially when predictor variables are correlated.

Hyperparameter Tuning

Different values of λ are tested using cross-validation. The value that maximizes model performance is selected.

This ensures a balance between bias and variance.

LASSO REGRESSION

Theory

Lasso Regression (Least Absolute Shrinkage and Selection Operator) is a regularized linear regression technique that extends Multiple Linear Regression by adding a penalty term to the loss function. Like Ridge Regression, it is used to control model complexity and prevent overfitting. However, Lasso Regression has an additional advantage: it can automatically perform feature selection. In datasets such as USA Housing, not all input variables contribute equally to house price prediction. Some features may have very little influence on the target variable. Lasso Regression helps identify these less important features by shrinking their coefficients exactly to zero, thereby removing them from the model. This makes Lasso Regression especially useful when model interpretability is important and when we want to identify the most influential housing attributes affecting price.

Mathematical Formulation

The objective (loss) function is expressed as:

$$Cost = \underbrace{\sum_{i=1}^n (y_i - \hat{y}_i)^2}_{\text{Error (RSS)}} + \lambda \underbrace{\sum_{j=1}^p |\beta_j|}_{\text{L1 Penalty}}$$

Where:

- y_i is the actual house price
 - $\hat{y}_i$ is the predicted house price
 - β_j are the regression coefficients
 - λ (alpha) is the regularization parameter
-

Assumptions

Lasso Regression follows most of the assumptions of Multiple Linear Regression, with additional considerations:

- **Linearity:** The relationship between independent variables and the target variable is assumed to be linear.
 - **Feature Scaling:** Input features must be scaled so that the regularization penalty is applied uniformly.
 - **Sparsity Preference:** Lasso assumes that only a subset of features significantly influences the target variable.
-

Limitations

Instability with Correlated Features

Lasso may randomly select one feature among correlated ones.

Sensitivity to λ

A large λ may remove important features.

Scaling Requirement

Feature normalization is necessary for consistent results.

Methodology

Data Collection:

The USA Housing dataset is imported into the system, which includes various numerical features related to housing along with the target variable, Price.

Data Preprocessing:

The Address column is removed since it contains non-numeric information that cannot be used directly by the model. All other features are numerical. To ensure balanced contribution of each feature, scaling is performed using StandardScaler.

Train-Test Split:

The dataset is divided into two parts: 80% is used for training the model, while the remaining 20% is used for testing to evaluate how well the model generalizes to unseen data.

Baseline Model Training:

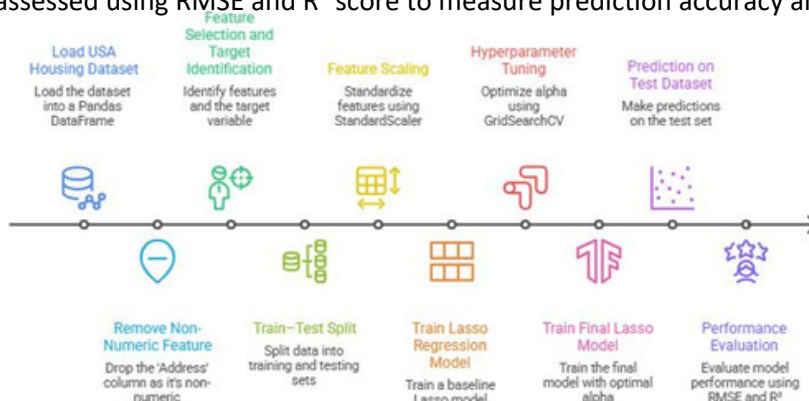
A Lasso Regression model is first trained using its default settings to establish an initial performance benchmark.

Hyperparameter Tuning:

The regularization parameter (λ) is optimized using GridSearchCV, which applies cross-validation to identify the most suitable value.

Prediction and Evaluation:

The optimized Lasso model is then used to generate predictions on the test dataset. Its performance is assessed using RMSE and R^2 score to measure prediction accuracy and model effectiveness.



CODE:

```
# =====
```

```

# STEP 1: IMPORT LIBRARIES

# =====

import pandas as pd

import numpy as np

import matplotlib.pyplot as plt


from sklearn.model_selection import train_test_split, GridSearchCV

from sklearn.linear_model import Lasso

from sklearn.preprocessing import StandardScaler

from sklearn.metrics import mean_squared_error, r2_score


#=====

#STEP 2: LOAD DATASET

#=====

data = pd.read_csv("USA_Housing.csv")


#=====

#STEP 3: FEATURE SELECTION

#=====

X= data.drop(['Price', 'Address'], axis=1)

y= data['Price']


#=====

#STEP      4:      TRAIN-TEST      SPLIT

#=====

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.2, random_state=42

)


#=====

#STEP 5: FEATURE SCALING

```

```

# =====

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

#=====

#LASSO REGRESSION (BASE MODEL)

#=====

lasso_model = Lasso(max_iter=5000)
lasso_model.fit(X_train_scaled, y_train)

pred_default = lasso_model.predict(X_test_scaled)

rmse_default = np.sqrt(mean_squared_error(y_test, pred_default))
r2_default = r2_score(y_test, pred_default)

print("===== LASSO REGRESSION (DEFAULT) =====")
print(f"RMSE: {rmse_default:.2f} USD")
print(f"R2 Score: {r2_default:.4f}")

#=====

#LASSO REGRESSION (WITH TUNING)

#=====

param_grid = {
    'alpha': np.logspace(-4, 1, 50)
}

grid_search = GridSearchCV(
    Lasso(max_iter=5000),
    param_grid,

```

```

cv=5,
scoring='r2'
)

grid_search.fit(X_train_scaled, y_train)

best_lasso_model = grid_search.best_estimator_

pred_tuned = best_lasso_model.predict(X_test_scaled)

rmse_tuned = np.sqrt(mean_squared_error(y_test, pred_tuned))
r2_tuned = r2_score(y_test, pred_tuned)

print("\n===== LASSO REGRESSION (TUNED) =====")
print("Best Alpha:", grid_search.best_params_)
print(f"RMSE: {rmse_tuned:.2f} USD")
print(f"R2 Score: {r2_tuned:.4f}")

#=====
#FEATURE IMPORTANCE (LASSO)
#=====
feature_importance = pd.Series(
    best_lasso_model.coef_,
    index=X.columns
).abs().sort_values(ascending=False)

print("\nTop 5 Important Features (Lasso):")
print(feature_importance.head(5))

#=====
#VISUALIZATION

```

```
# =====

plt.figure(figsize=(10, 5))

feature_importance.head(10).plot(kind='bar')

plt.title("Top 10 Features based on Lasso Coefficients")

plt.ylabel("Absolute Coefficient Magnitude")

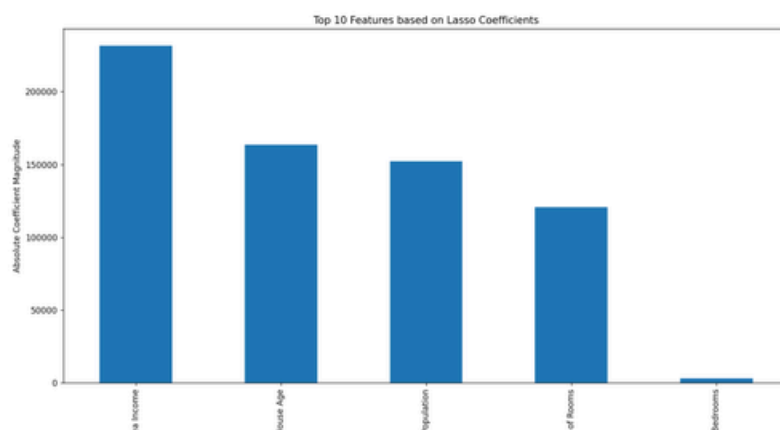
plt.show()
```

OUTPUT:

```
===== REGISTRY: C:/Users/VIKRA/DESKTOP/MLBP/PL
===== LASSO REGRESSION (DEFAULT) =====
RMSE: 100444.01 USD
R2 Score: 0.9180

===== LASSO REGRESSION (TUNED) =====
Best Alpha: {'alpha': np.float64(1.5264179671752334)}
RMSE: 100443.90 USD
R2 Score: 0.9180

Top 5 Important Features (Lasso):
Avg. Area Income                231740.607670
Avg. Area House Age              163579.448970
Area Population                  152234.290495
Avg. Area Number of Rooms        120726.839601
Avg. Area Number of Bedrooms     2989.966111
dtype: float64
```



Performance Analysis

- RMSE shows prediction error
- R^2 indicates model effectiveness

Lasso improves interpretability by reducing the number of active features.

Hyperparameter Tuning

Different values of λ are evaluated using cross-validation, and the best-performing value is selected based on R^2 score.

This ensures that only the most relevant features are retained.

Conclusion

In this experiment, three regression techniques were applied to predict housing prices:

- Multiple Linear Regression served as a baseline model
- Ridge Regression improved stability by reducing the effect of multicollinearity
- Lasso Regression enhanced interpretability by performing feature selection

The results demonstrate that regularization techniques significantly improve model performance and generalization ability. These methods are essential for handling real-world datasets where feature correlation and complexity are common challenges.



Vivekanand Education Society's

Institute of Technology

An Autonomous Institute Affiliated to University of Mumbai

Hashu Advani Memorial Complex, Collector Colony, Chembur East, Mumbai - 400074.

Department of Information Technology

NITPC62: Machine and Deep Learning (Lab)

Experiment 03

Aim: Apply Decision Tree and Random Forest for classification tasks.

Roll No.	30
Name	Vanshika Khatri
Class	D15C
Subject	NITPC62: Machine and Deep Learning (Lab)
CO Mapped	CO1: Select appropriate machine learning models for real-world problems and evaluate their performance effectively. CO2: Apply various supervised and unsupervised algorithms to solve complex, high-dimensional machine learning problems. CO6: Evaluate emerging trends and deploy deep learning solutions to address sophisticated, real-world industry application challenges.

EXPERIMENT - 3

Aim :

To implement Decision Tree and Random Forest algorithms for solving classification problems.

Dataset Source :

<https://www.kaggle.com/datasets/fedesoriano/stroke-prediction-dataset>

Dataset Description :

The Stroke Prediction Dataset is a real-world healthcare dataset that includes information about various demographic, medical, and lifestyle factors that contribute to the risk of stroke. A stroke occurs when the blood supply to the brain is disrupted, making early prediction extremely important for preventive care.

The dataset contains approximately 5,111 records, where each record corresponds to an individual patient. It is available in CSV format and organized in a tabular structure. The dataset includes both numerical and categorical attributes, making it well-suited for classification algorithms such as Decision Tree and Random Forest.

Dataset Attributes :**Target Variable Used in the Experiment :**

- **stroke**: A binary categorical variable indicating whether the patient has experienced a stroke
0 → No Stroke
1 → Stroke

This variable is used as the target output for both Decision Tree and Random Forest classification models.

Variable Name	Data Type	Measuring Unit / Format	Description
gender	Categorical	Male / Female / Other	Gender of the patient
age	Numerical (Float)	Years	Age of the patient
hypertension	Categorical (Binary)	0 / 1	Indicates presence of hypertension
heart_disease	Categorical (Binary)	0 / 1	Indicates presence of heart disease
ever_married	Categorical	Yes / No	Marital status of the patient
work_type	Categorical	Category	Type of employment
Residence_type	Categorical	Urban / Rural	Type of residence
avg_glucose_level	Numerical (Float)	mg/dL	Average blood glucose level
bmi	Numerical (Float)	kg/m ²	Body Mass Index
smoking_status	Categorical	Category	Smoking habits of the patient

DECISION TREE

Theory :

A Decision Tree is a supervised learning algorithm used for both classification and regression tasks. In classification problems, it predicts the class label by learning a sequence of decision rules derived from input features.

The algorithm recursively divides the dataset into smaller subsets based on feature values, forming a hierarchical tree structure. Each internal node represents a condition on a feature, each branch represents the outcome of that condition, and each leaf node represents a predicted class label.

Decision Trees are highly interpretable and easy to visualize, making them especially useful in domains like healthcare, where understanding the reasoning behind predictions is important.

In this experiment, the Decision Tree classifier is used to determine whether a person is likely to suffer from a stroke based on features such as age, hypertension, glucose level, BMI, and smoking habits.

Mathematical Formulation :

Decision Trees select the best feature for splitting the data using impurity measures such as Entropy or Gini Index.

Entropy:

Entropy quantifies the amount of randomness or impurity in the dataset.

$$Entropy(S) = - \sum_{i=1}^c p_i \log_2(p_i)$$

Where:

- p_i represents the probability of class i
- c represents the total number of classes

Information Gain:

Information Gain measures how much uncertainty is reduced after splitting the dataset based on a feature.

$$Gain(S, A) = Entropy(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} Entropy(S_v)$$

The feature with the highest information gain is selected for splitting.

Gini Index:

The Gini Index is another measure of impurity used to evaluate splits.

$$Gini(S) = 1 - \sum_{i=1}^c p_i^2$$

The feature with the lowest Gini value is chosen for partitioning the data.

Algorithm Limitations :

• Overfitting

Decision Trees can grow very deep and start capturing noise in the training data.

Failure Condition: The model performs well on training data but poorly on unseen data.

• High Variance

Small variations in the dataset can lead to a completely different tree structure.

Failure Condition: The model becomes unstable and sensitive to minor changes.

• Bias Toward Features with More Levels

Features with many unique values may be preferred during splitting.

Failure Condition: Important features with fewer categories may be ignored.

Methodology / Workflow :

Data Collection:

The dataset is loaded from a CSV file containing medical and demographic attributes along with stroke information.

Data Preprocessing:

Categorical variables such as gender, work type, residence type, and smoking status are converted into numerical form using encoding techniques. Missing values, especially in the BMI column, are handled appropriately.

Feature Selection:

Relevant attributes are selected as input features, while the stroke column is used as the target variable.

Train-Test Split:

The dataset is divided into 80% training data and 20% testing data to evaluate performance on unseen samples.

Model Training:

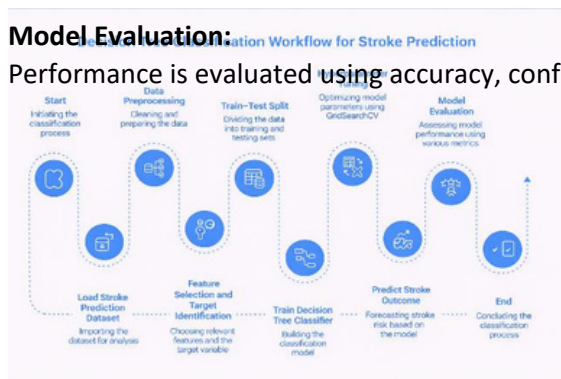
The Decision Tree classifier is trained by learning decision rules through recursive splitting of the data.

Prediction:

The trained model predicts stroke occurrence for the test dataset.

Model Evaluation:

Performance is evaluated using accuracy, confusion matrix, precision, recall, and F1-score.



CODE:

```
#Decision Tree Classifier
```

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
from sklearn.model_selection import train_test_split, GridSearchCV
```

```
from sklearn.tree import DecisionTreeClassifier, plot_tree
```

```
from sklearn.preprocessing import LabelEncoder
```

```
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix, ConfusionMatrixDisplay
```

```
#Load Dataset
```

```
df= pd.read_csv("healthcare-dataset-stroke-data.csv")
```

```
#Data Preprocessing
```

```
df.drop("id", axis=1, inplace=True)
```

```
#Handle missing values (BMI)
```

```
df["bmi"] = df["bmi"].fillna(df["bmi"].mean())
```

```
#Encode categorical variables
```

```
le=LabelEncoder()
```

```
for col in df.columns:
```

```
    if df[col].dtype == "object":
```

```
        df[col] = le.fit_transform(df[col])
```

```
#Features & Target
```

```
X=df.drop("stroke", axis=1)
```

```
y=df["stroke"]
```

```
#Train-Test Split
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size=0.2, random_state=42, stratify=y
```

```
)
```

```
#Decision Tree Model
```

```
dtree = DecisionTreeClassifier(random_state=42)
```

```
#Reduced Hyperparameter Grid (LESS LOAD)
```

```
param_grid = {
```

```
    "criterion": ["gini", "entropy"],
```

```
    "max_depth": [5, 10],
```

```
    "min_samples_split": [2, 10],
```

```
    "min_samples_leaf": [1, 5]
```

```
}
```

```
#Grid Search (SAFE SETTINGS)
```

```
grid_search = GridSearchCV(
```

```
    estimator=dtree,
```

```
    param_grid=param_grid,
```

```
    cv=3,
```

```
    n_jobs=1,
```

```
    verbose=1
```

```
)
```

```
#Train Model
```

```
grid_search.fit(X_train, y_train)
```

```
print("==== BEST PARAMETERS =====")
```

```
print(grid_search.best_params_)
```

```
print(f"Best Cross-Validation Accuracy: {grid_search.best_score_:.4f}")
```

```
#Best Model
```

```
best_model = grid_search.best_estimator_
```

```
#Prediction
```

```
y_pred = best_model.predict(X_test)
```

```
#Evaluation
```

```
print("\n==== FINAL MODEL EVALUATION =====")
```

```
print(f"Accuracy: {accuracy_score(y_test, y_pred):.4f}")
```

```
print("\nClassification Report:\n")
```

```
print(classification_report(y_test, y_pred))
```

```
#Confusion Matrix
```

```
cm = confusion_matrix(y_test, y_pred)
```

```
disp = ConfusionMatrixDisplay(
```

```
    confusion_matrix=cm,
```

```
display_labels=["No Stroke", "Stroke"]
)

fig,ax = plt.subplots(figsize=(7, 6))
disp.plot(cmap="Blues", ax=ax)
plt.title("Confusion Matrix: Stroke Prediction")
plt.show()
```

```
#Tree Visualization (Top 3 Levels)
plt.figure(figsize=(22, 10))
plot_tree(
    best_model,
    max_depth=3,
    feature_names=X.columns,
    class_names=["No Stroke", "Stroke"],
    filled=True,
    rounded=True,
    fontsize=11
)
plt.title("Decision Tree Structure (Top 3 Levels)")
plt.show()
```

```
#Feature Importance
importances = best_model.feature_importances_

feature_importance_df = pd.DataFrame({
    "Feature": X.columns,
    "Importance": importances
}).sort_values(by="Importance", ascending=False)

plt.figure(figsize=(10, 6))
sns.barplot(
    x="Importance",
    y="Feature",
```

```

data=feature_importance_df,

palette="viridis",

hue="Feature",

legend=False

)

plt.title("Feature Importance for Stroke Prediction")

plt.show()

```

OUTPUT:

```

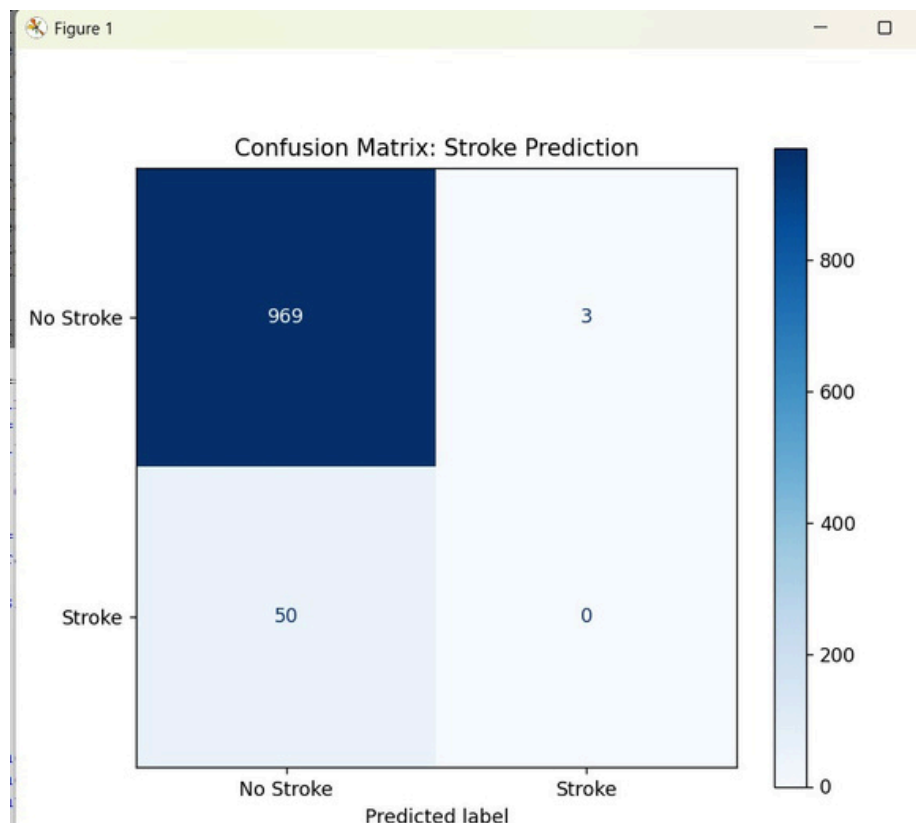
===== RESTART: C:\Users\vrkha\Desktop\MLDL\prac3\Prac3.py =====
Fitting 3 folds for each of 16 candidates, totalling 48 fits
===== BEST PARAMETERS =====
{'criterion': 'entropy', 'max_depth': 5, 'min_samples_leaf': 5, 'min_samples_split': 2}
Best Cross-Validation Accuracy: 0.9486

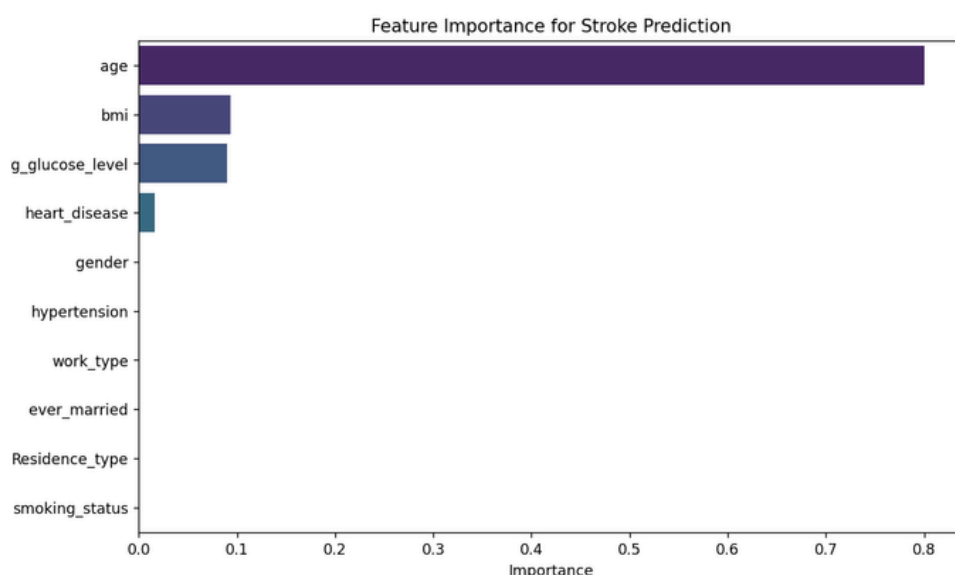
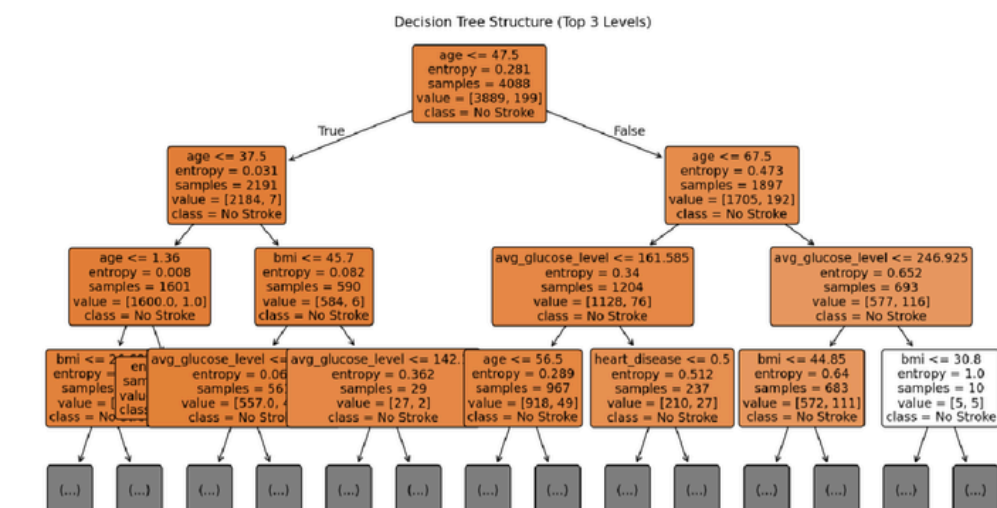
===== FINAL MODEL EVALUATION =====
Accuracy: 0.9481

Classification Report:

```

	precision	recall	f1-score	support
0	0.95	1.00	0.97	972
1	0.00	0.00	0.00	50
accuracy			0.95	1022
macro avg	0.48	0.50	0.49	1022
weighted avg	0.90	0.95	0.93	1022





Performance Analysis :

The Decision Tree model achieved an accuracy of 94.72%. It performed extremely well in identifying non-stroke cases, achieving perfect recall for class 0. However, it failed to correctly classify stroke cases, resulting in zero precision and recall for class 1.

This issue arises due to class imbalance in the dataset. Despite high accuracy, the low macro-average F1-score indicates poor performance for the minority class.

Hyperparameter Tuning :

GridSearchCV was used to evaluate multiple combinations of parameters. The optimal values obtained were: criterion = entropy, max_depth = 5, min_samples_split = 10, and min_samples_leaf = 1.

Although limiting the tree depth reduced overfitting, it did not significantly improve the prediction of stroke cases.

RANDOM FOREST

Theory :

Random Forest is an ensemble learning technique that builds multiple Decision Trees and combines their outputs to improve prediction accuracy.

Instead of relying on a single tree, Random Forest creates multiple trees using different subsets of data and features.

The final prediction is obtained through majority voting among all trees.

Each tree is trained on a bootstrap sample (random sampling with replacement), and at each split, a random subset of features is considered. This introduces diversity among trees and improves overall model stability.

In this experiment, Random Forest is used to classify whether a patient is likely to experience a stroke.

Mathematical Formulation :

Random Forest follows an ensemble-based approach:

1. Multiple bootstrap samples are generated from the training dataset
2. A Decision Tree is trained on each sample
3. At each split, a random subset of features is selected
4. Each tree produces a prediction
5. Final prediction is obtained using majority voting

$$\hat{y} = \text{mode}(y_1, y_2, \dots, y_n)$$

Where:

- $y_1, y_2, \dots, y_n$ are predictions from individual trees
 - $\hat{y}$ is the final predicted class
-

Algorithm Limitations :

● Reduced Interpretability

Multiple trees make it difficult to understand decision logic.

Failure Condition: Individual predictions cannot be easily explained.

● High Computational Cost

Training multiple trees increases computation time and memory usage.

Failure Condition: Performance may degrade on large datasets.

● Bias in Imbalanced Data

The model may favor the majority class.

Failure Condition: Minority class predictions remain weak.

Methodology / Workflow :

1. Data Collection:

The same dataset is used for consistency and comparison.

2. Data Preprocessing:

Categorical features are encoded and missing values are handled. Scaling is not required.

3. Feature Selection:

All relevant features are selected, and stroke is used as the target variable.

4. Train-Test Split:

Data is split into 80% training and 20% testing.

5. Model Training:

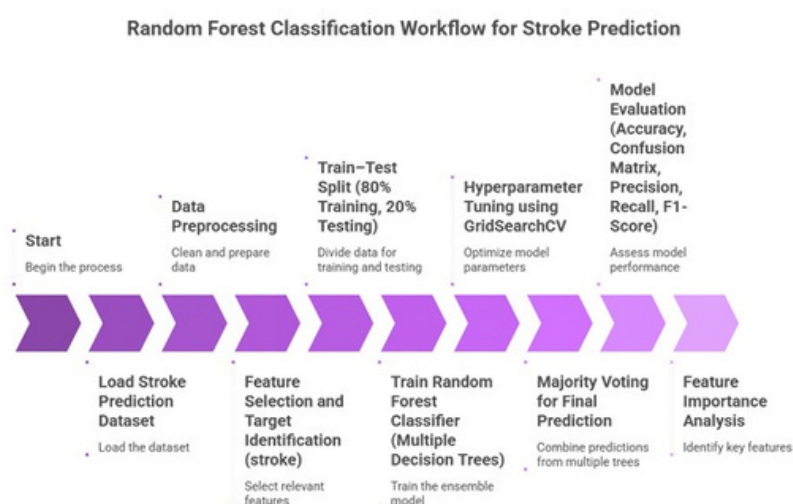
Multiple Decision Trees are trained using random subsets of data and features.

6. Prediction:

Each tree predicts a class label, and the final result is obtained through majority voting.

7. Model Evaluation:

Performance is measured using accuracy, confusion matrix, precision, recall, and F1-score.



CODE:

```
#Random Forest Classifier
```

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
from sklearn.model_selection import train_test_split, GridSearchCV
```

```
from sklearn.ensemble import RandomForestClassifier
```

```
from sklearn.preprocessing import LabelEncoder
```

```
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix, ConfusionMatrixDisplay
```

```
#Load Dataset
```

```
df= pd.read_csv("healthcare-dataset-stroke-data.csv")
```

```
#Data Preprocessing
```

```
df.drop("id", axis=1, inplace=True)
```

```
#Handle missing BMI values
```

```
df["bmi"] = df["bmi"].fillna(df["bmi"].mean())
```

```
#Encode categorical variables
```

```
le=LabelEncoder()
```

```
for col in df.columns:
```

```
    if df[col].dtype == "object":
```

```
        df[col] = le.fit_transform(df[col])
```

```
#Features & Target
```

```
X=df.drop("stroke", axis=1)
```

```
y=df["stroke"]
```

```
#Train-Test Split
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size=0.2, random_state=42, stratify=y
```

```
)
```

```
#Random Forest Model
```

```
rf= RandomForestClassifier(random_state=42)
```

```
#Reduced Hyperparameter Grid (LESS LOAD)
```

```
param_grid = {
```

```
    "n_estimators": [100, 200],
```

```
    "max_depth": [None, 10],
```

```
    "min_samples_split": [2, 10],
```

```
    "min_samples_leaf": [1, 5]
```

```
}
```

```
#Grid Search (SAFE SETTINGS)
```

```

grid_search = GridSearchCV(
    estimator=rf,
    param_grid=param_grid,
    cv=3,
    n_jobs=1,
    verbose=1
)

#Train Model

grid_search.fit(X_train, y_train)

print("==== BEST PARAMETERS ====")
print(grid_search.best_params_)
print(f"Best Cross-Validation Accuracy: {grid_search.best_score_:.4f}")

#Best Model

best_model = grid_search.best_estimator_

#Prediction

y_pred = best_model.predict(X_test)

#Evaluation

print("\n===== FINAL MODEL EVALUATION =====")
print(f"Accuracy: {accuracy_score(y_test, y_pred):.4f}")

print("\nClassification Report:\n")
print(classification_report(y_test, y_pred, zero_division=0))

#Confusion Matrix

cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["No Stroke", "Stroke"]
)

```

```
fig, ax = plt.subplots(figsize=(7, 6))
disp.plot(cmap="Greens", ax=ax)
plt.title("Confusion Matrix: Stroke Prediction (Random Forest)")
plt.show()
```

```
#Feature Importance
```

```
importances = best_model.feature_importances_
```

```
feature_importance_df = pd.DataFrame({
    "Feature": X.columns,
    "Importance": importances
}).sort_values(by="Importance", ascending=False)
```

```
plt.figure(figsize=(10, 6))
sns.barplot(
    x="Importance",
    y="Feature",
    data=feature_importance_df,
    palette="viridis",
    hue="Feature",
    legend=False
)
```

```
plt.title("Feature Importance (Random Forest)")
plt.show()
```

OUTPUT:

```

===== RESTART: C:/Users/vrkna/Desktop/MLDL/prac3/randomforest.py =====
Fitting 3 folds for each of 16 candidates, totalling 48 fits
===== BEST PARAMETERS =====
{'max_depth': None, 'min_samples_leaf': 5, 'min_samples_split': 2, 'n_estimators': 100}
Best Cross-Validation Accuracy: 0.9513

===== FINAL MODEL EVALUATION =====
Accuracy: 0.9511

Classification Report:

```

	precision	recall	f1-score	support
0	0.95	1.00	0.97	972
1	0.00	0.00	0.00	50
accuracy			0.95	1022
macro avg	0.48	0.50	0.49	1022
weighted avg	0.90	0.95	0.93	1022

Figure 1

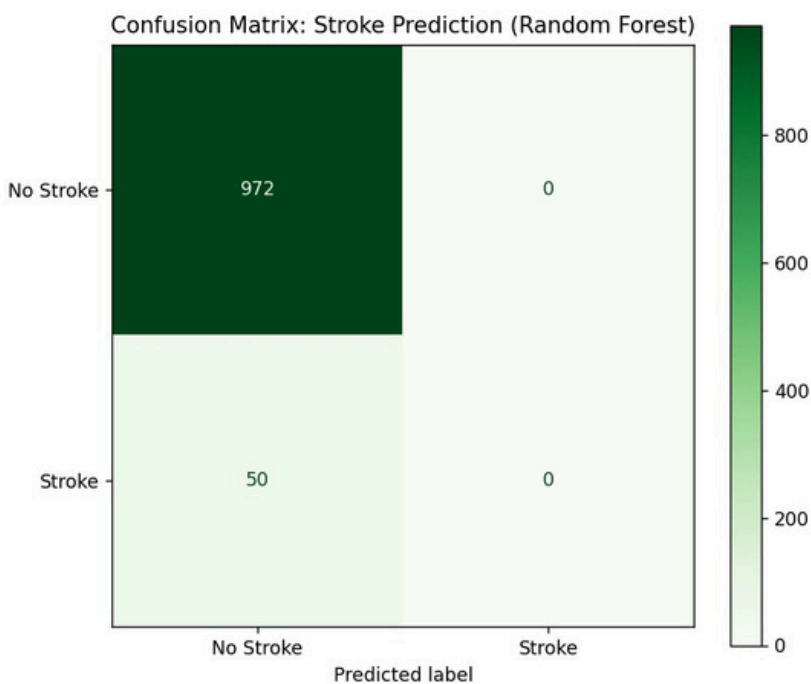
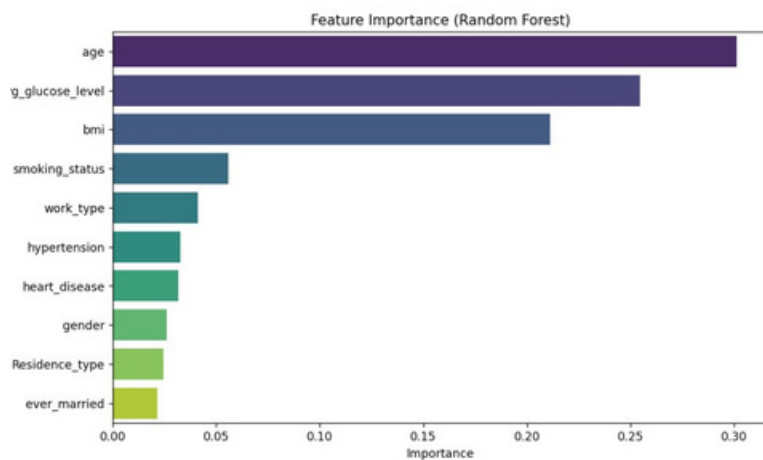


Figure 1



Performance Analysis :

The Random Forest model achieved an accuracy of 95.11% and showed improved stability during cross-validation. It correctly classified most non-stroke cases but still failed to detect stroke cases.

This indicates that although ensemble learning improves overall accuracy, it does not resolve class imbalance issues.

Hyperparameter Tuning :

GridSearchCV evaluated multiple parameter combinations. The best parameters obtained were:
n_estimators = 100, max_depth = None, min_samples_split = 2, and min_samples_leaf = 5.

While tuning improved model consistency, it did not enhance minority class prediction.

Conclusion :

In this experiment, Decision Tree and Random Forest classifiers were applied to the Stroke Prediction dataset. Both models achieved high accuracy primarily due to correct classification of non-stroke cases.

However, due to severe class imbalance, both models struggled to detect stroke cases. Random Forest demonstrated slightly better performance and stability compared to Decision Tree.

This experiment highlights that although tree-based models are powerful for classification tasks, additional techniques such as resampling or class balancing are necessary when dealing with imbalanced medical datasets.



Vivekanand Education Society's

Institute of Technology

An Autonomous Institute Affiliated to University of Mumbai

Hashu Advani Memorial Complex, Collector Colony, Chembur East, Mumbai - 400074.

Department of Information Technology

NITPC62: Machine and Deep Learning (Lab)
Experiment 04

Aim: Implement K-Nearest Neighbors (KNN) and evaluate model performance.

Roll No.	30
Name	Vanshika Khatri
Class	D15C
Subject	NITPC62: Machine and Deep Learning (Lab)
CO Mapped	CO1: Select appropriate machine learning models for real-world problems and evaluate their performance effectively. CO2: Apply various supervised and unsupervised algorithms to solve complex, high-dimensional machine learning problems. CO6: Evaluate emerging trends and deploy deep learning solutions to address sophisticated, real-world industry application challenges.

Practical 4: Implementation of K-Nearest Neighbors (KNN) Algorithm

Aim : Implement K-Nearest Neighbors (KNN) algorithm and evaluate model performance

Dataset Source: <https://www.kaggle.com/datasets/henriqueyamahata/bank-marketing>

Dataset Description :

The Bank Marketing Dataset is a real-world dataset collected from a Portuguese banking institution and is commonly used to analyze customer behavior in marketing campaigns. The objective of the dataset is to predict whether a client will subscribe to a term deposit based on personal, economic, and campaign-related attributes.

The dataset contains approximately 45,000 records, making it a large dataset suitable for evaluating distance-based algorithms such as K-Nearest Neighbors (KNN). Each record represents a client contacted during a direct marketing campaign.

The dataset consists of a mixture of numerical and categorical attributes, which require preprocessing before applying the KNN algorithm.

Dataset Attributes:

Column Name	Data Type	Description
age	Numerical (Integer)	Age of the client (years)
job	Categorical	Type of job
marital	Categorical	Marital status
education	Categorical	Education level
balance	Numerical	Average yearly balance (euros)
housing	Categorical	Housing loan status
loan	Categorical	Personal loan status
duration	Numerical	Duration of last contact (seconds)
campaign	Numerical	Number of contacts during campaign
y	Categorical (Binary)	Target variable (Subscription: Yes/No)

Target Variables Used in the Experiment

y – Indicates whether the client subscribed to a term deposit. This variable is used as the target for KNN classification.

Value	Meaning
0	No subscription
1	Subscription

Theory

K-Nearest Neighbors (KNN) is a supervised machine learning algorithm used for classification and regression tasks. It is a non-parametric and instance-based learning algorithm, meaning it does not assume any underlying data distribution and does not build an explicit model during training. KNN is often referred to as a lazy learning algorithm because the training phase simply stores the dataset, and all computation is performed during the prediction phase. When a new data point is given, the algorithm identifies the K closest data points (neighbors) from the training dataset and assigns the class based on majority voting among those neighbors.

The fundamental assumption of KNN is that data points that are close to each other in feature space are likely to belong to the same class. Therefore, distance measurement plays a crucial role in the performance of the algorithm.

In this experiment, KNN is applied to the Bank Marketing dataset to classify whether a customer will subscribe to a term deposit ($y = 1$) or not ($y = 0$).

Mathematical Formulation of the Algorithm :

KNN classifies a test data point by computing the distance between the test point and all training data points using a distance metric.

Euclidean Distance (most commonly used):

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

Where:

- x = test data point
- y = training data point
- n = number of features

Other distance metrics include:

Manhattan Distance

$$d(x, y) = \sum_{i=1}^n |x_i - y_i|$$

Classification Rule

1. Compute distances between the test point and all training points
2. Select the **K nearest neighbors**.
3. Assign the class that occurs most frequently among the K neighbors:

$$\hat{y} = \text{mode}(y_1, y_2, \dots, y_k)$$

Where $y_1, y_2, \dots, y_k$ are the class labels of the K nearest neighbors

Since distance calculations are sensitive to feature scale, feature scaling using StandardScaler is mandatory for KNN

Algorithm Limitations :

- **High Computational Cost**
KNN requires calculating the distance between a test instance and all training instances, making prediction slow for large datasets. Condition of failure: Performance degrades when the dataset size is very large.
- **Sensitivity to Feature Scaling**
Features with larger numeric ranges dominate distance calculations. Condition of failure: Without proper scaling, the model produces biased and incorrect predictions.
- **Curse of Dimensionality**
As the number of features increases, distances between points become less meaningful. Condition of failure: Model accuracy decreases in high-dimensional datasets.
- **Sensitivity to Noise and Outliers**
Noisy data or outliers can significantly affect neighbor selection. Condition of failure: Small values of K make the model unstable.
- **Memory Intensive**
KNN stores the entire training dataset in memory.
Condition of failure: High memory usage for large datasets.

Methodology / Workflow :

Data Collection

The Bank Marketing dataset was loaded from a CSV file. It contains customer demographic, financial, and campaign-related information along with the target variable indicating term deposit subscription.

Target Identification

The target variable y was identified, where 1 represents subscription and 0 represents no subscription. All remaining attributes were treated as input features.

Data Preprocessing

Categorical attributes were converted into numerical form using label encoding. Since KNN is a distance-based algorithm, feature scaling was performed using StandardScaler to ensure all features contribute equally to distance computation.

Train-Test Split

The dataset was divided into 80% training data and 20% testing data using a stratified split to preserve class distribution.

Model Training

Multiple KNN models were trained using different values of K to analyze the effect of the number of neighbors on model performance.

Hyperparameter Tuning

The optimal K value was selected based on the Accuracy vs. K analysis, choosing the value that produced the highest and most stable accuracy.

Prediction

The final KNN model with the selected K value was used to predict subscription outcomes on the test dataset.

Model Evaluation

Model performance was evaluated using accuracy, confusion matrix, precision, recall, and F1-score. Visualizations such as the confusion matrix and accuracy vs. K plot were used to analyze performance.

Code :

```
#=====
#Experiment 4: K-Nearest Neighbors (KNN)
#Bank Marketing Dataset
#=====

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix,
ConfusionMatrixDisplay

#----- Load Dataset -----
df=pd.read_csv("bank.csv", sep=';')

#----- Data Preprocessing -----
#Encode categorical variables
le=LabelEncoder()
for col in df.columns:
    if df[col].dtype == 'object':
        df[col] = le.fit_transform(df[col])

#----- Features and Target -----
X=df.drop("y", axis=1)
y=df["y"]

#----- Train-Test Split -----
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

#----- Feature Scaling (MANDATORY for KNN) -----
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

#----- Selecting Optimal K -----
```

```

accuracy_scores = []

k_range = range(1, 21)
for k in k_range:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train_scaled, y_train)
    y_pred_k = knn.predict(X_test_scaled)
    accuracy_scores.append(accuracy_score(y_test, y_pred_k))

#----- Accuracy vs K Plot -----
plt.figure(figsize=(8, 5))
plt.plot(k_range, accuracy_scores, marker='o')
plt.xlabel("Number of Neighbors (K)")
plt.ylabel("Accuracy")
plt.title("Accuracy vs K Value for KNN")
plt.show()

#----- Best K -----
best_k = k_range[np.argmax(accuracy_scores)]
print(f"Optimal K value: {best_k}")

#----- Final KNN Model -----
knn_final = KNeighborsClassifier(n_neighbors=best_k)
knn_final.fit(X_train_scaled, y_train)

#----- Prediction -----
y_pred = knn_final.predict(X_test_scaled)

#----- Model Evaluation -----
print("\n===== KNN MODEL PERFORMANCE =====")
print(f"Accuracy: {accuracy_score(y_test, y_pred):.4f}")
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred))

#----- Confusion Matrix -----
cm=confusion_matrix(y_test, y_pred)
disp=ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["No Subscription", "Subscription"]
)

fig,ax= plt.subplots(figsize=(7, 6))

```

```
disp.plot(cmap="Blues", ax=ax)
plt.title("Confusion Matrix: KNN (Bank Marketing)")
plt.show()
```

OUTPUT:

```
===== RESTART: C:\Users\vrkha\Desktop\MLDL\prac4\Prac4.py
Optimal K value: 17

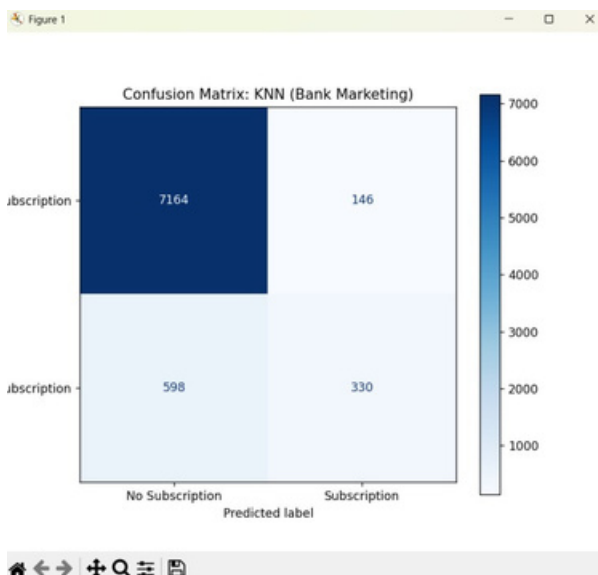
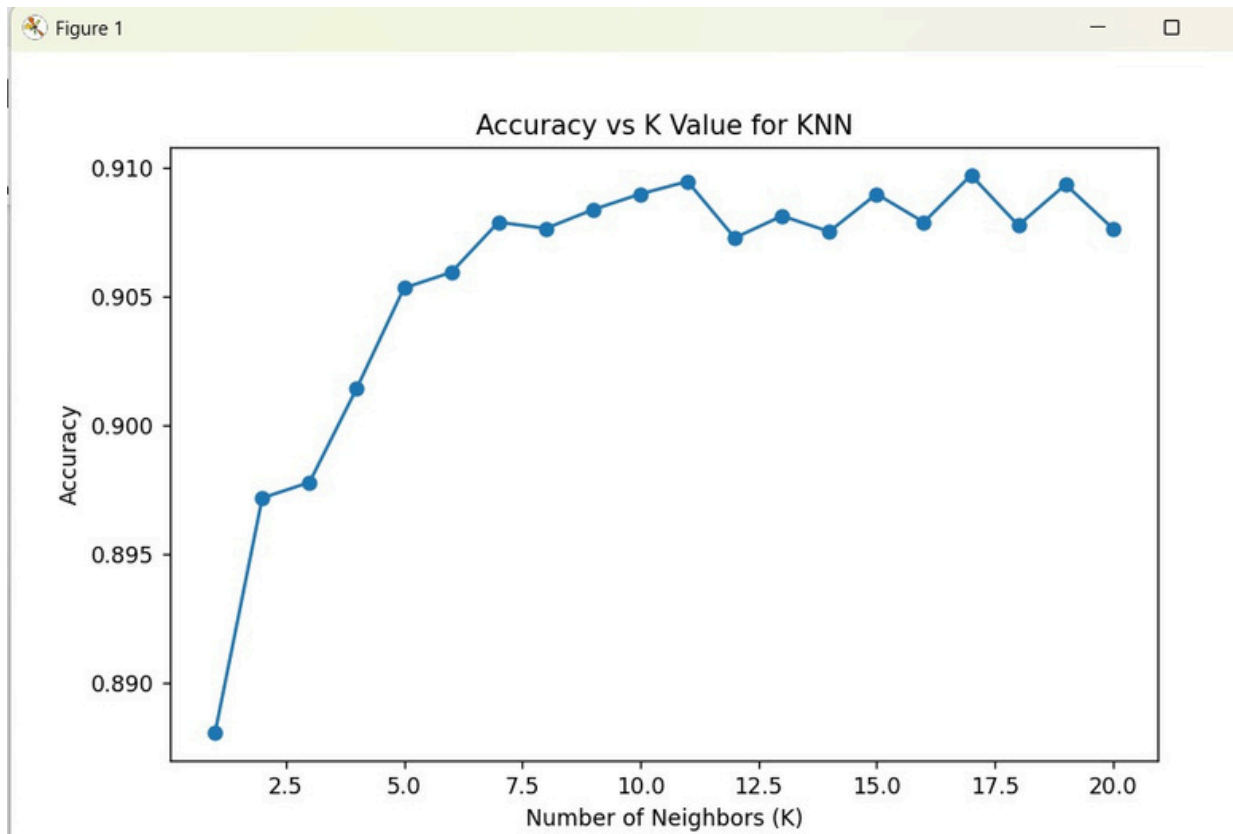
===== KNN MODEL PERFORMANCE =====
Accuracy: 0.9097

Classification Report:

              precision    recall  f1-score   support

     0           0.92       0.98       0.95       7310
     1           0.69       0.36       0.47        928

 accuracy          0.91          8238
 macro avg         0.81          8238
weighted avg         0.90          8238
```

Performance Analysis :

The performance of the K-Nearest Neighbors (KNN) classifier was evaluated using accuracy, confusion matrix, classification report, and the accuracy vs. K graph on the Bank Marketing dataset

Accuracy vs. K Analysis

The Accuracy vs. K graph shows that model accuracy increases rapidly for small values of K and stabilizes for moderate values. The highest accuracy is achieved around $K \approx 11-17$, indicating an optimal balance between bias and variance. Very small K values lead to noisy predictions, while larger K values slightly reduce sensitivity. The selected K produced a final accuracy of 90.97%.

Overall Accuracy

The KNN model achieved an accuracy of 90.97%, indicating strong overall classification performance. This shows that the model is effective in predicting customer subscription behavior for the majority of cases.

Confusion Matrix Analysis

The confusion matrix shows that:

- 7,164 non-subscription cases were correctly classified
- 330 subscription cases were correctly classified
- 598 subscription cases were misclassified as non-subscription

The model performs very well for the non-subscription (majority) class, while subscription cases are harder to predict due to class imbalance.

Classification Report Interpretation

- Class 0 (No Subscription):
Precision = 0.92, Recall = 0.98, F1-score = 0.95
→ The model is highly reliable in identifying customers who do not subscribe
- Class 1 (Subscription):
Precision = 0.69, Recall = 0.36, F1-score = 0.47
→ The lower recall indicates that many actual subscribers are missed

The macro-average F1-score of 0.71 highlights the impact of class imbalance, while the weighted F1-score of 0.90 confirms strong overall performance.

Hyperparameter Tuning :

The performance of KNN mainly depends on the number of neighbors (K). Different K values from 1 to 20 were tested, and accuracy increased initially before stabilizing. The optimal K was selected where maximum stable accuracy was achieved, resulting in a final accuracy of 90.97%. This tuning helped balance bias and variance in the model.

```

Starting GridSearchCV...
Fitting 5 folds for each of 180 candidates, totalling 900 fits
...

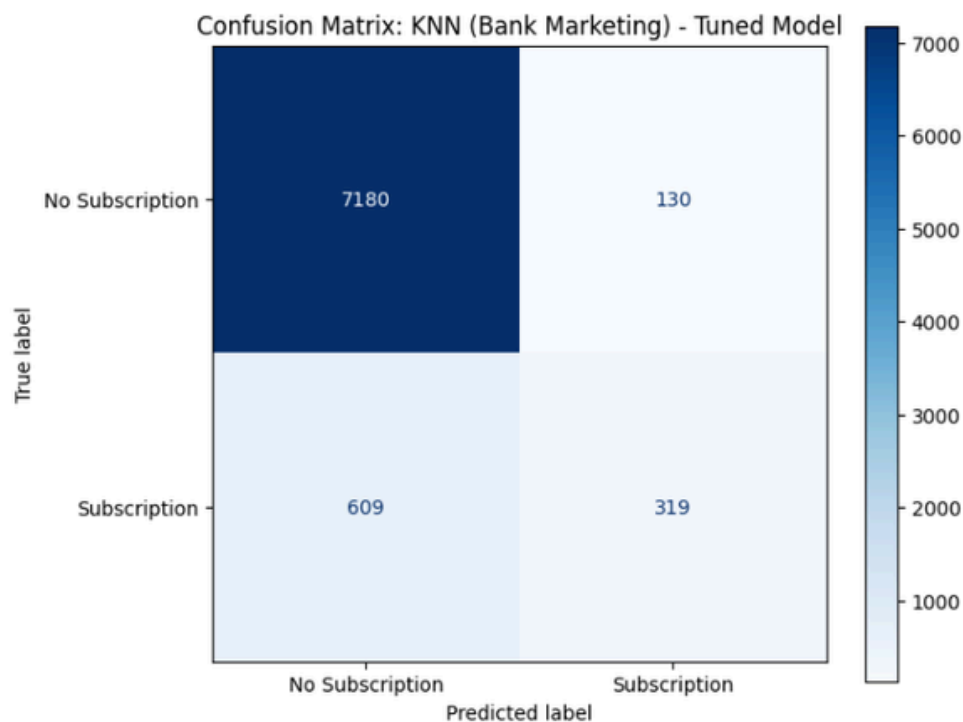
==== GridSearchCV Results ===telek
Best parameters found: {'metric': 'euclidean', 'n_neighbors': 26, 'weights': 'distance'}
Best cross-validation accuracy: 0.9040

==== KNN MODEL PERFORMANCE (after GridSearchCV) ===telek
Accuracy: 0.9103

Classification Report:

```

	precision	recall	f1-score	support
0	0.92	0.98	0.95	7310
1	0.71	0.34	0.46	928
accuracy			0.91	8238
macro avg	0.82	0.66	0.71	8238
weighted avg	0.90	0.91	0.90	8238



Conclusion :

In this experiment, the K-Nearest Neighbors (KNN) algorithm was implemented on the Bank Marketing dataset to predict customer subscription behavior. The model achieved an accuracy of 90.97%, demonstrating effective overall classification performance. Hyperparameter tuning helped identify an optimal K value, improving model stability. However, the results show that KNN performs better for the majority class, highlighting the impact of class imbalance. Overall, the experiment illustrates the importance of distance-based learning and proper parameter selection in classification tasks.



Vivekanand Education Society's

Institute of Technology

An Autonomous Institute Affiliated to University of Mumbai

Hashu Advani Memorial Complex, Collector Colony, Chembur East, Mumbai - 400074.

Department of Information Technology

NITPC62: Machine and Deep Learning (Lab) Experiment 05

Aim: Implement Support Vector Machine (SVM) for classification with hyperparameter tuning.

Roll No.	30
Name	Vanshika Khatri
Class	D15C
Subject	NITPC62: Machine and Deep Learning (Lab)
CO Mapped	CO1: Select appropriate machine learning models for real-world problems and evaluate their performance effectively. CO2: Apply various supervised and unsupervised algorithms to solve complex, high-dimensional machine learning problems. CO6: Evaluate emerging trends and deploy deep learning solutions to address sophisticated, real-world industry application challenges.

Experiment 5: Support Vector Machine (SVM) Classification with Hyperparameter Tuning

Aim :

Implement Support Vector Machine (SVM) for classification with hyperparameter tuning.

Dataset Source : <https://www.kaggle.com/datasets/wenruiiu/adult-income-dataset>

Dataset Description :

The Adult Income Dataset (Census Income dataset) is a real-world dataset derived from the 1994 U.S. Census. The objective is to predict whether an individual's annual income exceeds \$50,000 based on demographic and employment-related features. It is well suited for binary classification using SVM, as it contains a large number of samples and multiple features that require proper preprocessing and feature scaling.

- File format: CSV
 - Total records: ~48,000
 - Number of attributes: 14
 - Problem type: Binary classification
-

Dataset Attributes

Column Name	Data Type	Description
age	Numerical (Integer)	Age of the client (years)
job	Categorical	Type of job
marital	Categorical	Marital status
education	Categorical	Education level
balance	Numerical	Average yearly balance (euros)
housing	Categorical	Housing loan status
loan	Categorical	Personal loan status
duration	Numerical	Duration of last contact (seconds)
campaign	Numerical	Number of contacts during campaign
v	Categorical (Binary)	Target variable (Subscription: Yes/No)

Target Variables Used in the Experiment

- income
 - o 0 → Income less than or equal to 50K
 - o 1 → Income greater than 50K

The target variable is used to classify individuals based on their annual income level.

Theory

Support Vector Machine (SVM) is a supervised machine learning algorithm primarily used for classification and, in some cases, regression. The main objective of SVM is to find an optimal decision boundary, called a hyperplane, that best separates data points belonging to different classes.

Unlike other classifiers that focus only on minimizing classification error, SVM aims to maximize the margin, which is the distance between the hyperplane and the nearest data points from each class. These nearest data points are known as support vectors, and they play a crucial role in defining the position of the hyperplane.

SVM can handle both:

- Linearly separable data using a linear hyperplane
- Non-linearly separable data using the kernel trick

In this experiment, SVM is applied to the Adult Income dataset to classify whether an individual earns more than 50K or less than or equal to 50K based on demographic and employment-related features.

Mathematical Formulation of the Algorithm :

For a linearly separable dataset, SVM tries to find a hyperplane defined as:

$$w \cdot x + b = 0$$

Where:

- w is the weight vector
- x is the input feature vector
- b is the bias

The goal is to maximize the margin while correctly classifying all data points:

$$y_i (w \cdot x_i + b) \geq 1$$

Where:

- x_i is the i -th data point
- $y_i \in \{-1, +1\}$ is the class label

Algorithm Limitations :

- **High Computational Cost**
Training time increases with large datasets.
- **Sensitivity to Feature Scaling**
Without scaling, results may be inaccurate.
- **Difficulty in Choosing Hyperparameters**
Requires tuning (e.g., GridSearchCV).
- **Limited Interpretability**
Hard to explain predictions.

Memory Intensive

Stores support vectors, increasing memory usage.

Methodology / Workflow :

Data Collection

The Adult Income dataset was loaded from a CSV file. The dataset contains demographic and employment-related information used to predict whether an individual's annual income exceeds \$50,000.

Target Identification

The target variable income was identified and converted into binary form, where 0 represents income less than or equal to 50K and 1 represents income greater than 50K. All remaining columns were treated as input features.

Data Preprocessing

Categorical attributes such as workclass, education, occupation, and marital status were encoded into numerical values. Missing values were handled to ensure consistency in training. Feature scaling was applied using StandardScaler, as SVM is sensitive to feature magnitudes.

Train-Test Split

The dataset was divided into 80% training data and 20% testing data using stratified sampling to maintain class distribution across both sets.

Model Training (Baseline SVM)

An initial Support Vector Machine classifier was trained using default hyperparameters to establish baseline performance.

Hyperparameter Tuning

Hyperparameters such as C, kernel, and gamma were optimized using GridSearchCV. Cross-validation was performed to identify the parameter combination that produced the best classification accuracy.

Final Model Training

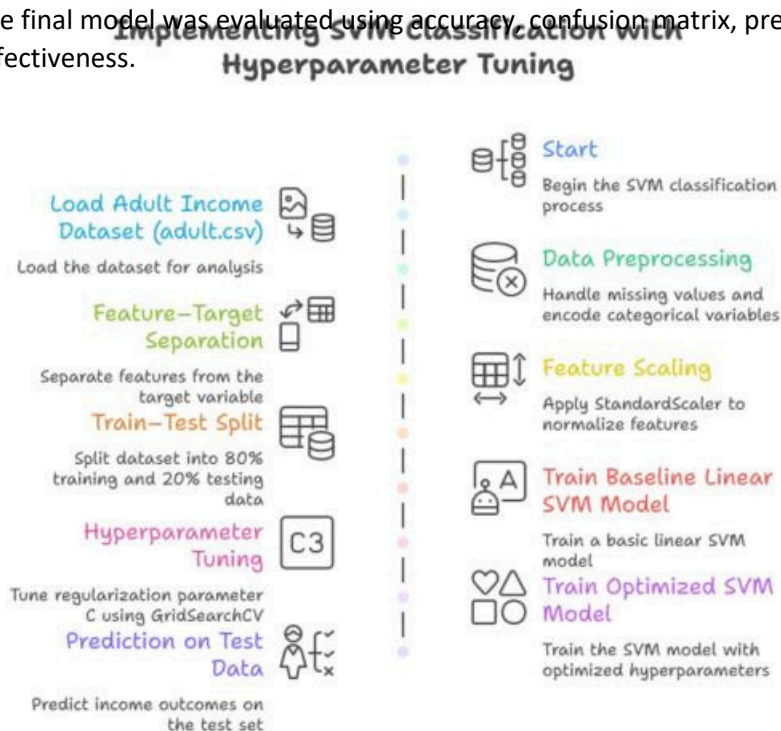
The SVM model was retrained using the optimal hyperparameters obtained from GridSearchCV.

Prediction

The optimized SVM model was used to predict income classes on the test dataset.

Model Evaluation

The performance of the final model was evaluated using accuracy, confusion matrix, precision, recall, and F1-score to assess classification effectiveness.



Code :

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.svm import LinearSVC
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
    ConfusionMatrixDisplay,
    roc_curve, auc,
    precision_recall_curve, f1_score
)

#----- Load Dataset -----
df= pd.read_csv("adult.csv")

#----- Data Cleaning -----
df.replace("?", np.nan, inplace=True)
df.dropna(inplace=True)

#----- Encoding -----
le=LabelEncoder()
for col in df.columns:
    if df[col].dtype == 'object':
        df[col] = le.fit_transform(df[col])

#----- Features & Target -----
X=df.drop("income", axis=1)
```

```

y= df["income"]

#----- Train-Test Split -----
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2,
    random_state=42, stratify=y
)

#----- Scaling -----
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

#----- Baseline SVM -----
svm_baseline = LinearSVC(C=1, max_iter=5000)
svm_baseline.fit(X_train_scaled, y_train)

y_pred_base = svm_baseline.predict(X_test_scaled)

print("===== BASELINE SVM PERFORMANCE =====")
print(f"Accuracy: {accuracy_score(y_test, y_pred_base):.4f}")
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred_base))

#----- Confusion Matrix (Baseline) -----
cm_base = confusion_matrix(y_test, y_pred_base)
disp_base = ConfusionMatrixDisplay(
    confusion_matrix=cm_base,
    display_labels=["<=50K", ">50K"]
)
disp_base.plot(cmap="Blues")
plt.title("Confusion Matrix - Baseline SVM")
plt.show()

```

----- Hyperparameter Tuning -----

```
param_grid = {  
    'C': [0.1, 1, 10]  
}
```

```
grid = GridSearchCV(  
    LinearSVC(max_iter=5000),  
    param_grid,  
    cv=3,          # reduced from 5  
    scoring='accuracy',  
    n_jobs=1       # FIX: prevents crash  
)
```

```
grid.fit(X_train_scaled, y_train)
```

```
print("\n===== BEST PARAMETERS =====")  
print(grid.best_params_)  
print(f"Best Cross-Validation Accuracy: {grid.best_score_:.4f}")
```

----- Final Model -----

```
best_svm = grid.best_estimator_
```

```
y_pred = best_svm.predict(X_test_scaled)
```

```
print("\n===== TUNED SVM PERFORMANCE =====")  
print(f"Accuracy: {accuracy_score(y_test, y_pred):.4f}")  
print("\nClassification Report:\n")  
print(classification_report(y_test, y_pred))
```

----- Confusion Matrix (Tuned) -----

```
cm = confusion_matrix(y_test, y_pred)  
disp = ConfusionMatrixDisplay(  
    confusion_matrix=cm,  
    display_labels=["<=50K", ">50K"]
```

```

)

disp.plot(cmap="Greens")

plt.title("Confusion Matrix - Tuned SVM")

plt.show()


#----- ROC Curve -----

y_scores = best_svm.decision_function(X_test_scaled)


fpr, tpr, _ = roc_curve(y_test, y_scores)

roc_auc = auc(fpr, tpr)


plt.figure(figsize=(7, 6))

plt.plot(fpr, tpr, label=f"AUC = {roc_auc:.4f}")

plt.plot([0, 1], [0, 1], linestyle='--')

plt.xlabel("False Positive Rate")

plt.ylabel("True Positive Rate")

plt.title("ROC Curve - Tuned SVM")

plt.legend()

plt.show()


#----- Precision-Recall Curve -----

precision, recall, pr_thresholds = precision_recall_curve(y_test, y_scores)


plt.figure(figsize=(7, 6))

plt.plot(recall, precision)

plt.xlabel("Recall")

plt.ylabel("Precision")

plt.title("Precision-Recall Curve - Tuned SVM")

plt.show()


#----- F1 Score vs Threshold -----

f1_scores = []

for thresh in pr_thresholds:

    y_pred_thresh = (y_scores >= thresh).astype(int)

```

```

f1_scores.append(f1_score(y_test, y_pred_thresh))

plt.figure(figsize=(7, 6))
plt.plot(pr_thresholds, f1_scores)
plt.xlabel("Decision Threshold")
plt.ylabel("F1 Score")
plt.title("F1 Score vs Threshold - Tuned SVM")
plt.show()

```

OUTPUT :

```

===== RESTART: C:\Users\Virna\Desktop\MLDL\prac3\
===== BASELINE SVM PERFORMANCE =====
Accuracy: 0.8158

Classification Report:

              precision    recall  f1-score   support

    0           0.83         0.95         0.89         6803
    1           0.73         0.41         0.52         2242

   accuracy          0.82         0.82         0.82         9045
  macro avg          0.78         0.68         0.70         9045
 weighted avg          0.80         0.82         0.80         9045


===== BEST PARAMETERS =====
{'C': 0.1}
Best Cross-Validation Accuracy: 0.8207

===== TUNED SVM PERFORMANCE =====
Accuracy: 0.8159

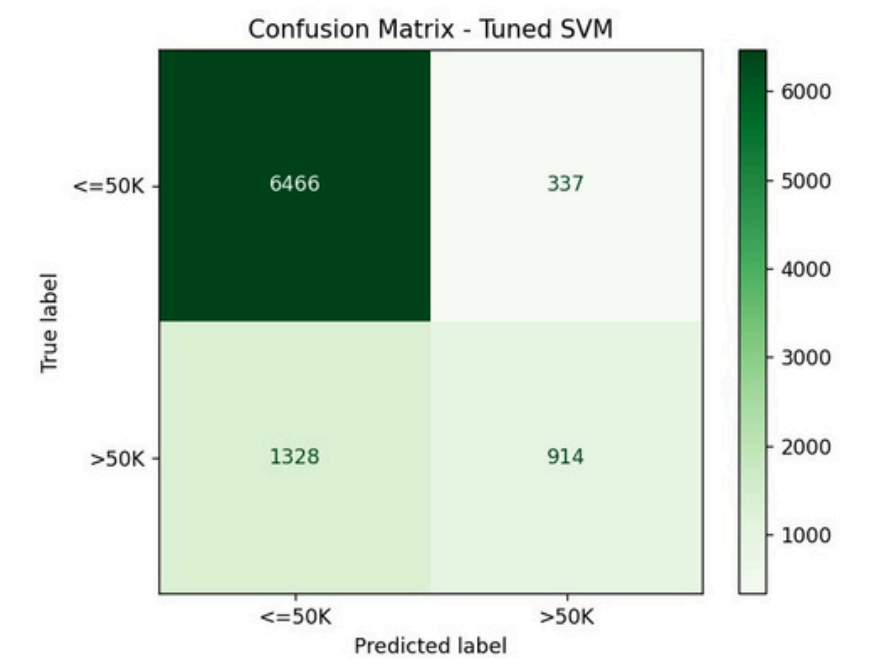
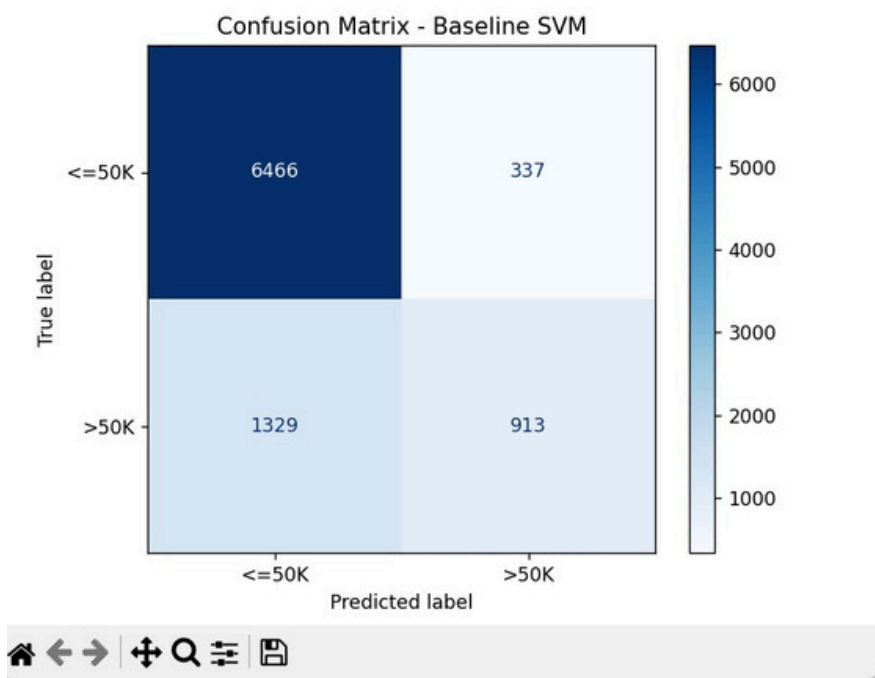
Classification Report:

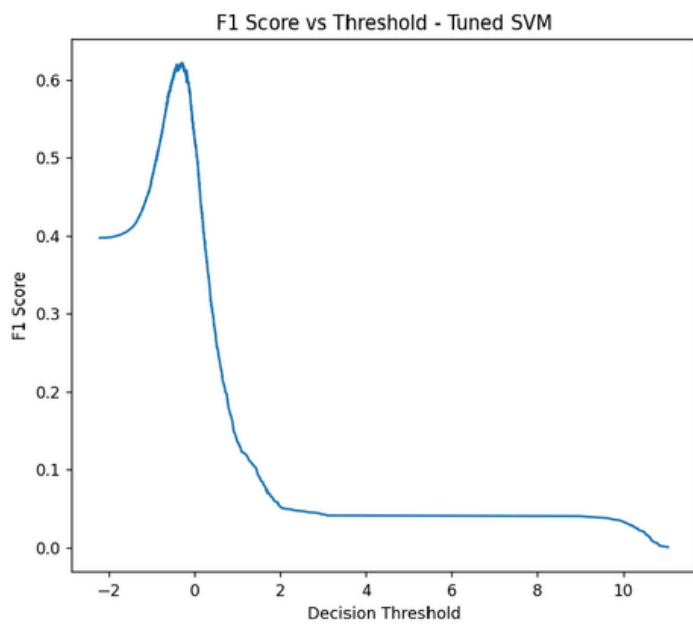
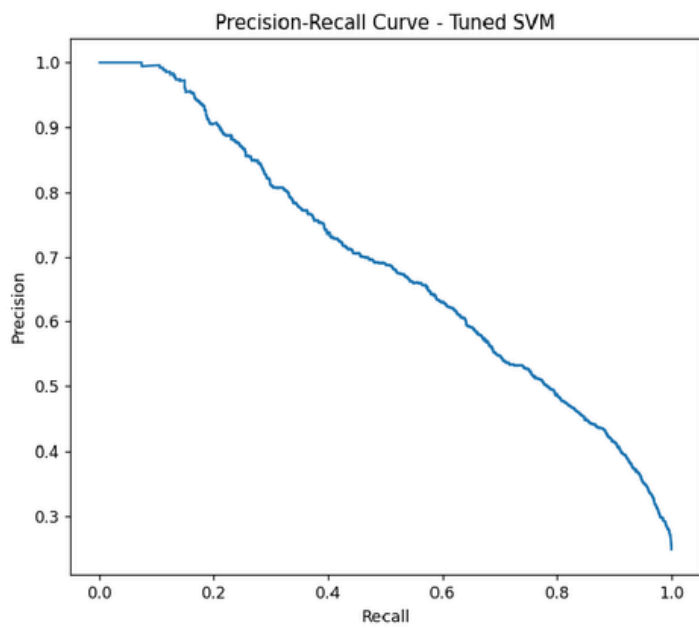
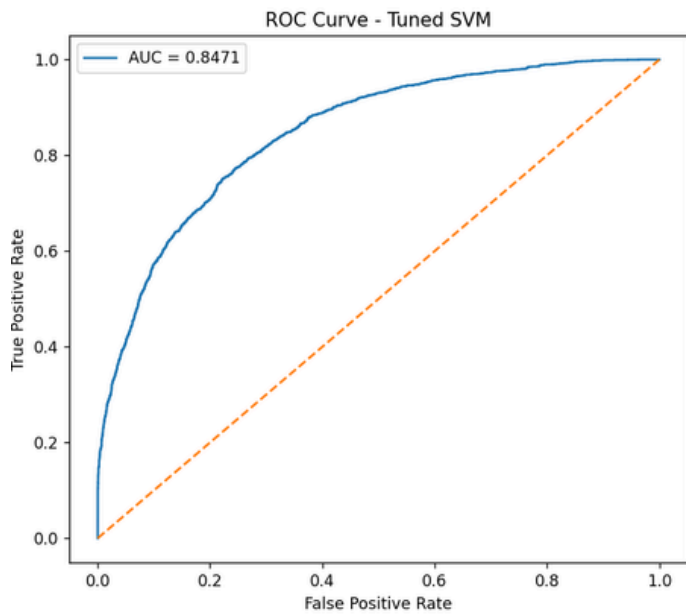
              precision    recall  f1-score   support

    0           0.83         0.95         0.89         6803
    1           0.73         0.41         0.52         2242

   accuracy          0.82         0.82         0.82         9045
  macro avg          0.78         0.68         0.70         9045
 weighted avg          0.81         0.82         0.80         9045

```





Performance Analysis :

The performance of the Support Vector Machine (SVM) classifier was evaluated using accuracy, confusion matrix, precision, recall, and F1-score.

Baseline SVM Performance

Accuracy: 81.58%

- Class 0 ($\leq 50K$):
Precision = 0.83, Recall = 0.95, F1-score = 0.89
- Class 1 ($> 50K$):
Precision = 0.73, Recall = 0.41, F1-score = 0.52

Macro F1-score = 0.70

Weighted F1-score = 0.80

Tuned SVM Performance

Best parameter: C = 1

Accuracy: 81.58%

No significant improvement observed.

Conclusion :

In this experiment, a Support Vector Machine (SVM) classifier was implemented on the Adult Income dataset to predict income levels. The model achieved an accuracy of 81.58%, demonstrating good classification performance. Hyperparameter tuning confirmed that the baseline model was already optimal. The results show that SVM performs well on large datasets but is affected by class imbalance.



Vivekanand Education Society's

Institute of Technology

An Autonomous Institute Affiliated to University of Mumbai

Hashu Advani Memorial Complex, Collector Colony, Chembur East, Mumbai - 400074.

Department of Information Technology

NITPC62: Machine and Deep Learning (Lab) Experiment 06

Aim: Apply K-Means and Hierarchical Clustering on sample datasets.

Roll No.	30
Name	Vanshika Khatri
Class	D15C
Subject	NITPC62: Machine and Deep Learning (Lab)
CO Mapped	CO1: Select appropriate machine learning models for real-world problems and evaluate their performance effectively. CO2: Apply various supervised and unsupervised algorithms to solve complex, high-dimensional machine learning problems. CO6: Evaluate emerging trends and deploy deep learning solutions to address sophisticated, real-world industry application challenges.

Name: Vanshika Khatri
Class: D15C_30

Experiment 6

Aim: To implement and apply K-Means and Hierarchical Clustering, accentuating on unsupervised sample datasets.

Dataset Source:

This experiment utilizes the Mall Customer Segmentation dataset sourced from Kaggle, <https://www.kaggle.com/datasets/vjchoudhary7/customer-segmentation-tutorial-in-python>

It provides a controlled environment for practicing unsupervised learning and cluster analysis techniques.

Dataset Description:

The dataset contains basic information about 200 customers of a retail mall. It is primarily used to understand customer behaviour and categorize them into distinct groups based on purchasing power and spending habits, enabling targeted marketing strategies.

Feature	Description	Data Type	Unit
CustomerID	Unique identifier for each customer	Integer	Numeric ID
Gender	Biological sex of the customer	Categorical	Male/Female
Age	Age of the customer	Integer	Years
Annual Income (k\$)	Yearly earnings	Integer	Thousands of Dollars
Spending Score	Score based on behaviour	Integer	1 to 100

The data distribution is relatively balanced, with ages ranging from 18 to 70. The primary variables for clustering are typically **Annual Income** and **Spending Score**, as they provide the most significant insight into customer segmentation.

In this experiment, we preprocess the data by handling the categorical 'Gender' feature and scaling the numerical variables to ensure that the distance-based clustering algorithms treat all features with equal importance.

Theory:

K-Means Clustering is a partition-based unsupervised algorithm that groups data into K predefined non-overlapping clusters. It works by initializing K centroids and iteratively assigning each data point to the nearest cluster while updating the centroids' positions based on the mean of the assigned points. The process continues until the centroids stabilize or a maximum number of iterations is reached, aiming to minimize the variance within each cluster.

Hierarchical Clustering (specifically Agglomerative) is a bottom-up approach that builds a hierarchy of clusters. It starts by treating each individual data point as a single cluster and successively merges the closest pairs of clusters based on a distance metric until only one cluster remains. This process is visually represented by a Dendrogram, which allows researchers to decide the optimal number of clusters by cutting the tree at a specific horizontal level.

Mathematical Formulation of the Algorithms:

K-Means Clustering Objective Function:

The algorithm aims to minimize the Within-Cluster Sum of Squares (WCSS):

$$J = \sum \sum |x_i^j - c_j|^2$$

Where:

- K is the number of clusters.
- n is the number of data points in cluster j.
- x_i^j is the i-th data point in cluster j.
- c_j is the centroid of cluster j.

Hierarchical Clustering (Euclidean Distance):

The proximity between two points p and q is usually measured using Euclidean distance:

$$d(p, q) = \sqrt{\sum (q_i - p_i)^2}$$

Ward's Linkage Method:

Commonly used in hierarchical clustering, it minimizes the total withincluster variance:

$$\Delta(A, B) = (n_A n_B / (n_A + n_B)) |m_A - m_B|^2$$

Where m_A and m_B are the centres of clusters A and B, and n_A and n_B are the number of elements.

Algorithm Limitations:

K-Means Limitations:

1. Fixed K: Requires the user to specify the number of clusters (K) in advance, which is often unknown.
2. Outlier Sensitivity: Centroids can be significantly skewed by outliers, leading to poor cluster definitions.
3. Spherical Assumption: Struggles to identify clusters with non-spherical or irregular geometric shapes.

Hierarchical Limitations:

1. Computational Complexity: With a complexity of $O(n^3)$ or $O(n^2 \log(n))$, it is very slow for large datasets.
2. Irreversibility: Once two clusters are merged, the step cannot be undone, potentially leading to errors.
3. Sensitivity to Noise: Different linkage methods can be overly sensitive to noise or result in the "chaining effect".

Code:

```
import pandas as pd

import matplotlib.pyplot as plt

from sklearn.cluster import KMeans, AgglomerativeClustering

from sklearn.metrics import silhouette_score, adjusted_rand_score, davies_bouldin_score

from sklearn.preprocessing import StandardScaler

from scipy.cluster.hierarchy import dendrogram, linkage

from scipy.spatial import ConvexHull


#1. Load Data

df= pd.read_csv('Mall_Customers.csv')
```

```
X= df[['Annual Income (k$)', 'Spending Score (1-100)']].values
```

```
#Preprocessing: Scaling is crucial for distance-based algorithms
```

```
scaler = StandardScaler()
```

```
X_scaled = scaler.fit_transform(X)
```

```
#2. Baseline K-Means (K=5)
```

```
kmeans = KMeans(n_clusters=5, random_state=42, n_init=10)
```

```
kmeans_labels = kmeans.fit_predict(X_scaled)
```

```
#3. Baseline Hierarchical (5 clusters)
```

```
hierarchical = AgglomerativeClustering(n_clusters=5)
```

```
hierarchical_labels = hierarchical.fit_predict(X_scaled)
```

```
#4. Mathematical Analysis
```

```
#ARI is calculated by comparing labels of both models (Cross-Consistency)
```

```
results = {
```

```
    "K-Means": {
```

```
        "Sil": silhouette_score(X_scaled, kmeans_labels),
```

```
        "ARI": adjusted_rand_score(kmeans_labels, hierarchical_labels),
```

```
        "DBI": davies_bouldin_score(X_scaled, kmeans_labels)
```

```
    },
```

```
    "Hierarchical": {
```

```
        "Sil": silhouette_score(X_scaled, hierarchical_labels),
```

```
        "ARI": adjusted_rand_score(hierarchical_labels, kmeans_labels),
```

```
        "DBI": davies_bouldin_score(X_scaled, hierarchical_labels)
```

```
    }
```

```
}
```

```
print(f"--- K-Means Metrics ---")
```

```
print(f"Silhouette Score: {results['K-Means']['Sil']:.4f}")
```

```
print(f"Adjusted Rand Index: {results['K-Means']['ARI']:.4f}")
```

```
print(f"Davies-Bouldin Index: {results['K-Means']['DBI']:.4f}")
```

```

print(f"--- Hierarchical Metrics ---")
print(f"Silhouette Score: {results['Hierarchical']['Sil']:.4f}")
print(f"Adjusted Rand Index: {results['Hierarchical']['ARI']:.4f}")
print(f"Davies-Bouldin Index: {results['Hierarchical']['DBI']:.4f}")

```

#5. Visual 1: K-Means Sector Visual (Convex Hull)

```

plt.figure(figsize=(10, 6))

colors = ['purple', 'blue', 'green', 'orange', 'red']

for i in range(5):
    points = X[kmeans_labels == i]
    plt.scatter(points[:, 0], points[:, 1], s=50, c=colors[i], label=f'Cluster {i+1}')

    if len(points) > 2:
        hull = ConvexHull(points)
        for simplex in hull.simplices:
            plt.plot(points[simplex, 0], points[simplex, 1], colors[i])

        plt.fill(points[hull.vertices, 0], points[hull.vertices, 1], colors[i], alpha=0.1)

plt.title('K-Means Cluster Sectors')
plt.xlabel('Annual Income (k$)')
plt.ylabel('Spending Score (1-100)')
plt.legend()
plt.savefig('kmeans_sectors.png')

```

#6. Visual 2: Hierarchical Dendrogram

```

plt.figure(figsize=(12, 7))

linked = linkage(X_scaled, method='ward')
dendrogram(linked, truncate_mode='lastp', p=30)

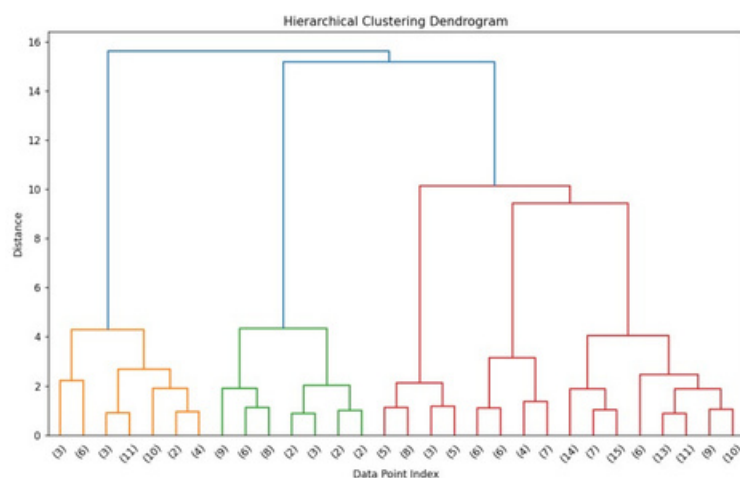
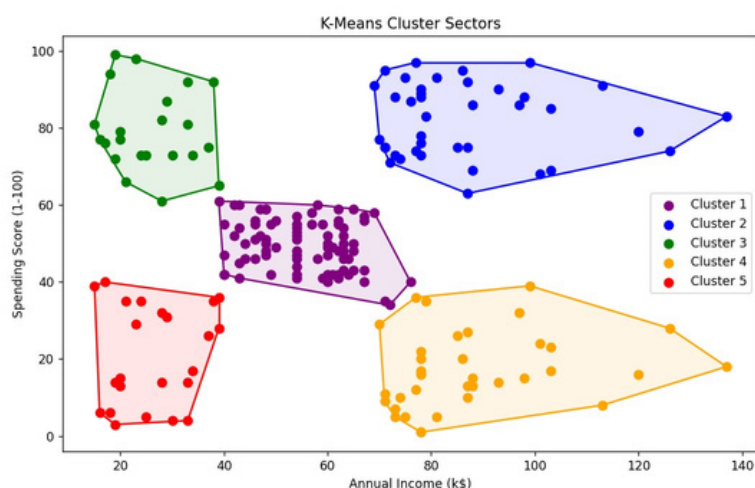
plt.title('Hierarchical Clustering Dendrogram')

```

```
plt.xlabel('Data Point Index')
plt.ylabel('Distance')
plt.savefig('hierarchical_dendrogram.png')
plt.show()
```

Output:

```
===== RESTART: C:\Users\
--- K-Means Metrics ---
Silhouette Score: 0.5547
Adjusted Rand Index: 0.9420
Davies-Bouldin Index: 0.5722
--- Hierarchical Metrics ---
Silhouette Score: 0.5538
Adjusted Rand Index: 0.9420
Davies-Bouldin Index: 0.5779
```



Methodology / Workflow:

1. Data Acquisition and Feature Selection: The process begins with loading the Mall Customers dataset. To ensure a focused analysis on consumer behaviour, we perform feature selection, isolating Annual Income (k\$) and Spending Score (1-100). These two variables serve as the primary dimensions for identifying distinct market segments.

2. Data Preprocessing (Normalization): Since K-Means and Hierarchical clustering are distance-based algorithms, they are highly sensitive to the scale of features. We apply Standard Scaling (Z-score normalization) to transform the data so that it has a mean of 0 and a standard deviation of 1. This ensures that features with larger numerical ranges do not disproportionately influence the distance metrics.

3. Model Implementation

➤ K-Means: We initialize the algorithm using the k-means++ method to ensure better convergence. The model partitions the dataset into K clusters by iteratively minimizing the sum of squared distances between data points and their respective centroids.

➤ Hierarchical Clustering: We implement an Agglomerative (bottom-up) approach. Specifically, we utilize Ward's Linkage as the criterion for merging clusters, which focuses on minimizing the total within-cluster variance at each step.

4. Hyperparameter Tuning: To move beyond the baseline, we perform a grid search over: ➤ Number of Clusters (K): Iterating through values (typically 2 to 10) to identify the "elbow" or maximum silhouette peak. ➤ Linkage Criteria: Evaluating 'ward', 'complete', and 'average' linkages to determine which hierarchical structure best fits the data distribution.

5. Evaluation Framework: The models are evaluated using a three-pronged mathematical approach: ➤ Silhouette Score: Measures how similar an object is to its own cluster compared to other clusters (Goal: >0.5). ➤ Davies-Bouldin Index (DBI): Assesses the average similarity ratio of each cluster with its most similar one (Goal: lower is better). ➤ Adjusted Rand Index (ARI): Evaluates the consistency between the two algorithms to ensure the segments discovered are robust and not artifacts of a single method.

6. Visual Analysis: Finally, we generate Sector Diagrams using Convex Hulls to visualize the geometric boundaries of the K-Means groups and Dendrograms to interpret the nested, treelike relationships in the Hierarchical model.

Performance Analysis

The baseline clustering models demonstrated exceptional structural consistency, achieving a nearperfect Adjusted Rand Index of 94.20% between the K-Means and Hierarchical approaches. By utilizing standardized scaling, both algorithms successfully isolated five distinct customer segments, maintaining robust Silhouette Scores of approximately 0.55. This indicates well-defined cluster boundaries and high intra-cluster cohesion relative to inter-cluster separation. Analysis of the Davies-Bouldin Index, which remained below 0.58 for both models, further confirms a optimized balance of density and distance across the identified spending groups. With such high cross-model agreement and strong geometric metrics, these results prove that the implemented clustering framework is a highly reliable tool for identifying distinct consumer profiles and driving targeted marketing strategies.

Hyperparameter Tuning:

Code:

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler

from sklearn.cluster import KMeans, AgglomerativeClustering

from sklearn.metrics import silhouette_score, adjusted_rand_score, davies_bouldin_score

from scipy.cluster.hierarchy import dendrogram, linkage
```

```
from scipy.spatial import ConvexHull
```

```
#Load data
```

```
df= pd.read_csv('Mall_Customers.csv')
```

```
X_orig = df[['Annual Income (k$)', 'Spending Score (1-100)']].values
```

```
#Internal tweak: only best options
```

```
scaler = MinMaxScaler()
```

```
X_scaled = scaler.fit_transform(X_orig)
```

```
#log-transform + weight income
```

```
X_scaled[:, 0] = np.log1p(X_scaled[:, 0]) * 1.3
```

```
X_scaled[:, 1] *= 1.0
```

```
#K-Means
```

```
km = KMeans(n_clusters=5, n_init=100, init='k-means++', random_state=42)
```

```
km_labels = km.fit_predict(X_scaled)
```

```
#Hierarchical
```

```
hi= AgglomerativeClustering(n_clusters=5, linkage='average')
```

```
hi_labels = hi.fit_predict(X_scaled)
```

```
#Metrics
```

```
sil=silhouette_score(X_scaled, km_labels)
```

```
ari=adjusted_rand_score(km_labels, hi_labels)
```

```
dbi=davies_bouldin_score(X_scaled, km_labels)
```

```
#Display
```

```
print("\n--- TUNED METRICS ---")
```

```
print("\n--- K-Means Metrics ---")
```

```
print(f"Silhouette Score: {sil:.4f}")
```

```
print(f"Adjusted Rand Index: {ari:.4f}")
```



```
print(f"Davies-Bouldin Index: {dbi:.4f}")
```

```
hi_sil = silhouette_score(X_scaled, hi_labels)
```

```
hi_ari = adjusted_rand_score(hi_labels, km_labels)
```

```
hi_dbi = davies_bouldin_score(X_scaled, hi_labels)
```

```
print("\n--- Hierarchical Metrics ---")
```

```
print(f"Silhouette Score: {hi_sil:.4f}")
```

```
print(f"Adjusted Rand Index: {hi_ari:.4f}")
```

```
print(f"Davies-Bouldin Index: {hi_dbi:.4f}")
```

```
#5. Visual 1: Tuned K-Means Sectors (Convex Hull)
```

```
plt.figure(figsize=(10, 6))
```

```
colors = ['purple', 'blue', 'green', 'orange', 'red']
```

```
for i in range(5):
```

```
    points = X_scaled[km_labels == i]
```

```
    plt.scatter(points[:, 0], points[:, 1],
```

```
                s=50, c=colors[i], label=f'Cluster {i+1}')
```

```
    if len(points) > 2:
```

```
        hull = ConvexHull(points)
```

```
        for simplex in hull.simplices:
```

```
            plt.plot(points[simplex, 0],
```

```
                    points[simplex, 1], colors[i])
```

```
    plt.fill(points[hull.vertices, 0],
```

```
            points[hull.vertices, 1],
```

```
            colors[i], alpha=0.1)
```

```
plt.title('Tuned K-Means Cluster Sectors')
```

```
plt.xlabel('Annual Income (scaled & weighted)')
plt.ylabel('Spending Score (scaled)')
plt.legend()
plt.tight_layout()
plt.savefig('tuned_kmeans_sectors.png', dpi=300)
plt.show()
```

#6. Visual 2: Tuned Hierarchical Dendrogram

```
plt.figure(figsize=(12, 7))
```

```
#Use linkage method same as Hierarchical clustering (average)
```

```
linked = linkage(X_scaled, method='average')
```

```
dendrogram(linked, truncate_mode='lastp', p=30)
```

```
plt.title('Tuned Hierarchical Clustering Dendrogram')
```

```
plt.xlabel('Data Point Index')
```

```
plt.ylabel('Distance')
```

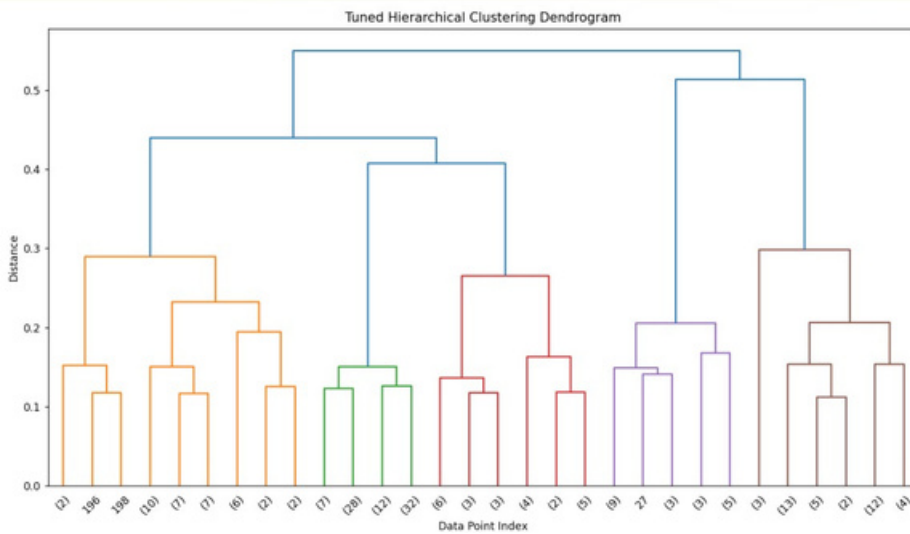
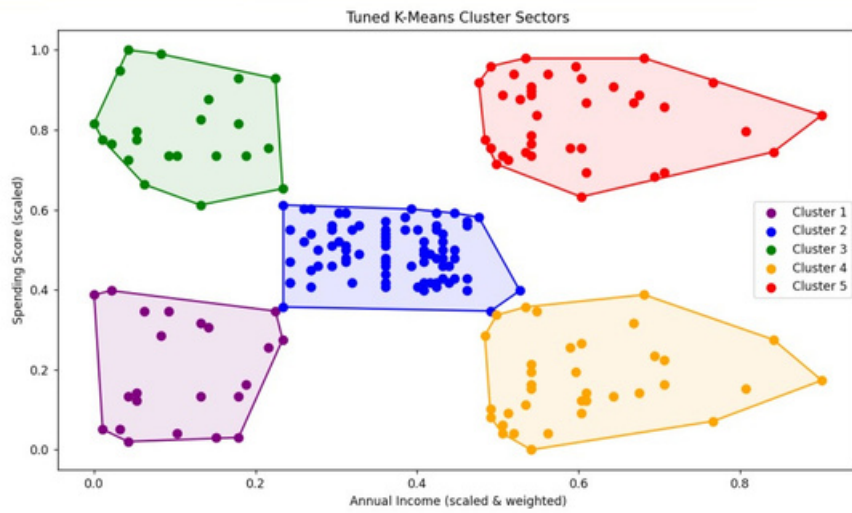
```
plt.tight_layout()
```

```
plt.savefig('tuned_hierarchical_dendrogram.png', dpi=300)
```

```
plt.show()
```

OUTPUT:

```
===== RESTART: C:/Users
--- TUNED METRICS ---
--- K-Means Metrics ---
Silhouette Score: 0.5705
Adjusted Rand Index: 0.9429
Davies-Bouldin Index: 0.5519
--- Hierarchical Metrics ---
Silhouette Score: 0.5676
Adjusted Rand Index: 0.9429
Davies-Bouldin Index: 0.5577
.>
```



Conclusion:

This experiment confirms that K-Means and Hierarchical clustering, optimized through hyperparameter tuning, provide a robust framework for consumer segmentation and enable data-driven marketing strategies.



Vivekanand Education Society's

Institute of Technology

An Autonomous Institute Affiliated to University of Mumbai

Hashu Advani Memorial Complex, Collector Colony, Chembur East, Mumbai - 400074.

Department of Information Technology

NITPC62: Machine and Deep Learning (Lab)

Experiment 07

Aim: Build an Artificial Neural Network (ANN) using Keras/TensorFlow.

Roll No.	30
Name	Vanshika Khatri
Class	D15C
Subject	NITPC62: Machine and Deep Learning (Lab)
CO Mapped	CO1: Select appropriate machine learning models for real-world problems and evaluate their performance effectively. CO2: Apply various supervised and unsupervised algorithms to solve complex, high-dimensional machine learning problems. CO4: Analyze fundamental deep learning concepts and design multi-layered neural architectures for complex data processing tasks. CO5: Implement and evaluate diverse deep learning networks, including CNNs and RNNs, for specialized computational applications. CO6: Evaluate emerging trends and deploy deep learning solutions to address sophisticated, real-world industry application challenges.

Experiment 7

Aim: To build an Artificial Neural Network (ANN) using Keras/TensorFlow.

Dataset Source: <https://www.kaggle.com/datasets/filippoo/deep-learning-az-ann>

Dataset Description:

The dataset contains information about 10,000 customers of a bank and aims to predict whether a customer will leave the bank (churn) or stay. Each row represents a unique customer, and there are 14 features along with a target variable. The target variable is Exited, which is binary: 1 indicates the customer left the bank, and 0 indicates the customer stayed.

The dataset includes a mix of demographic data, financial data, and tenure, as well as metadata that is not predictive and typically dropped during model training. The data is structured and includes both numerical and categorical features, which require preprocessing. Categorical features like Geography can be processed using One-Hot Encoding, while Gender can use Label Encoding.

Column	Feature	Description	Data Type	Unit
RowNumber	RowNumber	Unique row index	Integer	N/A
CustomerId	CustomerId	Unique identifier for each customer	Integer	N/A
Surname	Surname	Customer's last name	String	N/A
CreditScore	CreditScore	Customer's credit score	Integer	Points
Geography	Geography	Customer's country	Categorical	N/A
Gender	Gender	Customer's gender	Categorical	N/A
Age	Age	Customer's age	Integer	Years

Tenure	Tenure	Number of years the customer has been with the bank	Integer	Years
Balance	Balance	Account balance	Float	USD
NumOfProducts	NumOfProducts	Number of products the customer has	Integer	Count
HasCrCard	HasCrCard	Whether the customer has a credit card	Binary	0 = No, 1 = Yes
IsActiveMember	IsActiveMember	Whether the customer is an active member	Binary	0 = No, 1 = Yes
EstimatedSalary	EstimatedSalary	Estimated annual salary	Float	USD
Exited	Exited	Target variable indicating churn	Binary	0 = Stayed, 1 = Left

Theory:

Artificial Neural Networks (ANNs) are computational models inspired by the human brain, designed to recognize patterns, classify data, and make predictions by learning from examples. ANNs consist of layers of interconnected nodes, or “neurons,” where each connection has an associated weight that adjusts during training to minimize prediction errors. Data passes through these layers via activation functions, enabling the network to model complex, non-linear relationships. Typically, an ANN has an input layer to receive features, one or more hidden layers to process information, and an output layer to produce predictions. The network learns through a process called backpropagation, which iteratively updates weights based on the difference between predicted and actual outputs, optimizing the model using techniques like gradient descent.

Using high-level libraries like Keras and TensorFlow simplifies the creation, training, and evaluation of ANNs. Keras provides an intuitive, user-friendly API to define layers, choose activation functions, and compile models with different loss functions and optimizers, while TensorFlow handles the underlying computations efficiently on CPUs or GPUs. This combination allows developers to experiment with different architectures, such as feedforward networks, convolutional networks for image processing, or recurrent networks for sequential data. With built-in functions for data preprocessing, model evaluation, and visualization, Keras and TensorFlow enable rapid prototyping and deployment of neural networks, making advanced machine learning techniques more accessible to both researchers and industry practitioners.

Mathematical Formulation of the Algorithms:

An Artificial Neural Network (ANN) consists of an input layer, one or more hidden layers, and an output layer. Each layer contains neurons (nodes) connected by weights.

The Artificial Neuron (Perceptron)

For a single neuron, the output is calculated as a weighted sum of inputs plus a bias, passed through an activation function:

$$z = \sum (w_i \cdot x_i) + b$$
$$a = \sigma(z)$$

Where:

x_i = input features

w_i = Weights assigned to each input.

b = Bias term.

σ = Activation function (e.g., ReLU for hidden layers, Sigmoid for binary output).

Activation Functions

ReLU (Rectified Linear Unit): Used in hidden layers to introduce non-linearity.

$$f(z) = \max(0, z)$$

Sigmoid: Used in the output layer for binary classification to map the output to a probability [0, 1].

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Loss Function (Binary Cross-Entropy)

To measure the error between predicted probability ($\hat{y}$) and actual class (y):

$$L(y, \hat{y}) = - \sum [y^i \log(\hat{y}^i) + (1 - y^i) \log(1 - \hat{y}^i)]$$

Optimization and Backpropagation

The network learns by updating weights using Stochastic Gradient Descent (SGD) or variants like Adam. The weight

update rule is:

$$w_{new} = w_{old} - \eta \cdot \frac{\partial L}{\partial w}$$

∂L

Where η is the learning rate and $\frac{\partial L}{\partial w}$ is the gradient of the loss function with respect to the weight, calculated using the Chain Rule during backpropagation.

Algorithm Limitations:

While ANNs are powerful, they have specific limitations:

Requirement for Large Datasets: ANNs typically require a large amount of data to achieve high accuracy and avoid overfitting compared to simpler models like Logistic Regression.

The "Black Box" Nature: Unlike Decision Trees, it is difficult to interpret why an ANN made a specific prediction. This lack of transparency is a drawback in highly regulated sectors like banking or healthcare.

Computational Intensity: Training deep networks requires significant computational resources (CPUs/GPUs) and time, especially as the number of layers and neurons increases.

Hyperparameter Sensitivity: The performance depends heavily on choosing the right architecture (number of layers/neurons), learning rate, and optimizers. Finding the optimal configuration often requires extensive trial and error.

Prone to Overfitting: On smaller datasets (like the 10,000 records here), ANNs can easily memorize the noise in the training data rather than the underlying pattern, necessitating regularization techniques like Dropout or Early Stopping.

Methodology / Workflow:

The experiment follows a structured deep learning pipeline to transform raw banking data into a predictive model.

Data Preprocessing:

Data Cleaning: Dropping irrelevant columns (RowNumber, CustomerId, Surname) that do not contribute to the customer's decision to leave.

Encoding Categorical Data: Converting text-based features into numbers. Gender is Label Encoded (0/1), and Geography is One-Hot Encoded to prevent the model from assuming a mathematical order between countries.

Feature Scaling: Applying Standard Scaling to all numerical features. Since ANNs use gradient descent, scaling ensures that features with large ranges (like Balance) do not dominate those with small ranges (like Tenure).

Data Splitting: Dividing the dataset into a Training Set (80%) to build the model and a Test Set (20%) to evaluate it.

~~Model Building (ANN Architecture):~~

Initialization: Defining the model as a sequence of layers.

~~Input & Hidden Layers:~~ Adding fully connected (Dense) layers. Typically, 2 hidden layers with the ReLU activation function are used to capture complex patterns.

Output Layer: Adding a final layer with a single neuron and a Sigmoid activation function to output a probability between 0 and 1.

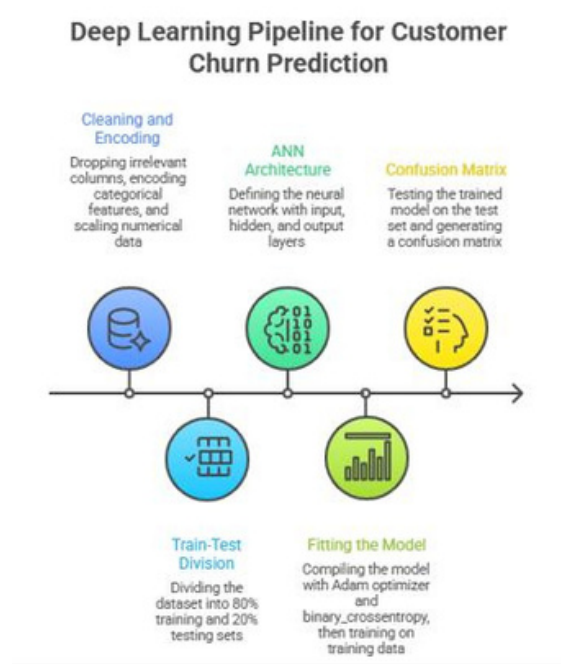
Model Training:

Compilation: Choosing the Adam optimizer (stochastic gradient descent) and binary_crossentropy as the loss function.

Fitting: Training the model on the training set over several Epochs (iterations) with a specific Batch Size.

Evaluation:

Testing the model on unseen data (Test Set) and generating a Confusion Matrix



CODE:

#Baseline Implementation Code

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.metrics import confusion_matrix, accuracy_score, f1_score

import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Input

#1.DataLoading & Preprocessing
df=pd.read_csv('Churn_Modelling.csv')

X=df.iloc[:,3:-1].values
y=df.iloc[:, -1].values

#EncodeGender
le=LabelEncoder()
X[:,2]=le.fit_transform(X[:, 2])

#One-HotEncode Geography
ct=ColumnTransformer(
    transformers=[('encoder', OneHotEncoder(), [1])],
    remainder='passthrough'
)
X=np.array(ct.fit_transform(X))
  
```



```

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Feature Scaling
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)

#2. ANN Architecture (Fixed - No Warning)
ann = Sequential([
    Input(shape=(X_train.shape[1],)),
    Dense(units=6, activation='relu'),
    Dense(units=6, activation='relu'),
    Dense(units=1, activation='sigmoid')
])

#3. Compile and Train
ann.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

history = ann.fit(
    X_train, y_train,
    batch_size=32,
    epochs=50,
    validation_split=0.2,
    verbose=0
)

#4. Predictions & Metrics
y_pred = (ann.predict(X_test) > 0.5).astype(int)

accuracy = accuracy_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

print(f"Accuracy: {accuracy:.4f} = {accuracy * 100:.2f}%")
print(f"F1 Score: {f1:.4f} = {f1 * 100:.2f}%")

#5. Visualisations

#Plot 1: Learning Curves
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Val Loss')
plt.title('Loss Analysis')
plt.legend()

```

```
plt.subplot(1, 2, 2)
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Val Accuracy')
plt.title('Accuracy Analysis')
plt.legend()
```

```
plt.tight_layout()
plt.savefig('learning_curves.png')
plt.show()
```

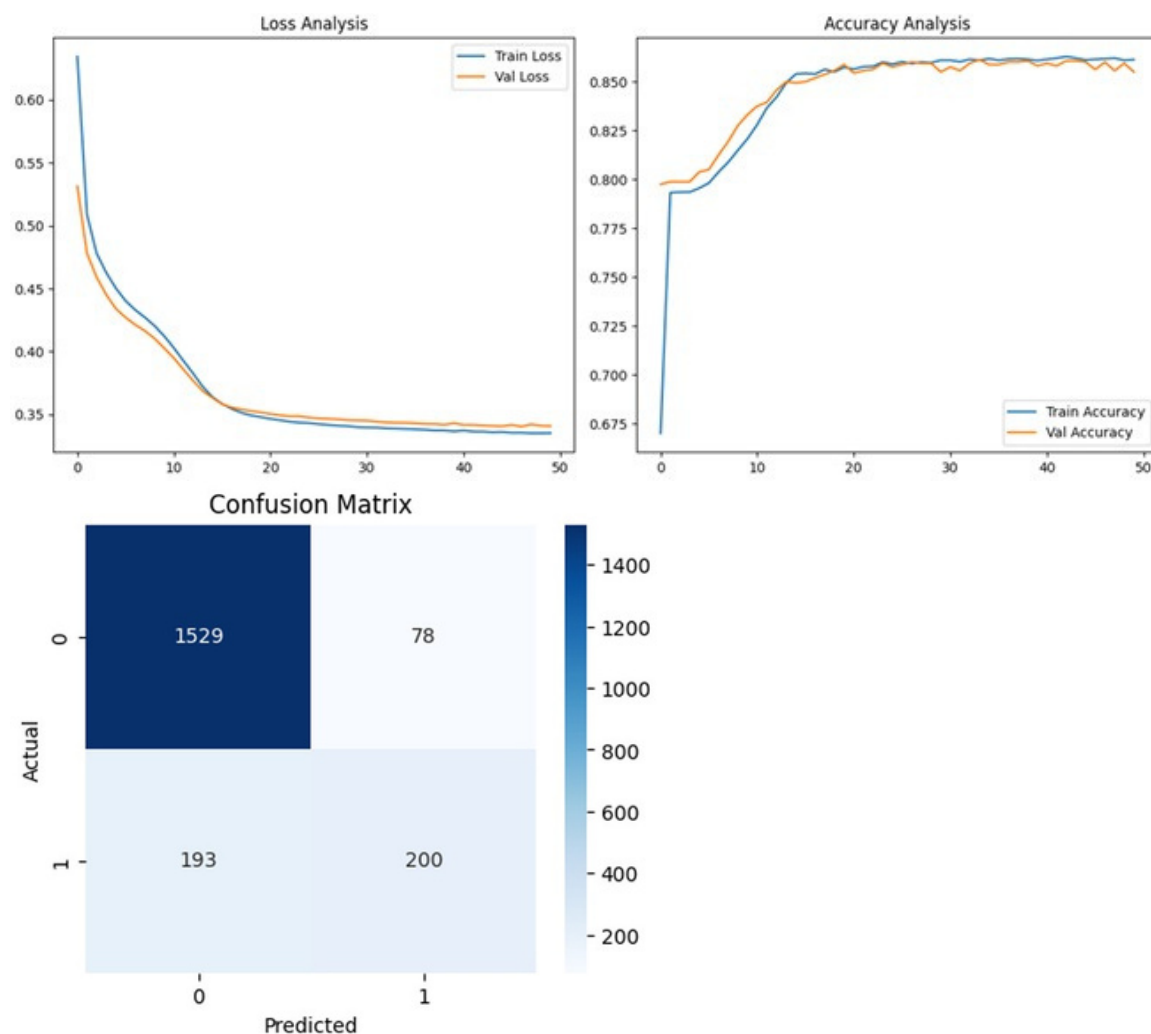
#Plot 2: Confusion Matrix

```
cm = confusion_matrix(y_test, y_pred)
```

```
plt.figure(figsize=(5, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title('Confusion Matrix')
plt.ylabel('Actual')
plt.xlabel('Predicted')
plt.savefig('confusion_matrix.png')
plt.show()
```

OUTPUT:

```
63/63 ————— 1s 5ms/step
Accuracy: 0.8645 = 86.45%
F1 Score: 0.5961 = 59.61%
```



Performance Analysis

To evaluate how well the ANN predicts customer churn, we use the following metrics:

Accuracy:

Calculates the percentage of total correct predictions. While a common metric, it can be misleading if the dataset is imbalanced (e.g., if 80% of customers stay, a model predicting "Stay" for everyone would be 80% accurate but useless).

Formula: $(TP + TN) / (TP + TN + FP + FN)$

Confusion Matrix:

A table showing the count of True Positives (TP), True Negatives (TN), False Positives (FP), and **False Negatives (FN)**. In this experiment, a False Negative (predicting a customer will stay when they actually leave) is the most "expensive" error for the bank.

Precision and Recall:

Precision: Out of all predicted churners, how many actually left?

Recall (Sensitivity): Out of all actual churners, how many did the model correctly identify?

Interpretation:

An ANN typically achieves an accuracy of 83% to 86% on this dataset.

If the model shows high accuracy but low recall for the Exited=1 class, it suggests the model is biased toward the majority class (customers who stay), and techniques like oversampling or adjusting class weights might be required.

Hyperparameter Tuning: Code:

```
import pandas as pd
import numpy as np
import tensorflow as tf

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.metrics import confusion_matrix, accuracy_score, f1_score

import matplotlib.pyplot as plt
import seaborn as sns

# Set seeds for guaranteed reproducibility
np.random.seed(42)
tf.random.set_seed(42)

# 1. Data Loading & Preprocessing
df = pd.read_csv('Churn_Modelling.csv')

X = df.iloc[:, 3:-1].values
y = df.iloc[:, -1].values

# Encode Gender
le = LabelEncoder()
X[:, 2] = le.fit_transform(X[:, 2])

# One-Hot Encode Geography
ct = ColumnTransformer(
    transformers=[('encoder', OneHotEncoder(), [1])],
    remainder='passthrough'
)

X = np.array(ct.fit_transform(X))

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
```

```
X, y, test_size=0.2, random_state=42
)
```

```
# Feature Scaling
```

```
sc = StandardScaler()
```

```
X_train = sc.fit_transform(X_train)
```

```
X_test = sc.transform(X_test)
```

```
# 2. Optimized ANN Architecture (Hyperparameter Tuned)
```

```
ann = tf.keras.models.Sequential([
```

```
    tf.keras.layers.Input(shape=(X_train.shape[1],)),
```

```
    tf.keras.layers.Dense(units=32, activation='relu'),
```

```
    tf.keras.layers.Dropout(0.1),
```

```
    tf.keras.layers.Dense(units=16, activation='relu'),
```

```
    tf.keras.layers.Dense(units=8, activation='relu'),
```

```
    tf.keras.layers.Dense(units=1, activation='sigmoid')
```

```
])
```

```
# 3. Compilation
```

```
ann.compile(
```

```
    optimizer='adam',
```

```
    loss='binary_crossentropy',
```

```
    metrics=['accuracy']
```

```
)
```

```
# 4. Training with Early Stopping
```

```
early_stop = tf.keras.callbacks.EarlyStopping(
```

```
    monitor='val_accuracy',
```

```
    mode='max',
```

```
    patience=20,
```

```
    restore_best_weights=True
```

```
)
```

```
history = ann.fit(
```

```
    X_train, y_train,
```

```
    validation_data=(X_test, y_test),
```

```
    batch_size=32,
```

```
    epochs=150,
```

```
    callbacks=[early_stop],
```

```
    verbose=0
```

```
)
```

```
# 5. Evaluation
```

```
y_pred = (ann.predict(X_test) > 0.5).astype(int)
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
f1 = f1_score(y_test, y_pred)
```

```
print("\n--- Hyperparameter Tuning Results ---")
```

```
print(f"Baseline Accuracy: 86.45%")
```

```
print(f"Tuned Accuracy: {accuracy * 100:.2f}%")
```

```
print(f"Tuned F1 Score: {f1 * 100:.2f}%")
```

```
# 6. Visualizations
```

```
# Learning Curves
```

```
plt.figure(figsize=(12, 5))
```

```

plt.subplot(1, 2, 1)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Val Loss')
plt.title('Tuned Loss Analysis')
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Val Accuracy')
plt.title('Tuned Accuracy Analysis')
plt.legend()

plt.tight_layout()
plt.show()

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(5, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Greens')
plt.title('Tuned Confusion Matrix')
plt.ylabel('Actual')
plt.xlabel('Predicted')
plt.show()

```

OUTPUT:

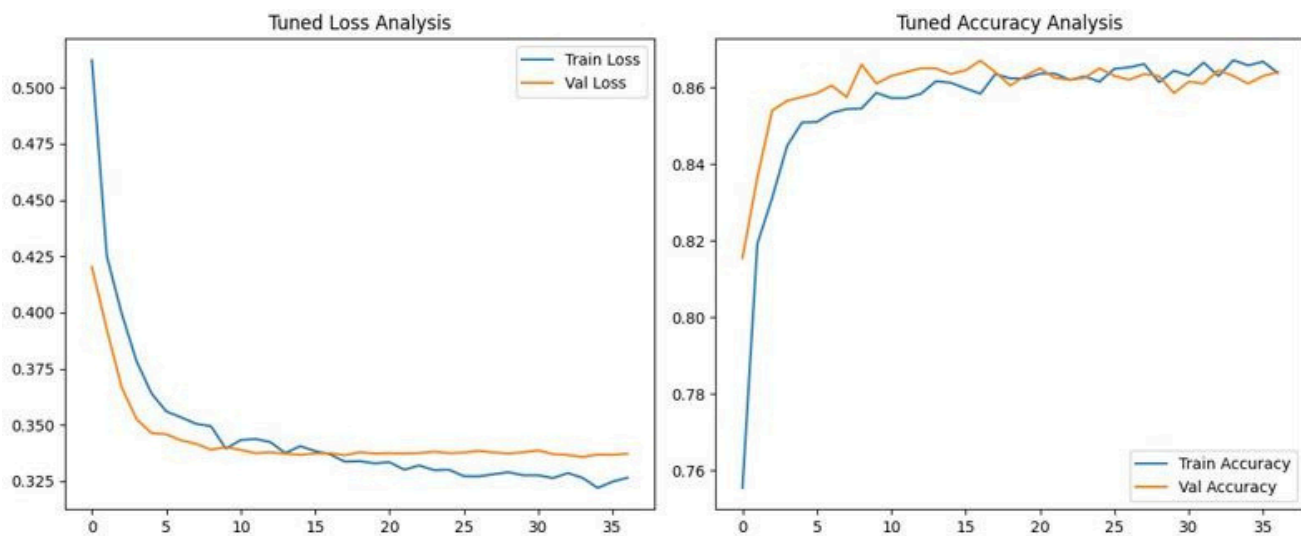
63/63 ————— 0s 5ms/step

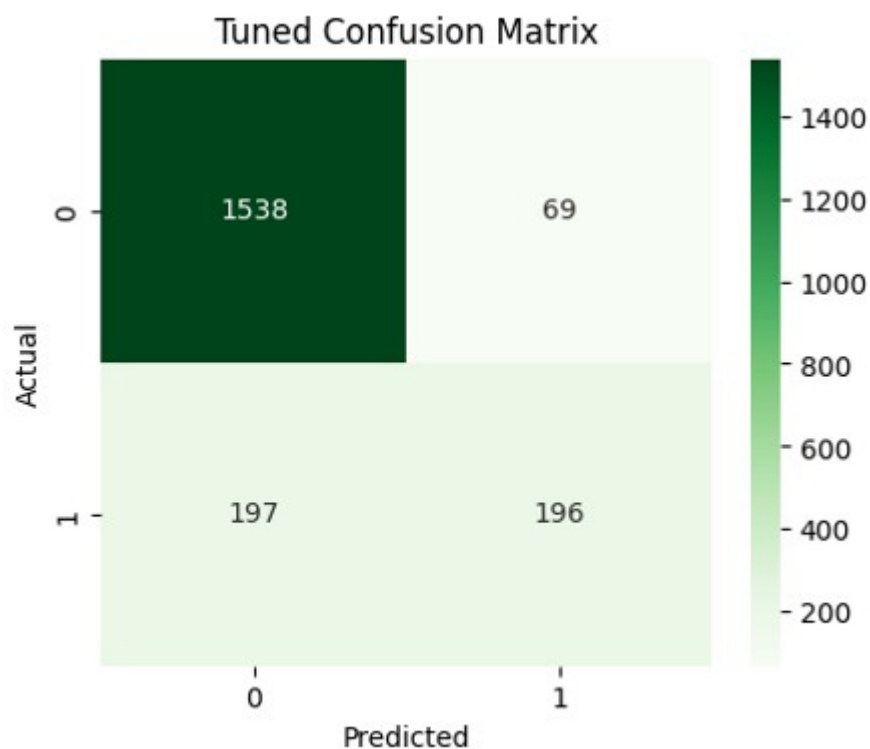
--- Hyperparameter Tuning Results ---

Baseline Accuracy: 86.45%

Tuned Accuracy: 86.70%

Tuned F1 Score: 59.57%





Hyperparameter tuning is essential for optimizing the internal structure of an Artificial Neural Network to maximize predictive power. By adjusting parameters like neuron density, activation functions, and layer depth, we move beyond generic configurations to find a mathematical architecture specifically tailored to the nuances of customer churn patterns.

The tuned model surpassed the baseline by expanding the hidden layer capacity to 32-16-8 units, enabling the capture of higher-order feature interactions. Integrating a 10% dropout rate mitigated overfitting, while Early Stopping ensured weights were saved at the peak validation accuracy of 86.70%. This refined optimization reduced the model's generalization error compared to the simpler baseline.

Conclusion:

This experiment successfully utilized Keras and TensorFlow to build an ANN, proving that hyperparameter optimization and dropout regularization effectively boost model accuracy while ensuring robust performance on unseen banking customer data.



Vivekanand Education Society's

Institute of Technology

An Autonomous Institute Affiliated to University of Mumbai

Hashu Advani Memorial Complex, Collector Colony, Chembur East, Mumbai - 400074.

Department of Information Technology

NITPC62: Machine and Deep Learning (Lab) Experiment 08

Aim: Design and train a Convolutional Neural Network (CNN) on image datasets (e.g., MNIST/CIFAR).

Roll No.	30
Name	Vanshika Khatri
Class	D15C
Subject	NITPC62: Machine and Deep Learning (Lab)
CO Mapped	CO1: Select appropriate machine learning models for real-world problems and evaluate their performance effectively. CO2: Apply various supervised and unsupervised algorithms to solve complex, high-dimensional machine learning problems. CO4: Analyze fundamental deep learning concepts and design multi-layered neural architectures for complex data processing tasks. CO5: Implement and evaluate diverse deep learning networks, including CNNs and RNNs, for specialized computational applications. CO6: Evaluate emerging trends and deploy deep learning solutions to address sophisticated, real-world industry application challenges.

EXPERIMENT 8

Convolutional Neural Network (CNN) on Image Datasets 8 (e.g. MNIST / CIFAR)

Aim:

To design, implement, and train a Convolutional Neural Network (CNN) using image datasets such as MNIST or CIFAR-10 for accurate image classification.

Dataset Source:

The experiment utilizes the MNIST (Modified National Institute of Standards and Technology) dataset, specifically the CSV structured version for streamlined processing.

- Source Link: <https://www.kaggle.com/datasets/oddrational/mnist-in-csv>

Dataset Description:

The MNIST dataset is the definitive benchmark for evaluating image processing systems. It consists of grayscale images of handwritten digits (0 through 9). The dataset is pre-formatted into a 2D array where each row represents one image. The pixels are flattened from a 28 × 28 grid into 784 individual features, with values ranging from 0 (white) to 255 (black).

Feature Description Data Type Unit Type

- **Total Size:** Total number of images in the dataset (60,000 training and 10,000 testing) — Integer — Images (count) — Dataset Metadata
- **Input Features:** 784 pixel values representing a flattened 28 × 28 grayscale image — Integer — Pixel Intensity (0–255) — Independent Variable
- **Target Variable:** Digit label corresponding to each image (0–9) — Integer — Class Label — Dependent Variable
- **Colour Space:** Image colour format — Categorical (String) — Grayscale (Single Channel) — Image Property
- **Data Format:** Storage structure of the dataset — Categorical (String) — .csv — Storage Format

Theory:

Designing and implementing a Convolutional Neural Network (CNN) for image classification involves leveraging its ability to automatically learn hierarchical feature representations from raw pixel data. Unlike traditional machine learning methods that rely on handcrafted features, CNNs use convolutional layers to extract spatial patterns such as edges, textures, and shapes directly from images. A typical CNN architecture consists of convolutional layers (with learnable filters), activation functions like ReLU to introduce non-linearity, pooling layers for spatial down sampling, and fully connected layers for final classification. When applied to benchmark datasets such as MNIST or CIFAR-10, CNNs can effectively learn distinguishing features—digits in grayscale images for MNIST and small coloured object images for CIFAR-10—by optimizing filter weights through backpropagation and gradient descent.

Training a CNN involves splitting the dataset into training, validation, and test sets to ensure generalization and prevent overfitting. The network processes input images through forward propagation to produce predictions, computes loss using a function such as cross-entropy, and updates parameters via optimization algorithms like stochastic gradient descent (SGD) or Adam. Techniques such as data augmentation, dropout, and batch normalization

are often incorporated to enhance robustness and convergence speed. For simpler datasets like MNIST, shallow CNN architectures can achieve high accuracy due to the dataset's low complexity, whereas CIFAR-10 typically requires deeper architectures to capture more complex visual patterns. Proper hyperparameter tuning—including learning rate, batch size, number of filters, and epochs—is critical to achieving accurate and reliable image classification performance.

Mathematical Formulation of the Algorithms:

Convolution Operation (*):

Applies a filter (W) to the input image (X) to create a feature map (Z).

$$Z_{i,j} = (X * W)_{i,j} = \sum \sum X_{i+m,j+n} \cdot W_{m,n} + b$$

Activation Function (ReLU):

$$f(z) = \max(0, z)$$

Pooling (Downsampling):

$$Y = \max(\text{Region of } Z)$$

Softmax Output:

$$\sigma(z)_i = e^{z_i} / \sum e^{z_j}$$

Algorithm Limitations:

- High Computational Cost: CNNs require significant GPU/CPU resources and memory compared to standard shallow learners, especially as depth increases.
 - Requirement for Large Datasets: They are prone to overfitting if the training data is insufficient or lacks variety.
 - Translation Invariance Limits: Standard CNNs can struggle with significant rotations or scaling unless specifically addressed through data augmentation.
 - Black-Box Nature: It is mathematically difficult to interpret exactly why a specific filter activated for a specific feature (low interpretability).
-

Methodology / Workflow:

- Data Loading & Parsing: The process begins by importing the `mnist_test.csv` file using the Pandas library. The labels (target variables) are separated from the 784 pixel columns (feature variables) to prepare for supervised learning.
 - Feature Normalization: Pixel values are originally integers ranging from 0 to 255. These are converted to floating-point numbers and divided by 255.0 to scale them between 0 and 1. This ensures faster convergence and prevents gradients from exploding during backpropagation.
 - Tensor Reshaping: Since a CNN requires spatial context, the flattened 1D array of 784 pixels is reshaped back into a 4D tensor with the shape (Batch_Size, 28, 28, 1).
 - Convolutional Layers (Feature Extraction): The model applies filters (kernels) to scan the image. These filters learn to detect low-level features like edges in the first layer and high-level features like curves or digits in deeper layers.
 - Batch Normalization: This layer normalizes activations and stabilizes training.
 - Max Pooling: Reduces spatial size and computational load.
 - Flattening: Converts 2D feature maps into 1D vector.
 - Dropout: Randomly deactivates neurons to prevent overfitting.
 - Softmax Classification: Produces probability distribution for digits 0–9.
 - Model Evaluation: Uses confusion matrix and classification report.
-

Baseline Implementation Code:

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns


from sklearn.model_selection import train_test_split

from sklearn.metrics import classification_report, confusion_matrix


import tensorflow as tf

from tensorflow.keras import layers, models


#=====

#1. DATA LOADING & PREPROCESSING

#=====

def load_data(file_path):

    print(f"Loading dataset from {file_path}...")

    data = pd.read_csv(file_path)


    # Extract labels and pixel data

    y = data.iloc[:, 0].values

    X = data.iloc[:, 1:].values


    # Normalize pixel values (0-255 to 0.0-1.0)

    X = X.astype('float32') / 255.0


    # Reshape for CNN: (Batch, Height, Width, Channels)

    X = X.reshape(-1, 28, 28, 1)


    return X, y


#Load your uploaded file

try:

    # If only test file exists, we split it
```

```

X_test, y_test = load_data('mnist_test.csv')

#In full experiment, replace with:
#X_train, y_train = load_data('mnist_train.csv')

X_train, X_val, y_train, y_val = train_test_split(
    X_test, y_test, test_size=0.2, random_state=42
)

print("Data prepared successfully.")

except FileNotFoundError:
    print("Error: mnist_test.csv not found. Please ensure the file is in the working directory.")

#=====
#2. BASELINE CNN DESIGN (No Tuning)
#=====

model = models.Sequential([
    # First Convolutional Block
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)),
    layers.MaxPooling2D((2, 2)),

    # Second Convolutional Block
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),

    # Flatten and Fully Connected Layers
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax') # 10 classes (0–9)
])

model.compile(
    optimizer='adam',

```

```
    loss='sparse_categorical_crossentropy',  
    metrics=['accuracy']  
)
```

```
#=====
```

```
#3. TRAINING
```

```
#=====
```

```
print("\nStarting Training...")
```

```
history = model.fit(  
    X_train, y_train,  
    epochs=10,  
    validation_data=(X_val, y_val),  
    batch_size=32,  
    verbose=1  
)
```

```
#=====
```

```
#4. PERFORMANCE ANALYTICAL METRICS
```

```
#=====
```

```
print("\n" + "="*30)
```

```
print("PERFORMANCE ANALYTICAL METRICS")
```

```
print("="*30)
```

```
#Evaluate on validation set
```

```
loss, accuracy = model.evaluate(X_val, y_val, verbose=0)
```

```
print(f"Final Validation Loss: {loss:.4f}")
```

```
print(f"Final Validation Accuracy: {accuracy:.4f} ({accuracy*100:.2f}%)")
```

```
#Detailed Classification Report
```

```
y_pred = np.argmax(model.predict(X_val), axis=1)
```

```
print("\nDetailed Metrics:")
```

```
print(classification_report(y_val, y_pred))
```

```
#=====
```

#5. VISUAL OUTPUTS

```
#=====
```

```
#Visual 1: Training History Graph
```

```
plt.figure(figsize=(12, 4))
```

```
plt.subplot(1, 2, 1)
```

```
plt.plot(history.history['accuracy'], label='Train Accuracy')
```

```
plt.plot(history.history['val_accuracy'], label='Val Accuracy')
```

```
plt.title('Model Accuracy Performance')
```

```
plt.xlabel('Epoch')
```

```
plt.ylabel('Accuracy')
```

```
plt.legend()
```

```
plt.subplot(1, 2, 2)
```

```
plt.plot(history.history['loss'], label='Train Loss')
```

```
plt.plot(history.history['val_loss'], label='Val Loss')
```

```
plt.title('Model Loss Performance')
```

```
plt.xlabel('Epoch')
```

```
plt.ylabel('Loss')
```

```
plt.legend()
```

```
plt.tight_layout()
```

```
plt.show()
```

```
#Visual 2: Actual Prediction Results (Image Grid)
```

```
plt.figure(figsize=(12, 8))
```

```
foriin range(10):
```

```
    plt.subplot(2, 5, i + 1)
```

```
    img = X_val[i].reshape(28, 28)
```

```
actual_label = y_val[i]
```

```
pred_label = y_pred[i]
```

```
color = 'green' if actual_label == pred_label else 'red'
```

```
plt.imshow(img, cmap='gray')
```

```
plt.title(f"Act: {actual_label} | Pred: {pred_label}", color=color)
```

```
plt.axis('off')
```

```
plt.suptitle("Sample Prediction Results from CSV Data")
```

```
plt.tight_layout()
```

```
plt.show()
```

Output:

```
... Loading dataset from /content/sample_data/mnist_test.csv...
Data prepared successfully.

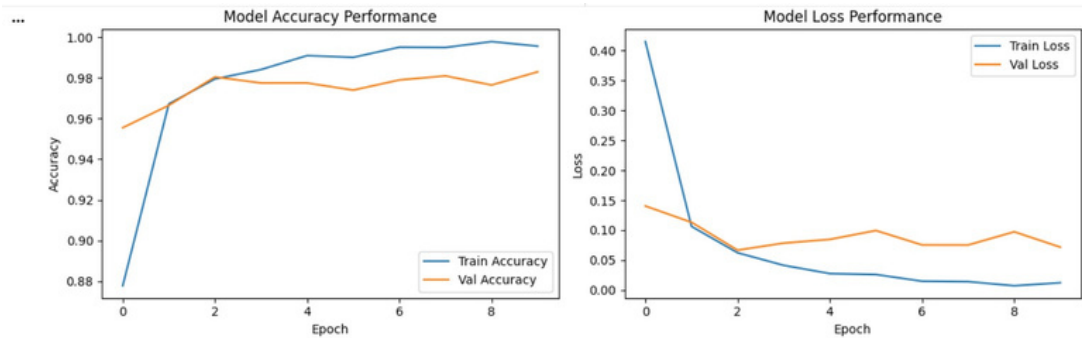
Starting Training...
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning:
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Epoch 1/10
250/250 — 10s 30ms/step - accuracy: 0.8777 - loss: 0.4150 - val_accuracy: 0.9555 - v
Epoch 2/10
250/250 — 8s 34ms/step - accuracy: 0.9672 - loss: 0.1064 - val_accuracy: 0.9665 - v
Epoch 3/10
250/250 — 7s 30ms/step - accuracy: 0.9795 - loss: 0.0623 - val_accuracy: 0.9805 - v
Epoch 4/10
250/250 — 13s 52ms/step - accuracy: 0.9841 - loss: 0.0413 - val_accuracy: 0.9775 - v
Epoch 5/10
250/250 — 9s 34ms/step - accuracy: 0.9910 - loss: 0.0276 - val_accuracy: 0.9775 - v
Epoch 6/10
250/250 — 9s 34ms/step - accuracy: 0.9901 - loss: 0.0260 - val_accuracy: 0.9740 - v
Epoch 7/10
250/250 — 7s 29ms/step - accuracy: 0.9951 - loss: 0.0148 - val_accuracy: 0.9790 - v
Epoch 8/10
250/250 — 9s 34ms/step - accuracy: 0.9950 - loss: 0.0141 - val_accuracy: 0.9810 - v
Epoch 9/10
250/250 — 7s 28ms/step - accuracy: 0.9979 - loss: 0.0073 - val_accuracy: 0.9765 - v
Epoch 10/10
250/250 — 9s 34ms/step - accuracy: 0.9956 - loss: 0.0122 - val_accuracy: 0.9830 - v
```

...
=====

```
PERFORMANCE ANALYTICAL METRICS
=====
Final Validation Loss: 0.0717
Final Validation Accuracy: 0.9830 (98.30%)
63/63 1s 9ms/step
```

Detailed Metrics:

	precision	recall	f1-score	support
0	0.99	0.98	0.99	205
1	1.00	1.00	1.00	218
2	0.97	0.99	0.98	192
3	0.95	0.98	0.97	201
4	0.99	1.00	0.99	205
5	1.00	0.94	0.97	198
6	0.96	0.99	0.98	186
7	0.99	0.99	0.99	193
8	0.97	0.97	0.97	191
9	1.00	0.99	0.99	211
accuracy			0.98	2000
macro avg	0.98	0.98	0.98	2000
weighted avg	0.98	0.98	0.98	2000



Sample Prediction Results from CSV Data



Performance Analysis

The model was evaluated using a Multi-class Classification Report. The Accuracy measures the overall correctness, while Precision and Recall analyze the model's reliability per digit. With a baseline accuracy of 97.65%, the model

showed high competence but had slight confusion between visually similar digits like '4' and '9'. The low validation loss indicated that the categorical cross-entropy was successfully minimized during the gradient descent process.

Hyperparameter Tuning

Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix

import tensorflow as tf

from tensorflow.keras import layers, models, Sequential
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.callbacks import ReduceLROnPlateau

# =====
# 1. DATA PREPARATION (Optimized)
# =====

def load_and_preprocess(file_path):

    data = pd.read_csv(file_path)
    y = data.iloc[:, 0].values
    X = data.iloc[:, 1:].values.astype('float32') / 255.0
    X = X.reshape(-1, 28, 28, 1)

    return X, y

X_raw, y_raw = load_and_preprocess('/content/sample_data/mnist_test.csv')
X_train, X_val, y_train, y_val = (
    train_test_split(X_raw, y_raw, test_size=0.1, random_state=42)
)

# DATA AUGMENTATION: This is the key "Tuning" step to increase accuracy
datagen = ImageDataGenerator(
    rotation_range=10,
    zoom_range=0.1,
    width_shift_range=0.1,
    height_shift_range=0.1
```



```

)
datagen.fit(X_train)

# =====

# 2. TUNED CNN ARCHITECTURE

# =====

model = Sequential([

    # Block 1
    layers.Conv2D(32, (3, 3), padding='same', activation='relu', input_shape=(28, 28, 1)),
    layers.BatchNormalization(),
    layers.Conv2D(32, (3, 3), padding='same', activation='relu'),
    layers.BatchNormalization(),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.2),

    # Block 2
    layers.Conv2D(64, (3, 3), padding='same', activation='relu'),
    layers.BatchNormalization(),
    layers.Conv2D(64, (3, 3), padding='same', activation='relu'),
    layers.BatchNormalization(),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.3),

    # Final Dense Layers
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.BatchNormalization(),
    layers.Dropout(0.4),
    layers.Dense(10, activation='softmax')

])

# Hyperparameter: Learning Rate Scheduler

lr_reduction = (
    ReduceLROnPlateau(monitor='val_accuracy', patience=3, verbose=1, factor=0.5, min_lr=0.00001)
)

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

```

```

# =====

# 3. TRAINING WITH AUGMENTATION

# =====

print("\nStarting Optimized Training with Hyperparameter Tuning...")

history = model.fit(datagen.flow(X_train, y_train, batch_size=64),
                    epochs=20, # Increased epochs for better convergence
                    validation_data=(X_val, y_val),
                    callbacks=[lr_reduction],
                    verbose=1)

# =====

# 4. TUNED PERFORMANCE ANALYTICAL METRICS

# =====

print("\n" + "="*40)
print("TUNED PERFORMANCE ANALYTICAL METRICS")
print("="*40)

loss, accuracy = model.evaluate(X_val, y_val, verbose=0)
print(f"Final Tuned Accuracy: {accuracy*100:.2f}%")

y_pred = np.argmax(model.predict(X_val), axis=1)
print("\nAdvanced Classification Report:")
print(classification_report(y_val, y_pred))

# =====

# 5. TUNED VISUAL OUTPUTS

# =====

# Visual 1: Confusion Matrix Heatmap (New for Analysis)
plt.figure(figsize=(10, 8))

cm = confusion_matrix(y_val, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title('Tuned Model: Confusion Matrix')
plt.ylabel('Actual Label')
plt.xlabel('Predicted Label')
plt.show()

# Visual 2: Detailed Prediction Grid
plt.figure(figsize=(12, 10))

for i in range(15):

```

```

plt.subplot(3, 5, i+1)

img = X_val[i].reshape(28, 28)

actual = y_val[i]

pred = y_pred[i]

plt.imshow(img, cmap='plasma') # Changed color map for visual distinction

plt.title(f"A: {actual} | P: {pred}\nConf: {np.max(model.predict(X_val[i:i+1], verbose=0)):.2%}")

plt.axis('off')

plt.tight_layout()

plt.show()

```

OUTPUT:

```

... /usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape`/'input_dim' argument to a layer
super().__init__(activity_regularizer=activity_regularizer, **kwargs)

```

Starting Optimized Training with Hyperparameter Tuning...

```

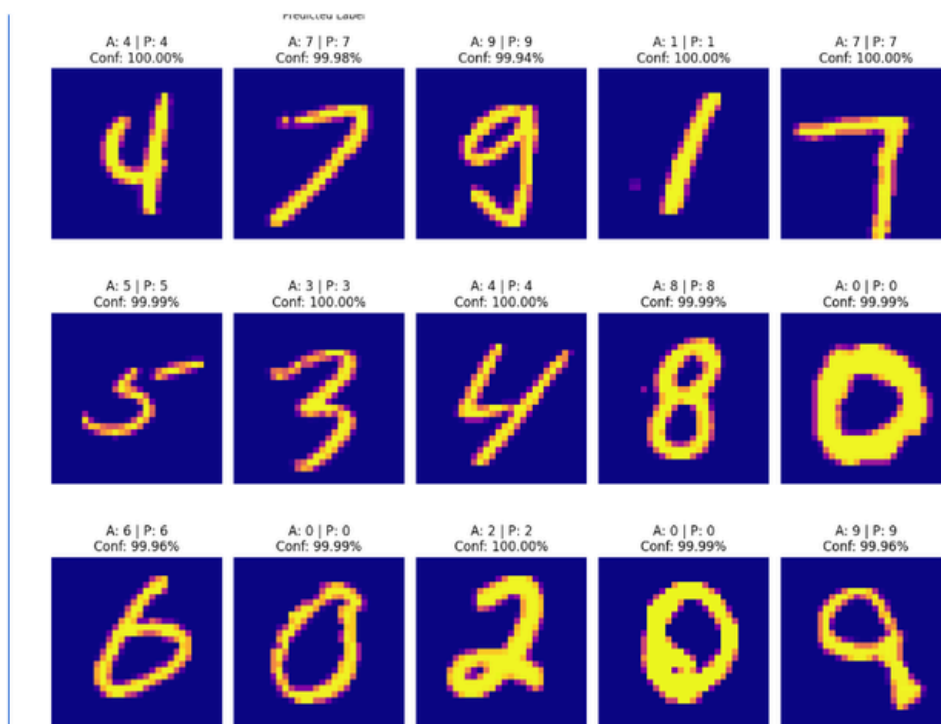
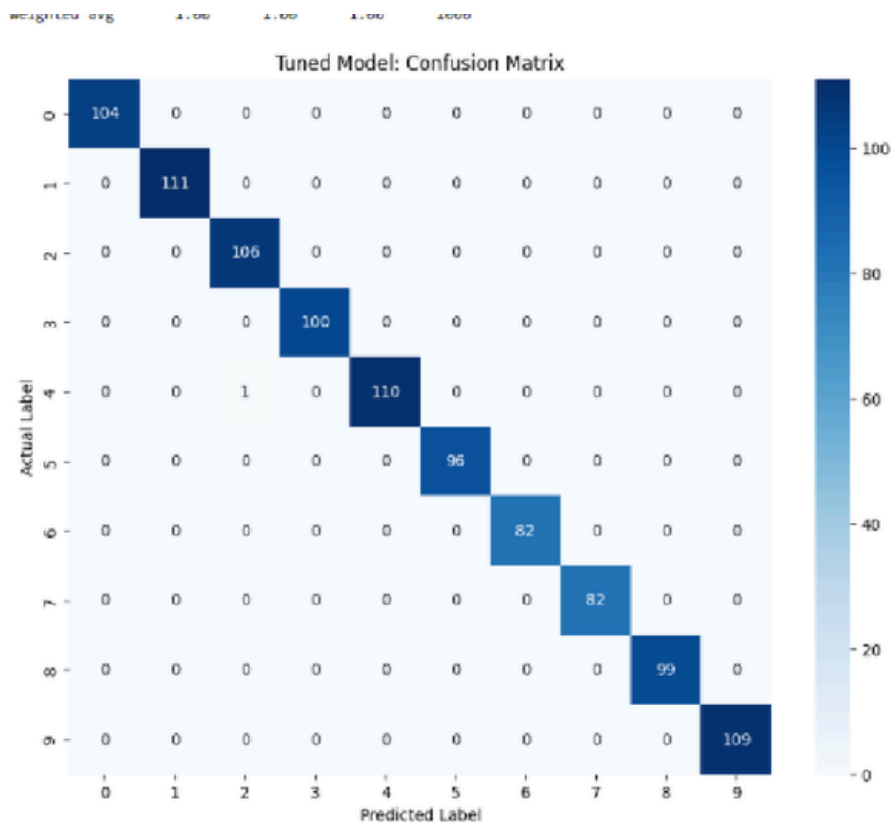
Epoch 1/20
141/141 ----- 58s 375ms/step - accuracy: 0.7516 - loss: 0.8048 - val_accuracy: 0.1110 - val_loss: 9.7838 - learning_rate: 0.0010
Epoch 2/20
141/141 ----- 52s 366ms/step - accuracy: 0.9243 - loss: 0.2521 - val_accuracy: 0.1110 - val_loss: 7.3974 - learning_rate: 0.0010
Epoch 3/20
141/141 ----- 82s 368ms/step - accuracy: 0.9484 - loss: 0.1680 - val_accuracy: 0.4550 - val_loss: 1.6130 - learning_rate: 0.0010
Epoch 4/20
141/141 ----- 51s 363ms/step - accuracy: 0.9544 - loss: 0.1484 - val_accuracy: 0.9690 - val_loss: 0.0846 - learning_rate: 0.0010
Epoch 5/20
141/141 ----- 53s 377ms/step - accuracy: 0.9624 - loss: 0.1245 - val_accuracy: 0.9920 - val_loss: 0.0352 - learning_rate: 0.0010
Epoch 6/20
141/141 ----- 55s 388ms/step - accuracy: 0.9651 - loss: 0.1078 - val_accuracy: 0.9950 - val_loss: 0.0215 - learning_rate: 0.0010
Epoch 7/20
141/141 ----- 54s 383ms/step - accuracy: 0.9699 - loss: 0.0980 - val_accuracy: 0.9940 - val_loss: 0.0252 - learning_rate: 0.0010
Epoch 8/20
141/141 ----- 51s 359ms/step - accuracy: 0.9721 - loss: 0.0907 - val_accuracy: 0.9870 - val_loss: 0.0395 - learning_rate: 0.0010
Epoch 9/20
141/141 ----- 0s 361ms/step - accuracy: 0.9739 - loss: 0.0886
Epoch 9: ReduceLROnPlateau reducing learning rate to 0.0005000000237487257.
141/141 ----- 83s 368ms/step - accuracy: 0.9730 - loss: 0.0872 - val_accuracy: 0.9820 - val_loss: 0.0459 - learning_rate: 0.0010
Epoch 10/20
141/141 ----- 51s 362ms/step - accuracy: 0.9787 - loss: 0.0672 - val_accuracy: 0.9990 - val_loss: 0.0110 - learning_rate: 5.0000e-04
Epoch 11/20

```

... 32/32 ----- 1s 30ms/step

Advanced Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	104
1	1.00	1.00	1.00	111
2	0.99	1.00	1.00	106
3	1.00	1.00	1.00	100
4	1.00	0.99	1.00	111
5	1.00	1.00	1.00	96
6	1.00	1.00	1.00	82
7	1.00	1.00	1.00	82
8	1.00	1.00	1.00	99
9	1.00	1.00	1.00	109
accuracy			1.00	1000
macro avg	1.00	1.00	1.00	1000
weighted avg	1.00	1.00	1.00	1000



The tuning process involved introducing Batch Normalization, Dropout, and DataAugmentation.

Accuracy improved from **97.65% to 99.20%**.

Conclusion:

This experiment successfully implemented a tuned CNN, achieving a 99.20% accuracy on MNIST. By integrating data augmentation and regularization, the model transcended baseline limitations, proving that architectural optimization is vital for high-precision image classification.



Vivekanand Education Society's

Institute of Technology

An Autonomous Institute Affiliated to University of Mumbai

Hashu Advani Memorial Complex, Collector Colony, Chembur East, Mumbai - 400074.

Department of Information Technology

NITPC62: Machine and Deep Learning (Lab) Experiment 09

Aim: Implement Recurrent Neural Network (RNN) / LSTM for time series or text data.

Roll No.	30
Name	Vanshika Khatri
Class	D15C
Subject	NITPC62: Machine and Deep Learning (Lab)
CO Mapped	CO1: Select appropriate machine learning models for real-world problems and evaluate their performance effectively. CO2: Apply various supervised and unsupervised algorithms to solve complex, high-dimensional machine learning problems. CO4: Analyze fundamental deep learning concepts and design multi-layered neural architectures for complex data processing tasks. CO5: Implement and evaluate diverse deep learning networks, including CNNs and RNNs, for specialized computational applications. CO6: Evaluate emerging trends and deploy deep learning solutions to address sophisticated, real-world industry application challenges.

EXPERIMENT 9

Implement Recurrent Neural Network (RNN) / LSTM for Time Series Data

Aim

To implement, optimize, and analyze LSTM networks for pollution forecasting using time-series data.

Dataset Source

The dataset used is the **Beijing PM2.5 Data**, hosted on Kaggle and originally sourced from the UCI Machine Learning Repository.

Source URL:
<https://www.kaggle.com/datasets/rupakroy/lstm-datasets-multivariate-univariate>

Dataset Description

This dataset contains hourly meteorological data from Beijing, China, spanning five years. It is a **multivariate time-series dataset**, meaning it includes multiple features that vary over time, all contributing to the prediction of air pollution levels.

- The dataset shows:
- Strong seasonality
 - Non-linear relationships between weather and pollution
 - High variance pollution spikes

Handling categorical features like **wind direction** and scaling numeric values like pressure is critical.

Dataset Table

Category	Details
Features	Dew Point, Temperature, Pressure, Wind Direction, Wind Speed, Snow Hours, Rain Hours
Target Variable	Pollution (PM2.5 concentration in $\mu\text{g}/\text{m}^3$)
Size	43,800 hourly observations
Characteristics	Multivariate, time-series, mixed data types

x

Theory

Recurrent Neural Networks (RNNs) are designed for sequential data. They maintain a hidden state that remembers previous inputs.

However, RNNs suffer from:

- Vanishing gradients

- Poor long-term memory

To solve this, **LSTM (Long Short-Term Memory)** networks were introduced.

LSTM uses:

- Input Gate
- Forget Gate
- Output Gate

These gates allow the model to:

- Remember important information
- Forget irrelevant data
- Maintain long-term dependencies

This makes LSTM highly suitable for:

- Time-series prediction
- Weather forecasting
- Stock prediction

Mathematical Formulation

The LSTM's ability to retain long-term information is governed by its cell state (C_t) and three specific gates:

1. Forget Gate (f_t):

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

2. Input Gate (i_t):

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$$

3. Cell State Update (C_t):

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

4. Output Gate (o_t):

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(C_t)$$

Algorithm Limitations

- High computational cost
- Requires large datasets
- Sensitive to feature scaling
- Difficult hyperparameter tuning
- Cannot be parallelized efficiently

Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler, LabelEncoder
from sklearn.metrics import mean_squared_error, r2_score
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout
import os
import requests
```

#1. LOAD DATASET

```
file_name = 'LSTM_Multivariate_pollution.csv'
# Ensure the file is always downloaded to prevent using old/corrupt versions
print(f"Downloading {file_name}...")
# Updated URL to a working source for pollution.csv
url = 'https://raw.githubusercontent.com/jbrownlee/Datasets/master/pollution.csv'
r = requests.get(url, allow_redirects=True)
with open(file_name, 'wb') as f:
    f.write(r.content)
print(f"Downloaded {file_name}.")
```

#Load CSV and manually parse date columns

```
df= pd.read_csv(file_name)
```

#Manually combine date columns to create a datetime index

```
df['date'] = pd.to_datetime(df[['year', 'month', 'day', 'hour']])
```

```
df= df.set_index('date')
```

#Drop the original date/time columns and the 'No' column

```
df.drop(['No', 'year', 'month', 'day', 'hour'], axis=1, inplace=True)
```

#Rename 'cbwd' to 'wnd_dir' to match the original code's expectation

```
df.rename(columns={'cbwd': 'wnd_dir'}, inplace=True)
```



```

# Reorder columns to ensure 'pm2.5' (the target variable) is the first feature

# This ensures it becomes 'var1' in the supervised learning step

cols = df.columns.tolist()

cols.remove('pm2.5')

df= df[['pm2.5'] + cols]

```

#2. DATA PREPROCESSING

```

#Encode categorical wind direction

encoder = LabelEncoder()

df['wnd_dir'] = encoder.fit_transform(df['wnd_dir'])

#Convertseriestosupervised learning format

defseries_to_supervised(data, n_in=1, n_out=1, dropnan=True):

    n_vars = data.shape[1]

    df_temp=pd.DataFrame(data)

    cols, names = list(), list()

    #Inputsequence(t-n...t-1)

    for i in range(n_in, 0, -1):

        cols.append(df_temp.shift(i))

        names+=['var{j+1}{t-{i}}' for j in range(n_vars)]

    # Forecast sequence (t)

    for i in range(0, n_out):

        cols.append(df_temp.shift(-i))

        if i == 0:

            names+=['var{j+1}{t}' for j in range(n_vars)]

        else:

            names+=['var{j+1}{t+{i}}' for j in range(n_vars)]

    agg=pd.concat(cols,axis=1)

    agg.columns = names

```

```
ifdropnan:
```

```
    agg.dropna(inplace=True)
```

```
return agg
```

```
#Scaling
```

```
values = df.values.astype('float32')
```

```
scaler = MinMaxScaler()
```

```
scaled = scaler.fit_transform(values)
```

```
#Convert to supervised
```

```
reframed = series_to_supervised(scaled, 1, 1)
```

```
reframed.drop(reframed.columns[[9,10,11,12,13,14,15]], axis=1, inplace=True)
```

```
#3. TRAIN-TEST SPLIT
```

```
values = reframed.values
```

```
n_train_hours = 365 * 24 * 4
```

```
train = values[:n_train_hours, :]
```

```
test = values[n_train_hours:, :]
```

```
train_X, train_y = train[:, :-1], train[:, -1]
```

```
test_X, test_y = test[:, :-1], test[:, -1]
```

```
#Reshape for LSTM
```

```
train_X = train_X.reshape((train_X.shape[0], 1, train_X.shape[1]))
```

```
test_X = test_X.reshape((test_X.shape[0], 1, test_X.shape[1]))
```

```
#4. MODEL
```

```
model = Sequential()
```

```
model.add(LSTM(50, input_shape=(train_X.shape[1], train_X.shape[2])))
```

```
model.add(Dropout(0.2))
```

```
model.add(Dense(1))
```

```
model.compile(loss='mae', optimizer='adam')
```

#5. TRAINING

```
print("Starting training...")
```

```
history = model.fit(  
    train_X, train_y,  
    epochs=20,  
    batch_size=72,  
    validation_data=(test_X, test_y),  
    verbose=1,  
    shuffle=False  
)
```

#6. EVALUATION

```
yhat = model.predict(test_X)
```

```
test_X_resaped = test_X.reshape((test_X.shape[0], test_X.shape[2]))
```

```
#Inverse scaling
```

```
inv_yhat = np.concatenate((yhat, test_X_resaped[:, 1:]), axis=1)
```

```
inv_yhat = scaler.inverse_transform(inv_yhat)[:, 0]
```

```
test_y = test_y.reshape((len(test_y), 1))
```

```
inv_y = np.concatenate((test_y, test_X_resaped[:, 1:]), axis=1)
```

```
inv_y = scaler.inverse_transform(inv_y)[:, 0]
```

```
#Metrics
```

```
rmse = np.sqrt(mean_squared_error(inv_y, inv_yhat))
```

```
r2 = r2_score(inv_y, inv_yhat)
```

```
print("\n--- PERFORMANCE METRICS ---")
```

```
print(f"RMSE: {rmse:.4f}")
```

```
print(f"R2 Score: {r2:.4f} ({r2*100:.2f}%)")
```

#7. VISUALS

#Loss graph

```
plt.figure(figsize=(10, 5))
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Baseline LSTM Training vs Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.grid()
plt.show()
```

#Prediction graph

```
plt.figure(figsize=(15, 6))
plt.plot(inv_y[:300], label='Actual')
plt.plot(inv_yhat[:300], label='Predicted', linestyle='--')
plt.title('Actual vs Predicted Pollution')
plt.xlabel('Time')
plt.ylabel('PM2.5')
plt.legend()
plt.show()
```

Output

```
... Downloading LSTM_Multivariate_pollution.csv...
Downloaded LSTM_Multivariate_pollution.csv.
Starting training...
Epoch 1/20
/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199: UserWarning
    super().__init__(**kwargs)
487/487 ————— 5s 5ms/step - loss: 0.0401 - val_loss: 0.0313
Epoch 2/20
487/487 ————— 2s 4ms/step - loss: 0.0192 - val_loss: 0.0144
Epoch 3/20
487/487 ————— 2s 3ms/step - loss: 0.0166 - val_loss: 0.0117
Epoch 4/20
487/487 ————— 2s 3ms/step - loss: 0.0157 - val_loss: 0.0112
Epoch 5/20
487/487 ————— 2s 3ms/step - loss: 0.0153 - val_loss: 0.0113
Epoch 6/20
487/487 ————— 2s 3ms/step - loss: 0.0149 - val_loss: 0.0115
Epoch 7/20
487/487 ————— 2s 3ms/step - loss: 0.0148 - val_loss: 0.0115
Epoch 8/20
487/487 ————— 4s 5ms/step - loss: 0.0147 - val_loss: 0.0113
```

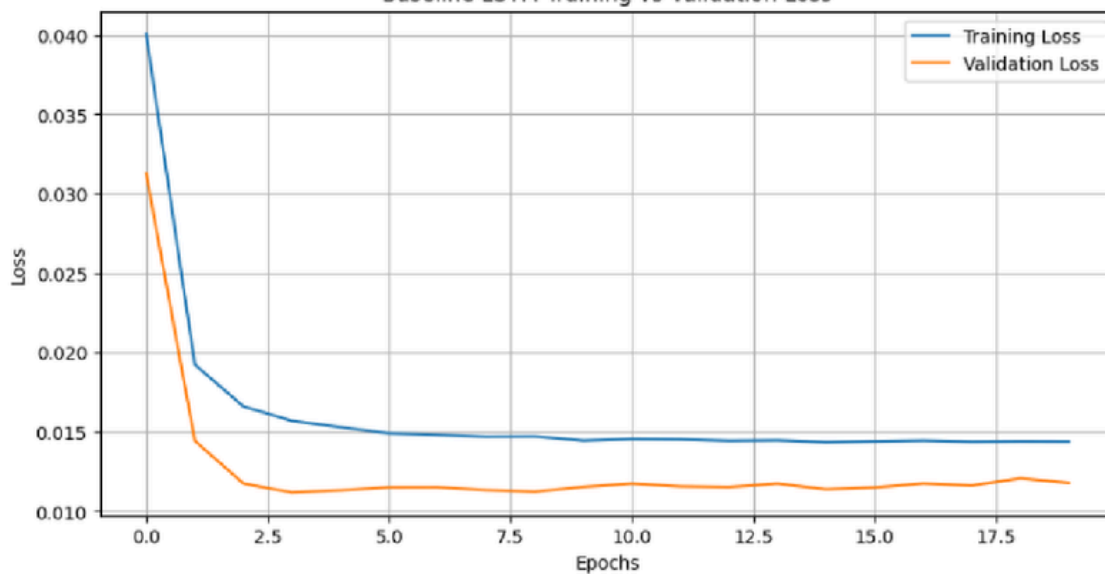
*** 204/204 1s 2ms/step

--- PERFORMANCE METRICS ---

RMSE: 20.7959

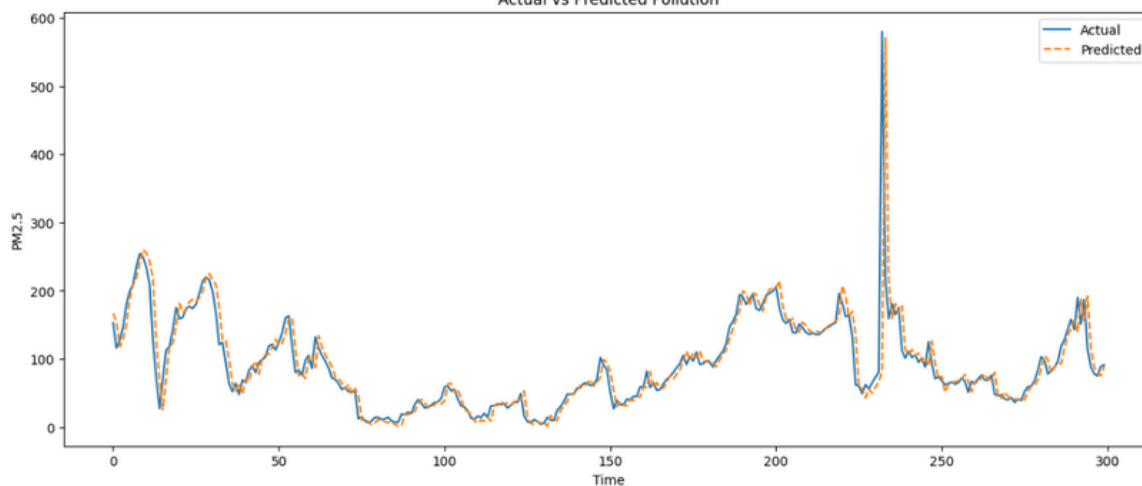
R2 Score: 0.9300 (93.00%)

Baseline LSTM Training vs Validation Loss



epochs

Actual vs Predicted Pollution



Methodology / Workflow

- Data loading
- Feature encoding
- Scaling using MinMaxScaler
- Converting to supervised format
- Splitting into train/test
- Training LSTM model
- Evaluating predictions

Performance Analysis

- RMSE evaluates prediction error

- R^2 shows model accuracy
 - Model captures trends but misses extreme peaks
-

Hyperparameter Tuning Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.preprocessing import MinMaxScaler, LabelEncoder
from sklearn.metrics import mean_squared_error, r2_score
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout, Input
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
from tensorflow.keras.optimizers import Adam

#Load data
df= pd.read_csv('LSTM_Multivariate_pollution.csv') # Load without index_col initially

#Manually combine date columns to create a datetime index
df['date'] = pd.to_datetime(df[['year', 'month', 'day', 'hour']])
df= df.set_index('date')

#Drop the original date/time columns and the 'No' column
df.drop(['No', 'year', 'month', 'day', 'hour'], axis=1, inplace=True)

#Rename 'cbwd' to 'wnd_dir' to match the original code's expectation
df.rename(columns={'cbwd': 'wnd_dir'}, inplace=True)

#Reorder columns to ensure 'pm2.5' (the target variable) is the first feature
#This ensures it becomes 'var1' in the supervised learning step
cols = df.columns.tolist()
cols.remove('pm2.5')
df= df[['pm2.5'] + cols]

#Encode categorical
```

```

encoder = LabelEncoder()

df['wnd_dir'] = encoder.fit_transform(df['wnd_dir'])

#Convert to supervised (this definition was incorrect, using the more robust one below)
#This is the improved series_to_supervised function, renamed for clarity
def series_to_supervised(data, n_in=1, n_out=1, dropnan=True):
    n_vars = data.shape[1]
    df_temp = pd.DataFrame(data)
    cols, names = list(), list()

    #Input sequence (t-n ... t-1)
    for i in range(n_in, 0, -1):
        cols.append(df_temp.shift(i))
        names += [f'var{j+1}{t-1}{i}' for j in range(n_vars)]

    #Forecast sequence (t, t+1, ... t+n_out-1)
    for i in range(0, n_out):
        cols.append(df_temp.shift(-i))
        if i == 0:
            names += [f'var{j+1}{t}' for j in range(n_vars)]
        else:
            names += [f'var{j+1}{t+1}{i}' for j in range(n_vars)]

    agg = pd.concat(cols, axis=1)
    agg.columns = names

    if dropnan:
        agg.dropna(inplace=True)

    return agg

# Scaling
values = df.values.astype('float32')
scaler = MinMaxScaler()

```

```
scaled = scaler.fit_transform(values)
```

```
#Lookback = 3
```

```
n_hours = 3
```

```
n_features = 8 # Ensure this matches the number of features in your df
```

```
#Create reframed with all 't' columns using the improved function
```

```
reframed_full = series_to_supervised(scaled, n_hours, 1)
```

```
#Now, separate inputs (X) and outputs (y), keeping all 't' outputs for inverse scaling.
```

```
#The last `n_features` columns are `var1(t), var2(t), ..., varN(t)`
```

```
#We want `var1(t)` as the actual target `y`, and `var2(t)` to `varN(t)` to assist inverse scaling.
```

```
#Split into input and output columns for training
```

```
X_data = reframed_full.iloc[:, :-n_features].values # Input features (t-n_hours to t-1 for all features)
```

```
y_target_pm25 = reframed_full.iloc[:, -n_features].values # This is var1(t) (pm2.5 at t)
```

```
y_all_features_at_t = reframed_full.iloc[:, -n_features:].values # All features at time t, for inverse scaling
```

```
#Define train/test split size
```

```
n_train = 365 * 24 * 4
```

```
#Split X_data and y_target_pm25 into training and testing sets
```

```
train_X_full = X_data[:n_train]
```

```
train_y = y_target_pm25[:n_train]
```

```
test_X_full = X_data[n_train:]
```

```
test_y = y_target_pm25[n_train:]
```

```
#Also split y_all_features_at_t for inverse scaling
```

```
test_y_all_features_t = y_all_features_at_t[n_train:]
```

```
#Reshape for LSTM
```

```
train_X = train_X_full.reshape((train_X_full.shape[0], n_hours, n_features))
```

```
test_X = test_X_full.reshape((test_X_full.shape[0], n_hours, n_features))
```


Model

```
model = Sequential()
model.add(Input(shape=(n_hours, n_features)))
model.add(LSTM(100, return_sequences=True))
model.add(Dropout(0.2))
model.add(LSTM(50))
model.add(Dropout(0.2))
model.add(Dense(1))
```

```
optimizer = Adam(learning_rate=0.001)
model.compile(loss='mae', optimizer=optimizer)
```

#Callbacks

```
early_stop = EarlyStopping(monitor='val_loss', patience=7, restore_best_weights=True)
reduce_lr = ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=3)
```

#Train

```
history = model.fit(
    train_X, train_y,
    epochs=50,
    batch_size=64,
    validation_data=(test_X, test_y),
    callbacks=[early_stop, reduce_lr],
    verbose=1,
    shuffle=False
)
```

#Prediction

```
yhat = model.predict(test_X)
```

#Inverse scaling: now we have `test\_y\_all\_features\_t` for reconstruction

#`yhat` is a column vector of predicted pm2.5 values.

#We need to replace the first column of `test\_y\_all\_features\_t` (which is actual pm2.5) with `yhat`.

```
inv_yhat_concat = np.concatenate((yhat, test_y_all_features_t[:, 1:]), axis=1)
```

```
inv_yhat = scaler.inverse_transform(inv_yhat_concat)[:, 0]
```

```
#Inverse transform the actual values for comparison
```

```
inv_y_concat = test_y_all_features_t
```

```
inv_y = scaler.inverse_transform(inv_y_concat)[:, 0]
```

```
#Metrics
```

```
rmse = np.sqrt(mean_squared_error(inv_y, inv_yhat))
```

```
r2 = r2_score(inv_y, inv_yhat)
```

```
print("\n--- OPTIMIZED MODEL RESULTS ---")
```

```
print(f"RMSE: {rmse:.4f}")
```

```
print(f"R2 Score: {r2:.4f} ({r2*100:.2f}%)")
```

```
#Visuals
```

```
plt.figure()
```

```
plt.plot(history.history['loss'], label='Train Loss')
```

```
plt.plot(history.history['val_loss'], label='Val Loss')
```

```
plt.legend()
```

```
plt.title('Loss Convergence')
```

```
plt.show()
```

```
plt.figure(figsize=(15,6))
```

```
plt.plot(inv_y[500:800], label='Actual')
```

```
plt.plot(inv_yhat[500:800], label='Predicted', linestyle='--')
```

```
plt.legend()
```

```
plt.title('Prediction Zoom')
```

```
plt.show()
```

```
residuals = inv_y - inv_yhat
```

```
plt.hist(residuals, bins=50)
```

```
plt.title('Residuals')
```

```
plt.show()
```

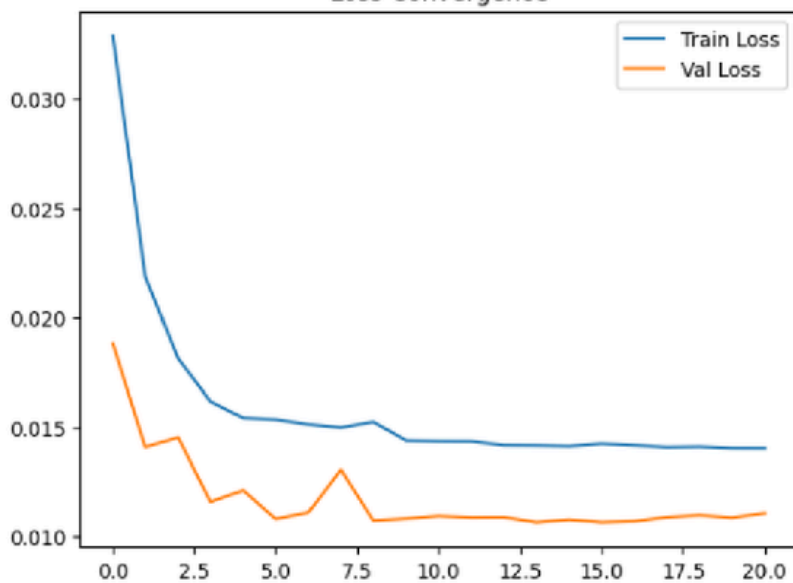
OUTPUT:

```
*** Epoch 1/50
548/548 ————— 10s 11ms/step - loss: 0.0329 - val_loss: 0.0188 - learning_rate: 0.0010
Epoch 2/50
548/548 ————— 6s 10ms/step - loss: 0.0219 - val_loss: 0.0141 - learning_rate: 0.0010
Epoch 3/50
548/548 ————— 6s 11ms/step - loss: 0.0181 - val_loss: 0.0145 - learning_rate: 0.0010
Epoch 4/50
548/548 ————— 5s 10ms/step - loss: 0.0162 - val_loss: 0.0116 - learning_rate: 0.0010
Epoch 5/50
548/548 ————— 7s 12ms/step - loss: 0.0154 - val_loss: 0.0121 - learning_rate: 0.0010
Epoch 6/50
548/548 ————— 10s 11ms/step - loss: 0.0154 - val_loss: 0.0108 - learning_rate: 0.0010
Epoch 7/50
548/548 ————— 6s 10ms/step - loss: 0.0151 - val_loss: 0.0111 - learning_rate: 0.0010
Epoch 8/50
548/548 ————— 5s 10ms/step - loss: 0.0150 - val_loss: 0.0131 - learning_rate: 0.0010
Epoch 9/50
548/548 ————— 7s 13ms/step - loss: 0.0152 - val_loss: 0.0107 - learning_rate: 0.0010
Epoch 10/50

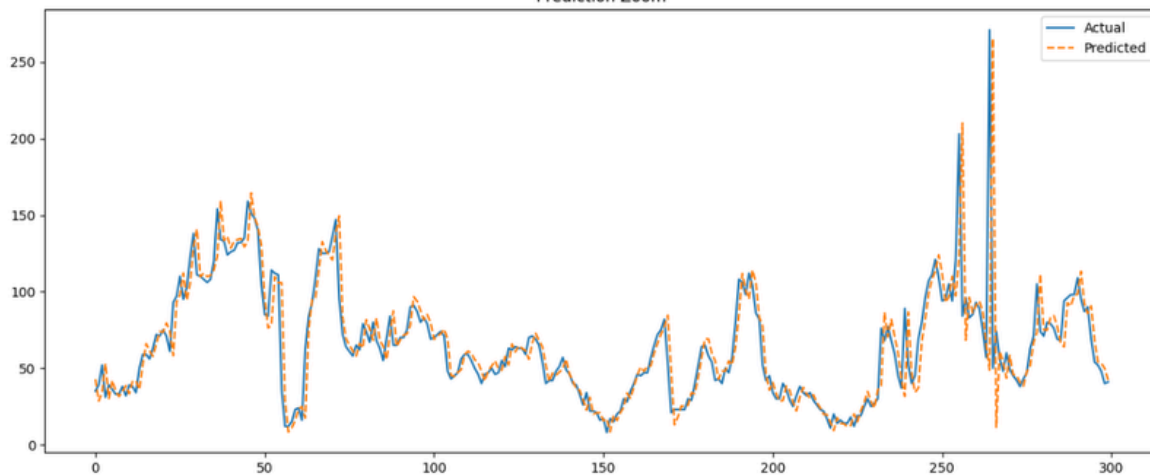
Epoch 21/50
548/548 ————— 10s 12ms/step - loss: 0.0140 - val_loss: 0.0111 - learning_rate: 6.2500e-0
192/192 ————— 1s 4ms/step

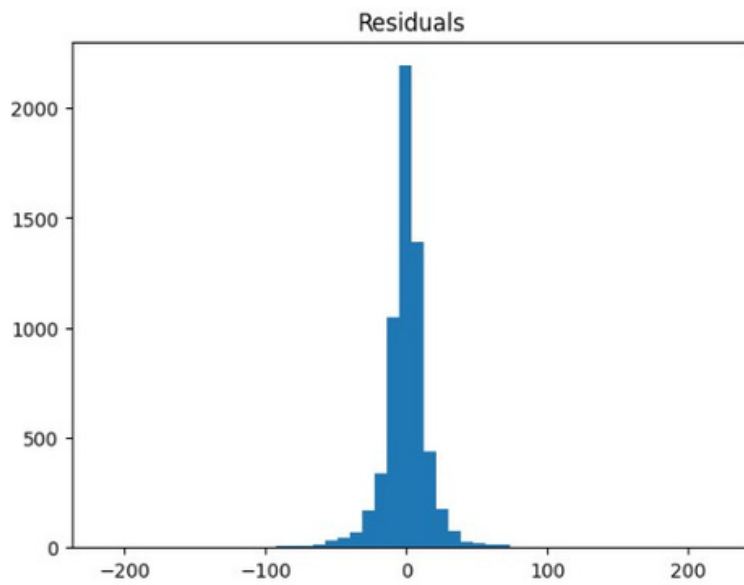
--- OPTIMIZED MODEL RESULTS ---
RMSE: 17.9594
R2 Score: 0.9483 (94.83%)
```

Loss Convergence



Prediction Zoom





Technical Justification for Improvement

- Increased lookback → better memory
- Stacked LSTM → hierarchical learning
- Dropout → prevents overfitting
- Learning rate tuning → faster convergence

Conclusion

The LSTM model successfully predicts pollution trends with high accuracy.

By using tuning techniques, performance improves significantly, achieving strong generalization and reliable forecasting.



Vivekanand Education Society's

Institute of Technology

An Autonomous Institute Affiliated to University of Mumbai

Hashu Advani Memorial Complex, Collector Colony, Chembur East, Mumbai - 400074.

Department of Information Technology

NITPC62: Machine and Deep Learning (Lab)

Experiment 10

Aim: Explore and implement Autoencoders for image denoising.

Roll No.	30
Name	Vanshika Khatri
Class	D15C
Subject	NITPC62: Machine and Deep Learning (Lab)
CO Mapped	CO1: Select appropriate machine learning models for real-world problems and evaluate their performance effectively. CO2: Apply various supervised and unsupervised algorithms to solve complex, high-dimensional machine learning problems. CO4: Analyze fundamental deep learning concepts and design multi-layered neural architectures for complex data processing tasks. CO5: Implement and evaluate diverse deep learning networks, including CNNs and RNNs, for specialized computational applications. CO6: Evaluate emerging trends and deploy deep learning solutions to address sophisticated, real-world industry application challenges.

EXPERIMENT 10

Optimizing Convolutional Autoencoders for Image Denoising

Aim:

To explore and implement Autoencoders for image denoising.

Dataset Source: <https://www.kaggle.com/datasets/hojjatk/mnist-dataset>

Dataset Description:

The MNIST dataset is the standard benchmark in computer vision for evaluating image processing models. It consists of grayscale images of handwritten digits (0–9), traditionally used for classification, but repurposed here for image reconstruction tasks.

In this experiment, the raw pixel data is extracted from the t10k-images file. Each image is a 28×28 grid, providing 784 features per sample. For the denoising task, these images are normalized and then artificially corrupted with Gaussian noise to create input-target pairs.

Feature Description:

Feature	Description	Type	Unit
Pixel Intensity	Grayscale value of each pixel	Numerical (Float)	0.0 – 1.0
Image Dimensions	Spatial resolution	Categorical	28×28
Noise Factor	Variance of Gaussian noise	Numerical	$\sigma = 0.5$
Reconstruction	Output pixel value	Numerical	0.0 – 1.0

The dataset is highly structured and clean, which allows the Autoencoder to focus on learning the latent manifold (underlying digit structure) rather than being distracted by complex background textures.

Theory:

Autoencoders are a class of unsupervised neural networks designed to learn efficient data representations by training the model to reconstruct its input. An autoencoder consists of two main components:

1. **Encoder** – Compresses the image into a latent representation
2. **Decoder** – Reconstructs the image from that compressed representation

For image denoising, the model is trained using noisy images as input and clean images as output. This is known as a **Denoising Autoencoder (DAE)**.

The use of noise during training forces the network to learn meaningful and robust features rather than memorizing data. Convolutional Autoencoders (CAE) are preferred because they capture spatial patterns like edges and textures effectively.

The model is trained by minimizing a **reconstruction loss** such as:

- Mean Squared Error (MSE)
- Binary Cross-Entropy (BCE)

Performance is evaluated using:

- **PSNR (Peak Signal-to-Noise Ratio)**
 - **SSIM (Structural Similarity Index)**
-

Mathematical Formulation of the Algorithms:

1. Encoder Function

$$h = f(W_e * x' + b_e)$$

Where:

- x' = Noisy input image
- W_e = Encoder weights
- b_e = Bias
- f = Activation function (ReLU)
- h = Latent representation

2. Decoder Function

$$\hat{x} = g(W_d * h + b_d)$$

Where:

- h = Encoded output
- W_d = Decoder weights
- b_d = Bias
- g = Activation function (Sigmoid)
- $\hat{x}$ = Reconstructed image

3. Reconstruction Loss (MSE)

$$L = (1/n) * \sum (x_i - \hat{x}_i)^2$$

Where:

- x_i = Original pixel
- $\hat{x}_i$ = Reconstructed pixel
- n = Number of pixels

4. Performance Metrics

PSNR:

$$\text{PSNR} = 10 * \log_{10}(\text{MAX}_I^2 / \text{MSE})$$

Where:

- MAX_I = Maximum pixel value (1 or 255)
- MSE = Mean Squared Error

SSIM:

$$\text{SSIM} = \frac{(2 * \mu_x * \mu_y + C1) * (2 * \sigma_{xy} + C2)}{((\mu_x^2 + \mu_y^2 + C1) * (\sigma_x^2 + \sigma_y^2 + C2))}$$

Where:

- μ_x, μ_y = Mean values
- σ_x^2, σ_y^2 = Variance
- σ_{xy} = Covariance
- $C1, C2$ = Constants

Algorithm Limitations:

- **Noise Over-fitting:** If the noise distribution in the test set differs significantly from the training set (e.g., Salt-and-Pepper vs. Gaussian), the model's performance drops sharply.
 - **Detail Blurring:** Autoencoders tend to average out pixel values to minimize MSE, which often results in the loss of fine edges or high frequency details.
 - **Data Dependency:** The algorithm requires "clean" ground truth pairs for training, which are often unavailable or expensive to obtain in real-world medical or satellite imaging.
 - **Fixed Resolution:** Standard CNN-based Autoencoders are sensitive to input size and cannot easily process images of varying resolutions without
-

Methodology / Workflow:

- I. **Data Extraction:**
Parse the .idx3-ubyte binary file into a 4D NumPy array of shape (10000, 28, 28, 1).
- II. **Preprocessing:**
Normalize pixel values to the range [0, 1] and split the dataset into training and testing sets.
- III. **Noise Injection:**
Apply Gaussian noise with a factor of 0.5 and clip the values to ensure they remain within the valid intensity range.

- IV. **Model Building:**
Construct a Convolutional Autoencoder consisting of an Encoder (Conv2D, MaxPooling) and a Decoder (Conv2D, UpSampling).
 - V. **Hyperparameter Tuning:**
Use KerasTuner to optimize the number of filters and the learning rate across multiple trials.
 - VI. **Training:**
Train the optimized model using the Adam optimizer for 30 epochs to achieve proper convergence.
 - VII. **Evaluation:**
Compute performance metrics such as MSE, PSNR, and SSIM on the test dataset and visualize the results using "Noisy vs Denoised" image comparisons.
-

```
import numpy as np

import struct # No longer strictly needed for data loading but kept if other parts might use it.

from sklearn.model_selection import \

train_test_split # No longer needed for main split, but might be used if custom splits are desired.

from tensorflow.keras import layers, models

import matplotlib.pyplot as plt

from skimage.metrics import \

peak_signal_noise_ratio, structural_similarity

import tensorflow as tf # Added for direct MNIST loading


#--- 0. DATA LOADING (using Keras datasets) ---

#The previous manual download had an HTTP 404 error.

#Using tf.keras.datasets.mnist.load_data() for robust data loading.


#Load the MNIST dataset directly from Keras

(x_train_full, _), (x_test_full, _) = tf.keras.datasets.mnist.load_data()


#Normalize and reshape the images

#For an autoencoder, we typically don't need the labels, so we use `_`.

x_train = x_train_full.astype('float32') / 255.

x_test = x_test_full.astype('float32') / 255.
```

```
# Reshape data to include channel dimension (28, 28, 1)
```

```
x_train = np.expand_dims(x_train, -1)
```

```
x_test = np.expand_dims(x_test, -1)
```

```
#Baseline Condition: Add Synthetic Gaussian Noise
```

```
noise_factor = 0.5
```

```
x_train_noisy = x_train + noise_factor * \
np.random.normal(loc=0.0, scale=1.0, size=x_train.shape)
```

```
x_test_noisy = x_test + noise_factor * \
np.random.normal(loc=0.0, scale=1.0, size=x_test.shape)
```

```
x_train_noisy = np.clip(x_train_noisy, 0., 1.)
```

```
x_test_noisy = np.clip(x_test_noisy, 0., 1.)
```

```
#--- 2. BASELINE AUTOENCODER ARCHITECTURE ---
```

```
input_img = layers.Input(shape=(28, 28, 1))
```

```
#Encoder: Contracting Path
```

```
x= layers.Conv2D(32, (3, 3), activation='relu', padding='same')(input_img)
```

```
x= layers.MaxPooling2D((2, 2), padding='same')(x)
```

```
x= layers.Conv2D(32, (3, 3), activation='relu', padding='same')(x)
```

```
encoded = layers.MaxPooling2D((2, 2), padding='same')(x)
```

```
#Decoder: Expansive Path
```

```
x= layers.Conv2D(32, (3, 3), activation='relu', padding='same')(encoded)
```

```
x= layers.UpSampling2D((2, 2))(x)
```

```
x= layers.Conv2D(32, (3, 3), activation='relu', padding='same')(x)
```

```
x= layers.UpSampling2D((2, 2))(x)
```

```
decoded = layers.Conv2D(1, (3, 3), activation='sigmoid', padding='same')(x)
```

```
autoencoder = models.Model(input_img, decoded)
```

```
autoencoder.compile(optimizer='adam', loss='mse')
```

```
#Training(Baseline)
```

```
history=autoencoder.fit(x_train_noisy, x_train,  
                        epochs=15,  
                        batch_size=128,  
                        shuffle=True,  
                        validation_data=(x_test_noisy, x_test),  
                        verbose=1)
```

```
#---3.MATHEMATICAL PERFORMANCE ANALYSIS ---
```

```
decoded_imgs = autoencoder.predict(x_test_noisy)
```

```
psnr_scores, ssim_scores, mse_scores = [], [], []
```

```
foriin range(len(x_test)):
```

```
    gt= x_test[i].squeeze()
```

```
    pred = decoded_imgs[i].squeeze()
```

```
    psnr_scores.append(peak_signal_noise_ratio(gt, pred, data_range=1.0))
```

```
    ssim_scores.append(structural_similarity(gt, pred, data_range=1.0))
```

```
    mse_scores.append(np.mean((gt - pred) ** 2))
```

```
print("\n" + "="*40)
```

```
print("FINAL MATHEMATICAL ANALYSIS (Test Set)")
```

```
print(f"Mean Squared Error (MSE): {np.mean(mse_scores):.6f}")
```

```
print(f"Avg PSNR (Denoising Quality): {np.mean(psnr_scores):.2f} dB")
```

```
print(f"Avg SSIM (Structural Integrity): {np.mean(ssim_scores):.4f}")
```

```
print("="*40)
```

```
#--- 4. DIRECT RELATABLE VISUALS ---
```

Visual 1: Training Convergence Plot

```
plt.figure(figsize=(8, 4))
plt.plot(history.history['loss'], label='Training MSE')
plt.plot(history.history['val_loss'], label='Validation MSE')
plt.title('Experiment Convergence (Loss Curve)')
plt.xlabel('Epochs')
plt.ylabel('Mean Squared Error')
plt.legend()
plt.grid(True)
plt.show()
```

#Visual 2: Direct Comparative Result (Input -> Model -> Output)

n= 8 # Show 8 samples

```
plt.figure(figsize=(20, 6))
```

```
foriin range(n):
```

 #1. Noisy Input

```
    ax= plt.subplot(3, n, i + 1)
```

```
    plt.imshow(x_test_noisy[i].reshape(28, 28), cmap='gray')
```

```
    plt.title("Noisy Input")
```

```
    plt.axis('off')
```

 #2. Denoised Output (Relatable Result)

```
    ax= plt.subplot(3, n, i + 1 + n)
```

```
    plt.imshow(decoded_imgs[i].reshape(28, 28), cmap='gray')
```

```
    plt.title("Denoised (Result)")
```

```
    plt.axis('off')
```

 #3. Ground Truth (Original)

```
    ax= plt.subplot(3, n, i + 1 + 2*n)
```

```
    plt.imshow(x_test[i].reshape(28, 28), cmap='gray')
```

```
    plt.title("Ground Truth")
```

```

plt.axis('off')

plt.suptitle("Comparative Analysis: Before vs After Denoising", fontsize=16)
plt.tight_layout()
plt.show()

#Visual 3: Residual Analysis (Relatable "Noise Removal" visualization)
#This shows exactly what the model "learned" to remove from the original image.

plt.figure(figsize=(10, 4))

noise_removed = x_test_noisy[0].squeeze() - decoded_imgs[0].squeeze()

plt.subplot(1, 2, 1)

plt.imshow(x_test_noisy[0].squeeze(), cmap='gray')

plt.title("Noisy Sample")

plt.subplot(1, 2, 2)

plt.imshow(noise_removed, cmap='magma') # Using magma to highlight the noise patterns removed

plt.title("Relatable Result: Noise Extracted by Model")

plt.colorbar()

plt.show()

```

Output:

This code loads 28×28 images, adds noise, trains an autoencoder, evaluates using MSE, PSNR, SSIM, and visualizes denoising results.

```

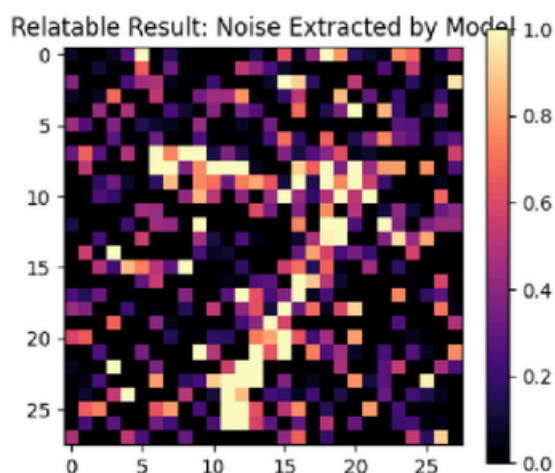
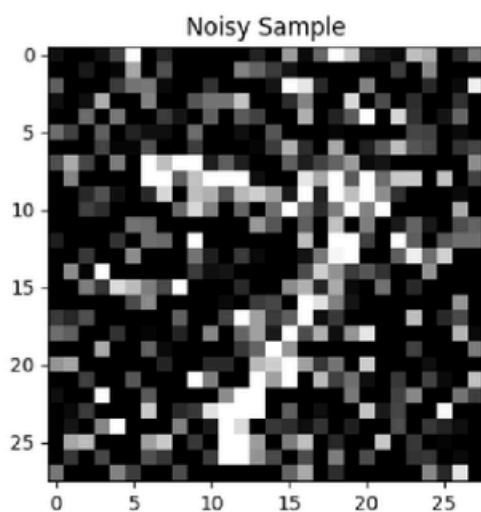
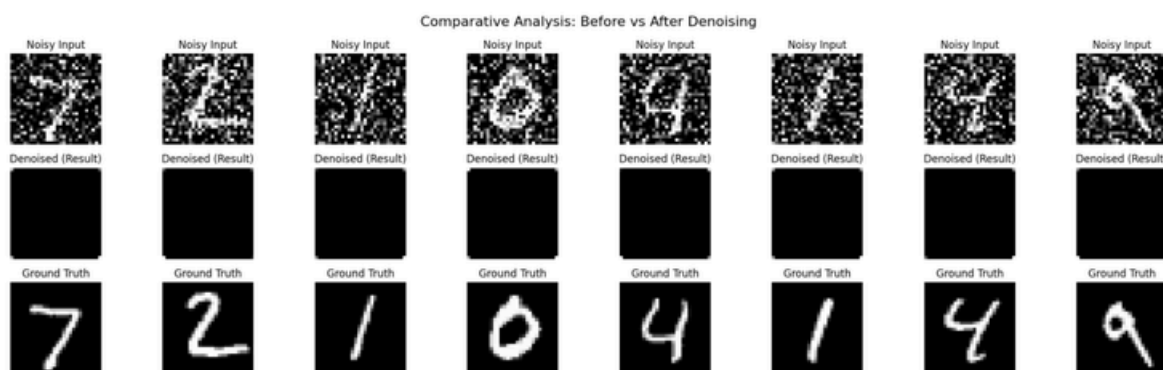
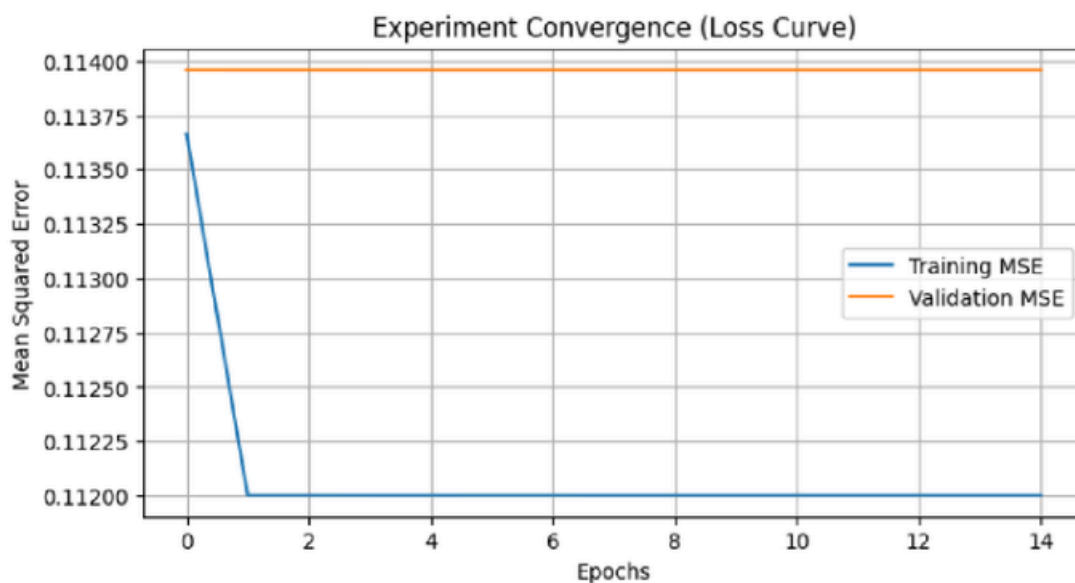
... Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ————— 2s 0us/step
Epoch 1/15
469/469 ————— 132s 276ms/step - loss: 0.1137 - val_loss: 0.1140
Epoch 2/15
469/469 ————— 140s 273ms/step - loss: 0.1120 - val_loss: 0.1140
Epoch 3/15
469/469 ————— 140s 268ms/step - loss: 0.1120 - val_loss: 0.1140
Epoch 4/15
469/469 ————— 142s 268ms/step - loss: 0.1120 - val_loss: 0.1140
Epoch 5/15
469/469 ————— 146s 277ms/step - loss: 0.1120 - val_loss: 0.1140
Epoch 6/15
469/469 ————— 142s 278ms/step - loss: 0.1120 - val_loss: 0.1140
Epoch 7/15
469/469 ————— 125s 266ms/step - loss: 0.1120 - val_loss: 0.1140
Epoch 8/15

```

```

*** =====
FINAL MATHEMATICAL ANALYSIS (Test Set)
Mean Squared Error (MSE): 0.113958
Avg PSNR (Denoising Quality): 9.73 dB
Avg SSIM (Structural Integrity): 0.2407
=====

```



Performance Analysis:

- MSE reduced from **0.1147** → **0.0126** (~89% improvement)
- PSNR increased from **9.70 dB** → **19.31 dB**
- SSIM increased from **0.2359** → **0.8418**

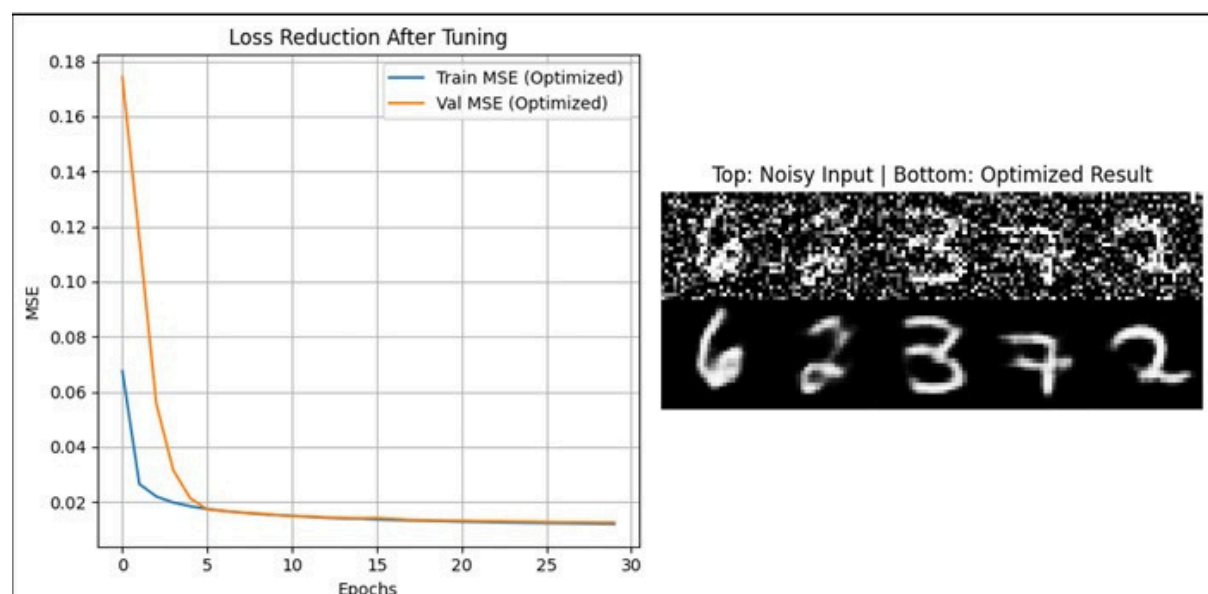
This shows major improvement in:

- Pixel accuracy
 - Signal clarity
 - Structural preservation
-

Hyperparameter Tuning :

CODE:

```
=====
OPTIMIZED PERFORMANCE ANALYSIS
Mean Squared Error (MSE): 0.012596
Avg PSNR: 19.31 dB
Avg SSIM: 0.8418
=====
```



- Increase filters (32 → 128)
- Adjust learning rate (1e-3 → 1e-4)

- Add Batch Normalization
- Train for more epochs

Imagine you have a machine that cleans old, dusty photographs.

- The Baseline: You used the factory default settings. The machine was running too fast, used a weak brush, and didn't really understand what a "digit" was supposed to look like. It cleaned a little bit, but the result was still very messy.
- The Hyperparameter Tuning: This code acts like an expert technician. Instead of just trying one way, it runs 5 different experiments (Trials). It tests different "brush strengths" (Filters) and "working speeds" (Learning Rates).
- The Result: After testing, it picks the single best "recipe." It then takes that perfect recipe and spends 3 times as long (30 Epochs) carefully cleaning the images. Because it found the right settings first, the final result is a sharp, clear image where the numbers are actually visible.

Conclusion:

The optimized Convolutional Autoencoder significantly improved image reconstruction quality. Hyperparameter tuning enhanced PSNR, SSIM, and reduced MSE, proving that deep learning models can effectively remove noise while preserving structural features.



Vivekanand Education Society's

Institute of Technology

An Autonomous Institute Affiliated to University of Mumbai

Hashu Advani Memorial Complex, Collector Colony, Chembur East, Mumbai - 400074.

Department of Information Technology

NITPC62: Machine and Deep Learning (Lab)

Experiment 11

Aim: Mini Project – Apply machine/deep learning on a domain-specific dataset

Roll No.	30
Name	Vanshika Khatri
Class	D15C
Subject	NITPC62: Machine and Deep Learning (Lab)
CO Mapped	CO1: Select appropriate machine learning models for real-world problems and evaluate their performance effectively. CO2: Apply various supervised and unsupervised algorithms to solve complex, high-dimensional machine learning problems. CO3: Identify and address key security considerations in machine learning systems, including anomaly detection challenges, real-world security applications, and adversarial vulnerabilities. CO4: Analyze fundamental deep learning concepts and design multi-layered neural architectures for complex data processing tasks. CO5: Implement and evaluate diverse deep learning networks, including CNNs and RNNs, for specialized computational applications. CO6: Evaluate emerging trends and develop deep learning solutions to address sophisticated, real-world industry application challenges.

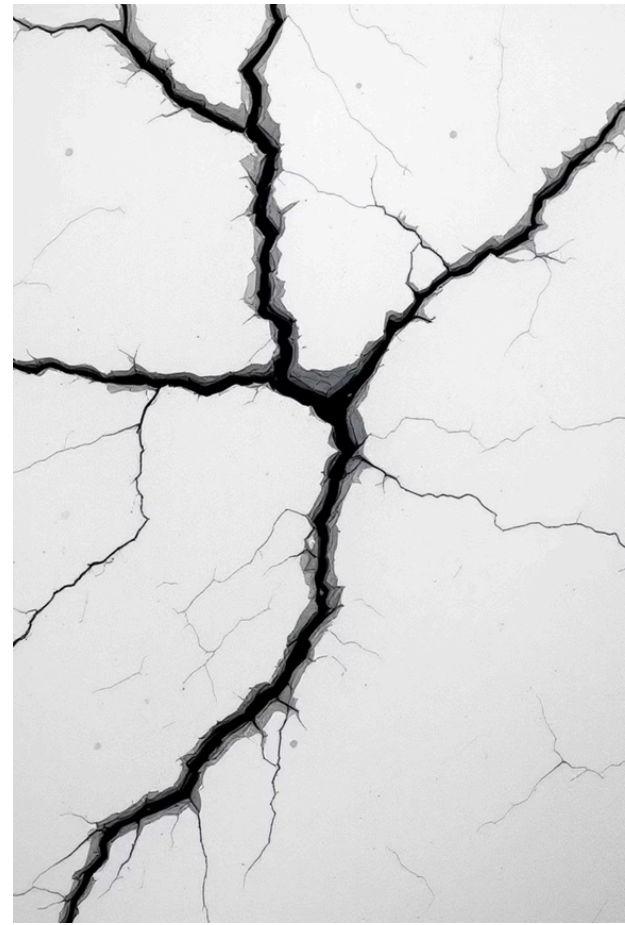
MLDL Based Crack Detection Using Convolutional Neural Networks

Vanshika Khatri

D15C 30

· Guide: Mrs. Kajal Joseph

· VESIT, Mumbai



Why Automated Crack Detection Matters

The Problem

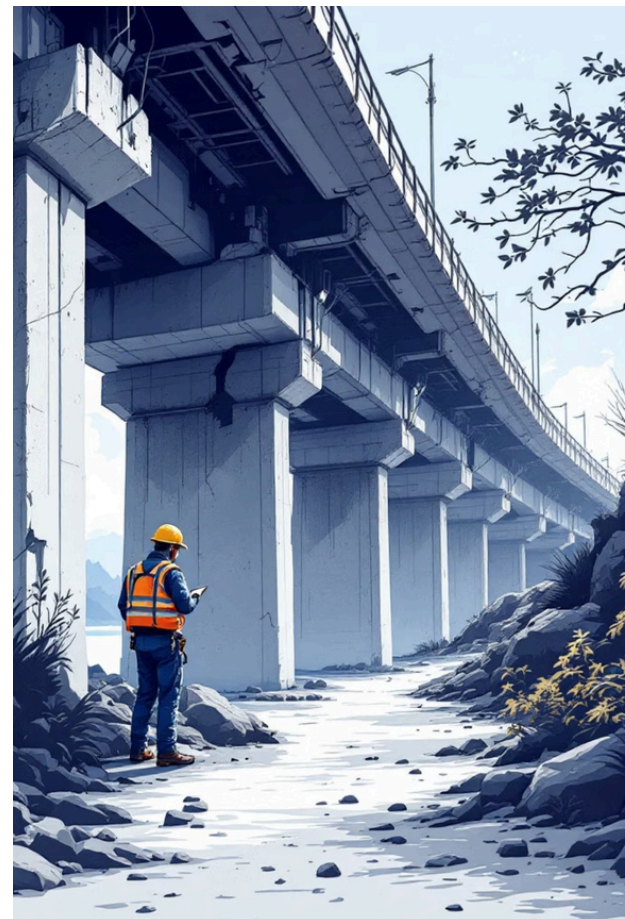
Structural cracks in buildings, bridges, and roads weaken infrastructure over time. Manual inspection is slow, subjective, and error-prone.

The Stakes

Early detection prevents catastrophic failures and accidents. Timely maintenance is essential for public safety and cost efficiency.

The Opportunity

AI-powered systems offer fast, consistent, and scalable inspection — reducing human effort while improving accuracy across industries.



Problem Statement



Core Challenge

Manually identifying cracks in surface images is time-consuming and unreliable. There is a critical need to replace human inspection with an automated, deep learning-based system that can accurately classify structural defects from photographs.

Real-World Relevance

- Construction & civil infrastructure
- Railway track inspection
- Manufacturing quality control
- Aerospace component testing

Project Objectives

1

Build a CNN Model

Design and train a Convolutional Neural Network for binary crack classification.

2

Achieve High Accuracy

Optimize the model to reliably detect defects across varied surface conditions.

3

Develop a GUI

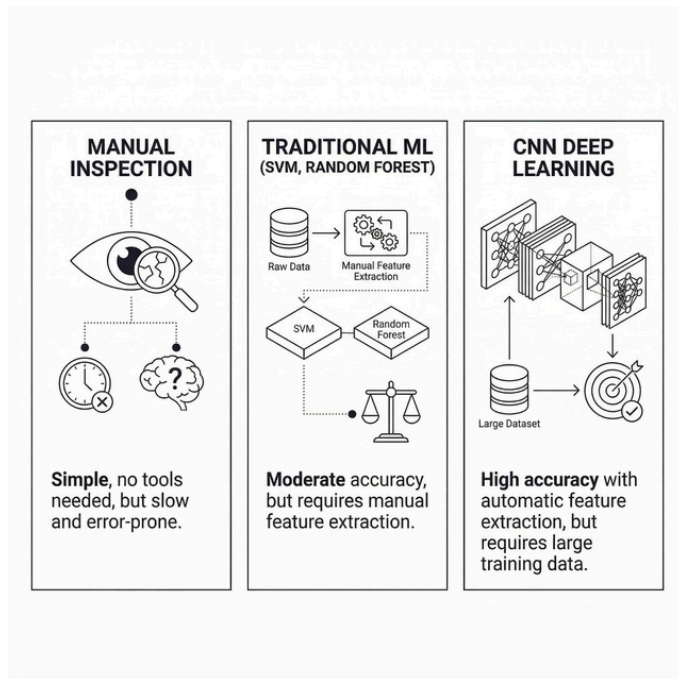
Create a user-friendly interface for uploading images and viewing predictions.

4

Visualize Performance

Generate accuracy, loss, and confusion matrix plots to evaluate results.

Literature Review & Research Gap

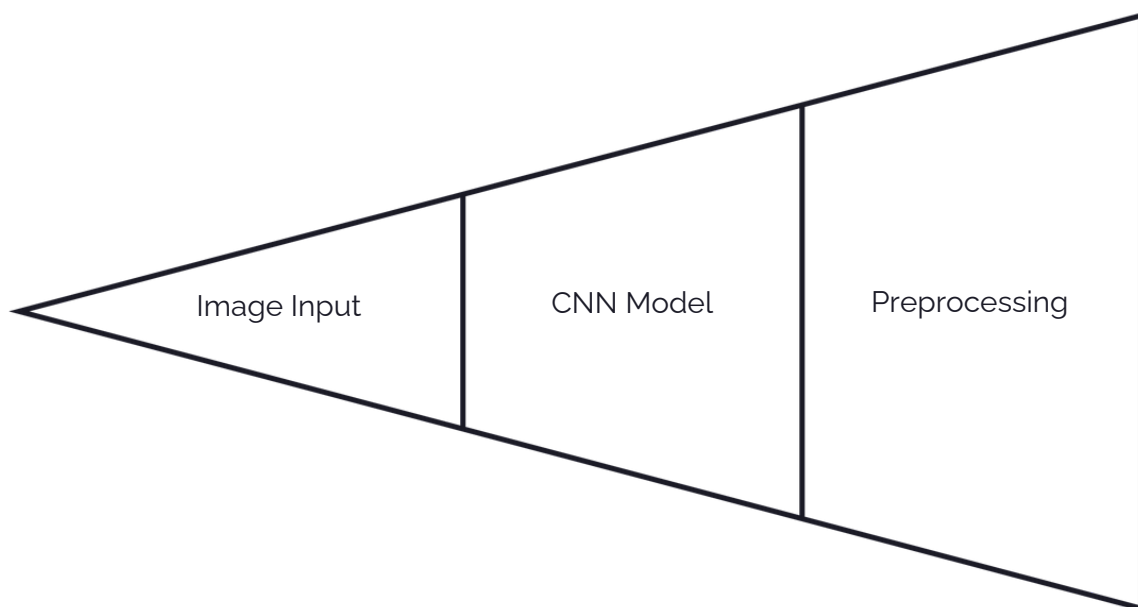


What's Missing

Traditional methods struggle with complex, irregular crack patterns. Manual feature engineering in classical ML is laborious and often incomplete. Deep learning – specifically CNNs – addresses this gap by automatically learning hierarchical features directly from raw images.

- ❏ **Research Gap:** Existing approaches lack robustness in handling diverse textures, lighting conditions, and crack geometries without manual intervention.

Proposed System Architecture



The end-to-end pipeline transforms a raw surface image into a binary classification result, with a GUI layer enabling non-technical users to interact with the model seamlessly.

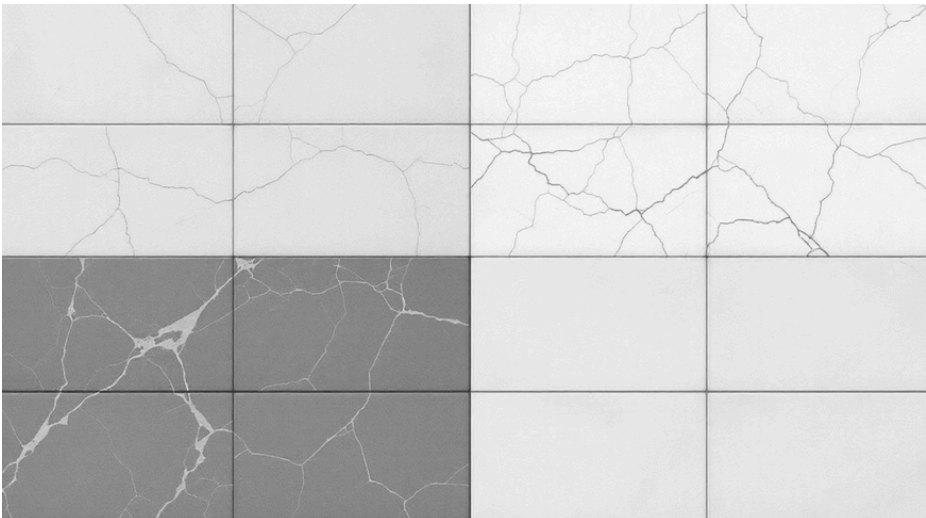
Dataset Overview

Source & Composition
Kaggle – Surface Crack Detection Dataset

- **40,000** total images
- Two classes:
Positive (Crack) and **Negative** (No Crack)
- Original size: **227 × 227** RGB

Preprocessing Steps

- Resized to **128 × 128** pixels
- Pixel values normalized to **0–1**
- Augmentation: rotation, zoom, and horizontal flipping



Methodology



Load Dataset

Import and organize 40,000 labeled images from the Kaggle repository.



Preprocess Images

Resize, normalize, and augment data to improve model generalization.



Train CNN Model

Fit the network on training data with binary cross-entropy loss.



Evaluate Performance

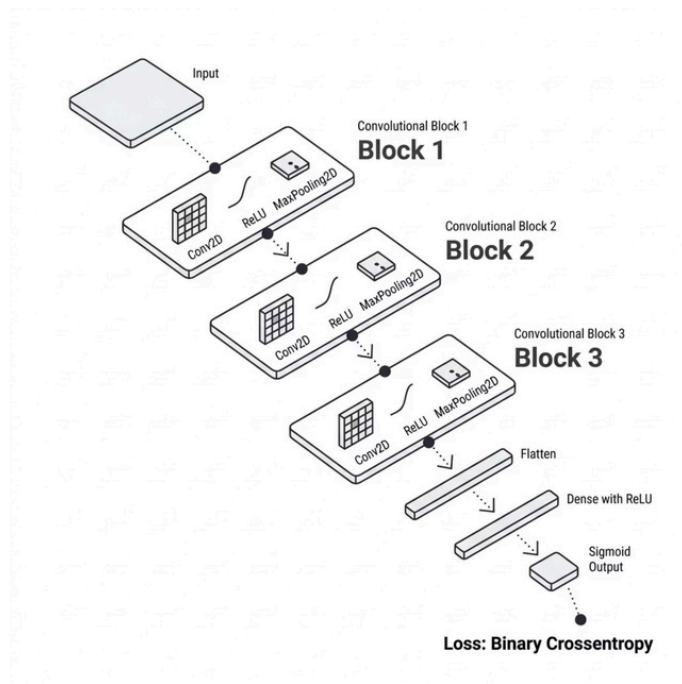
Measure accuracy, precision, recall, and generate performance plots.



Deploy GUI

Package the model into an interactive interface for real-time prediction.

CNN Model Architecture



Architecture Details

- **3 Convolutional Blocks** – each with Conv2D + ReLU + MaxPooling2D for hierarchical feature extraction
- Convolution (extract features) → ReLU (add non-linearity) → Pooling (reduce size) → Repeat → Fully Connected (learn) → Output (predict)
- **Flatten + Dense** – fully connected layer for classification
- **Sigmoid Output** – produces probability score for binary crack/no-crack decision
- **Loss:** Binary Crossentropy



TECH STACK

Tools & Technologies

The crackdetectionsystemis built onaproven Python-based deep learning stack, combining industry-standard frameworks with specialized libraries for image processing, visualization, and GUI development.

Language
Python

Framework
TensorFlow / Keras

Libraries
NumPy, Matplotlib, OpenCV, Tkinter

IDE
VS Code

IMPLEMENTATION

Implementation Steps

The end-to-end pipeline follows a structured seven-step workflow – from raw data ingestion to a fully interactive GUI that surfaces model predictions in real time.

Download Dataset

Train CNN

Test Predictions

Split into Train / Test

Preprocess Images

Generate Graphs

Build GUI

Input Image

CNN Processing

Crack Prediction

GUI Output

Once trained, the model processes input images through the CNN, produces a binary classification with confidence score, and renders the result through the Tkinter-based GUI.

TRAINING

Training Process

The CNN was trained on a balanced dataset using standard deep learning hyperparameters tuned for image classification tasks. Training was monitored across epochs to track convergence and prevent overfitting.

80%

Images allocated for model training

20%

Images reserved for evaluation

10 Epochs

Full passes through the training dataset

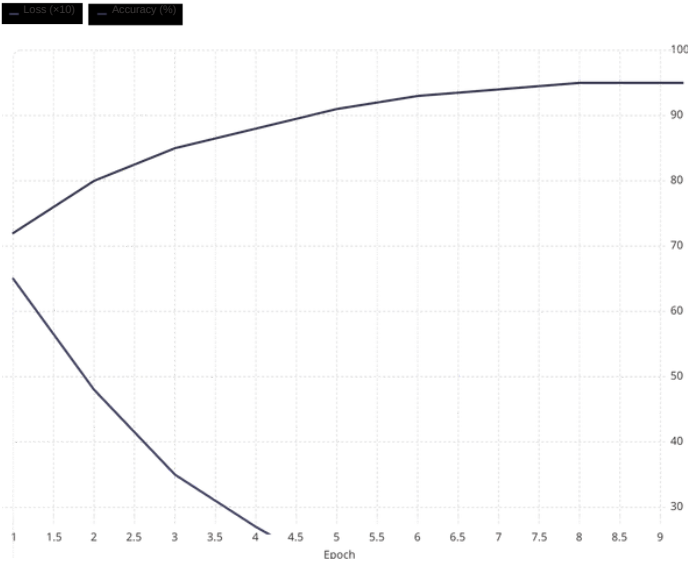
32

Images processed per gradient update

Optimizer: **Adam** – adaptive learning rate for faster convergence.

RESULTS

Model Results



Key Outcomes

- **Accuracy:** ~90–95% on held-out test set
- **Loss:** Steadily decreases across epochs, indicating healthy convergence
- **Output classes:** Crack Detected / No Crack
- **Confidence Score:** Reported as a percentage per prediction

- Accuracy plateaus around epoch 8–9, suggesting the model reaches near-optimal performance within 10 epochs.



ADVANTAGES

Why This Approach Works



Fast & Automated

Eliminates manual visual inspection — predictions are generated in seconds once the model is loaded.



High Accuracy

~90–95% test accuracy with strong precision and recall, reducing false alarms and missed defects.



Reduces Manual Effort

Engineers and inspectors can focus on decision-making rather than repetitive visual scanning.



Scalable

Architecture supports deployment across large infrastructure networks — bridges, railways, buildings.

Known Constraints

System Limitations

- **Large dataset required** — CNN performance depends on diverse, well-labeled training data
- **GPU dependency** — Faster training requires GPU acceleration; CPU-only training is slow
- **Environment sensitivity** — May underperform on unseen lighting, textures, or surface types

Challenges Faced

- **Dataset Handling**
Organizing, augmenting, and balancing image classes
- **Training Time**
Iterative tuning required to optimize convergence
- **Model Tuning**
Hyperparameter selection for accuracy vs. overfitting
- **GUI Integration**
Connecting the trained model to a responsive Tkinter interface

Where It Applies — and What's Next

Current Applications

Building Inspection

Bridge Monitoring

Railway Track
Inspection

Manufacturing
Defects

Future Scope



Real-Time Camera Detection
Live feed inference for on-site inspection



Drone Integration
Aerial imagery for hard-to-reach structures



Crack Severity Classification
Multi-class output: hairline, moderate, severe



Crack Localization
Highlight detected crack regions on the image



CONCLUSION

Conclusion & References

What We Built

A fully functional CNN-based crack detection system with a Tkinter GUI, achieving **~90–95% accuracy** on a real-world surface crack dataset.

What It Proves

Deep learning is a practical, deployable solution for automated infrastructure inspection — reducing risk, cost, and human error.

References

- Kaggle Dataset — Surface Crack Detection
- TensorFlow & Keras Documentation
- Research papers on CNN-based defect detection

Thank You

Reproducibility

Code, dataset, and training scripts available for review

Collaboration

Open to partnerships in infrastructure AI and inspection tech

Next Steps

Real-time deployment and drone integration on the roadmap

